CAPSTONE PROJECT

Formation « Claude Code pour les Dev & DevOps » — application fil rouge ECommerce

Système multi-agents orchestré pour moderniser une application e-commerce

CodeScene · Migration PostgreSQL & Next.js · Tests · DevOps k3d & CI/CD · Monitoring Grafana

Orchestrateur et sous-agents Claude Code, packagés en plugin



Document	Cahier des charges & Rapport de définition du projet (RDP)
Livrable logiciel	Plugin Claude Code `ecom-modernizer` (marketplace `ecom-agents`)
Version / date	v1.0 — 21 octobre 2026
Auteur	Bassem Ben Hamed
Statut	Version de référence soumise à revue

Fiche de contrôle du document

Titre	Système multi-agents orchestré pour moderniser une application e-commerce — Cahier des charges & RDP
Projet	Capstone — application fil rouge ECommerce (.NET 10, Aspire 13)
Référence	CAP-ECOM-MAS-CDC-RDP
Version	1.0
Date	21 octobre 2026
Auteur	Bassem Ben Hamed
Diffusion	Équipe projet, encadrement, jury
Fichiers sources	`Capstone project/latex/` (LaTeX) et `Capstone project/ecom-agents-marketplace/` (plugin)

Historique des versions

Version	Date	Objet
0.9	14 octobre 2026	Cadrage : périmètre des cinq agents, choix de Claude Code et de MCP
1.0	21 octobre 2026	Version de référence : cahier des charges, RDP, squelette de plugin validé

Comment lire ce document

Le document comporte deux parties complémentaires :

- la **partie I — cahier des charges** décrit le *quoi* : contexte, objectifs mesurables, périmètre, exigences fonctionnelles et non fonctionnelles, contraintes ;
- la **partie II — rapport de définition du projet (RDP)** décrit le *comment* : architecture, rôle exact de l'orchestrateur, fiches des cinq sous-agents, serveurs MCP, hooks, skills et commandes, scénario de bout en bout, modèle d'exécution (one-shot ou boucle), packaging en plugin, qualité, sécurité et pilotage.

Conventions

- Les agents sont repérés par un badge : Orchestrateur CodeScene Migration Tests DevOps Monitoring ; l'opérateur humain par Humain.
- Les points de validation humaine (gates) sont notés G0 à G5.
- Les identifiants techniques sont écrits en `police-machine` ; les agents du plugin se nomment `ecom-modernizer:<agent>` et ses commandes `/ecom-modernizer:<commande>`.
- Les fichiers cités en annexe proviennent du squelette de référence livré avec ce document ; ils sont

inclus tels quels (source unique).

Table des matières

Fiche de contrôle du document i

Résumé exécutif x

I Cahier des charges 1

1 Contexte et analyse de l'existant 2

1.1 L'application fil rouge . . . 2

1.2 Diagnostic de l'existant . . . 3

1.3 Problématique . . . 5

2 Objectifs, périmètre et parties prenantes 6

2.1 Objectif général . . . 6

2.2 Objectifs spécifiques et indicateurs . . . 6

2.3 Périmètre . . . 7

2.4 Parties prenantes et rôles humains . . . 7

2.5 Hypothèses . . . 8

3 Exigences 9

3.1 Exigences fonctionnelles . . . 9

3.2 Exigences non fonctionnelles . . . 12

4 Contraintes et environnement 14

4.1 Contraintes techniques . . . 14

4.2 Environnement de référence . . . 14

4.3 Comptes, jetons et secrets . . . 15

4.4 Contraintes organisationnelles . . . 15

4.5 Contraintes de coûts . . . 15

II Rapport de définition du projet (RDP) 17

5 Architecture du système multi-agents 18

5.1 Vue d'ensemble . . . 18

5.2 Principes d'architecture . . . 19
5.3 Briques Claude Code mobilisées . . . 19
5.4 Architecture cible de l'application . . . 21
5.5 Organisation du dépôt cible . . . 22
5.6 Communication entre agents . . . 22

6 L'orchestrateur 24

6.1 Ce qu'est exactement l'orchestrateur . . . 24
6.2 Responsabilités . . . 24
6.3 Définition et activation . . . 25
6.4 Machine à états du run . . . 27
6.5 Plan d'exécution et parallélisme . . . 27
6.6 Contrat de tâche . . . 28
6.7 Quality gates . . . 28
6.8 Routage des échecs . . . 29
6.9 Gates humaines . . . 29

7 Les sous-agents 31

7.1 Choix des modèles et budgets . . . 31
7.2 Agent CodeScene — gardien de la santé du code . . . 31
7.2.1 Taxonomie et procédure . . . 32
7.2.2 Le cas du code Next.js généré . . . 33
7.2.3 Le référentiel Code Health du projet (sans clé d'API) . . . 33
7.3 Agent Migration — architecte de modernisation . . . 35
7.3.1 Tâche 1 — persistance PostgreSQL . . . 36
7.3.2 Tâche 2 — refonte Blazor vers Next.js . . . 37
7.4 Agent Tests — ingénieur qualité . . . 37
7.4.1 Playwright : le MCP pour explorer, les tests pour prouver . . . 38
7.4.2 Tri des échecs . . . 38
7.5 Agent DevOps — ingénieur plateforme . . . 39
7.5.1 Cluster et déploiement . . . 39
7.5.2 Ouverture dans le navigateur . . . 40
7.5.3 Chaîne CI/CD . . . 40
7.6 Agent Monitoring — ingénieur observabilité (SRE) . . . 41

8 Serveurs MCP 43

8.1 Le protocole MCP en bref . . . 43
8.2 Intégration dans Claude Code et dans le plugin . . . 43
8.3 Catalogue des serveurs . . . 44
8.4 Matrice agents × serveurs . . . 47
8.5 Configuration du plugin . . . 47
8.6 Sécurité des serveurs MCP . . . 48

9 Hooks, skills, commandes et utilitaires 49
9.1 Hooks : la couche de garanties . . . 49
9.1.1 Contrat d'entrée et de sortie . . . 49
9.1.2 Catalogue des hooks du plugin . . . 49
9.1.3 Cycle de vie d'une délégation . . . 51
9.1.4 Vérification des hooks . . . 51
9.1.5 Variantes de hooks . . . 52
9.2 Skills : les savoir-faire . . . 52
9.3 Commandes : les points d'entrée . . . 54
9.4 Utilitaires, workflows et monitor . . . 56

10 Scénario de référence et déroulement détaillé 57
10.1 Vue d'ensemble du scénario S1 « modernisation complète » . . . 57
10.2 Déroulement phase par phase . . . 58
10.3 Qui fait quoi et à quel moment . . . 60
10.4 Scénarios alternatifs . . . 60

11 Modèle d'exécution : one-shot ou boucle ? 62
11.1 Quatre niveaux de boucles . . . 62
11.2 Mode d'exécution par agent . . . 63
11.3 Mécanismes de la plateforme . . . 63
11.4 Garde-fous contre l'emballement . . . 64
11.5 Pourquoi ne pas tout mettre en boucle . . . 64

12 Packaging en plugin et utilisation 65
12.1 Structure du plugin . . . 65
12.2 Manifeste et configuration utilisateur . . . 65
12.3 Marketplace et distribution . . . 66
12.4 Installation . . . 67

12.5 Modes d'utilisation 67
12.6 Activer tous les sous-agents ou un sous-ensemble 68
12.7 Variantes de packaging 70
12.8 Cycle de vie du plugin 70

13 Qualité du système, sécurité et risques 71
13.1 Évaluer les agents eux-mêmes 71
13.1.1 Évaluations du plugin 71
13.1.2 Indicateurs AgentOps 72
13.2 Sécurité 72
13.3 Risques et parades 73

14 Pilotage du projet 75
14.1 Organisation 75
14.2 Lots de travail et planning 75
14.3 Jalons et revues 76
14.4 Livrables 76
14.5 Critères d'acceptation et grille d'évaluation 77
14.6 Scénario de démonstration (20 minutes) 77

A Matrice de traçabilité 78

B Glossaire 79

C Fichiers de référence du plugin 81
C.1 Hooks 81
C.2 Agent orchestrateur (prompt complet) 85
C.3 Quality gates et état 86
C.4 Workflow déterministe 89
C.5 Référentiel Code Health local 90

D Sources et références 94

Table des figures

1.1 Architecture actuelle (« as-is ») de l'application ECommerce 2

5.1 Architecture en couches du système multi-agents `ecom-modernizer` 18
5.2 Architecture cible de l'application modernisée, déployée sur k3d 21

6.1 Machine à états d'un run (statuts persistés dans `.ecom/state.json`) 27
6.2 DAG des phases du profil full, gates humaines et boucles de rétroaction 27

7.1 Chaîne d'analyse du référentiel Code Health local 34
7.2 Pyramide de tests générée par l'agent Tests 38
7.3 Pipeline CI/CD généré par l'agent DevOps 40

8.1 Architecture MCP : un hôte, un client par serveur, des serveurs locaux ou distants 43

9.1 Hooks traversés lors d'une délégation de l'orchestrateur à un sous-agent 51

10.1 Chronogramme du scénario S1 : couloirs par agent, phases P0 à P7, gates et rétroaction . 57
10.2 Séquence d'une boucle de rétroaction : test en échec, correctif, nouvelle exécution 59

11.1 Les quatre niveaux de boucles du système 62

12.1 Cinq niveaux d'activation des sous-agents 69

14.1 Planning indicatif sur huit semaines 75

Liste des tableaux

1 Les six agents en un coup d'œil x

1.1 Projets de la solution `ECommerce.slnx` 3
1.2 Contrat de l'API REST exposée par la gateway 3
1.3 Bloqueurs identifiés dans le code existant 4

2.1 Objectifs mesurables du projet 6
2.2 Périmètre du projet 7
2.3 Rôles humains 7

3.1 Exigences fonctionnelles 9
3.2 Exigences non fonctionnelles 12

4.1 Versions de référence 14
4.2 Secrets nécessaires et mode de stockage 15

5.1 Principes d'architecture et leur mise en œuvre 19
5.2 Briques de la plateforme et leur rôle dans le système 20
5.3 Artefacts partagés du répertoire .ecom 23

6.1 Quatre manières d'orchestrer et leur place dans le projet 24
6.2 Responsabilités de l'orchestrateur 25
6.3 Critères des quality gates par phase 29
6.4 Table de routage des échecs 29
6.5 Gates humaines 30

7.1 Modèle, effort et budget par agent 31
7.2 Fiche signalétique de l'agent CodeScene 32
7.3 Modèle de notation du référentiel (extrait de rules.json) 34
7.4 Fiche signalétique de l'agent Migration 35
7.5 Matrice de parité Blazor → Next.js 37
7.6 Fiche signalétique de l'agent Tests 37
7.7 Fiche signalétique de l'agent DevOps 39
7.8 Fiche signalétique de l'agent Monitoring 41
7.9 Tableaux de bord, SLO et alertes créés par l'agent Monitoring 42

8.1 Accès des agents aux serveurs MCP (✎ écriture, 👁 lecture, — aucun) 47

9.1 Contrat d'un hook de type commande 49
9.2 Les 16 hooks du plugin `ecom-modernizer` 50
9.3 Cas de test des hooks et résultats observés 51
9.4 Catalogue des 18 skills 53
9.5 Catalogue des 17 commandes 54
9.6 Utilitaires du dossier bin (dans le PATH des agents) 56

10.1 Phases du scénario S1 57
10.2 Matrice RACI (R réalise, A approuve, C consulté, I informé) 60
10.3 Scénarios alternatifs 61

11.1 Mode d'exécution de chaque agent 63
11.2 Mécanismes de boucle disponibles et leur usage 63

12.1 Modes d'utilisation du plugin 67
12.2 Profils prédéfinis 69

13.1 Cas d'évaluation du squelette 71
13.2 Indicateurs de fonctionnement du système multi-agents 72
13.3 Menaces propres au système multi-agents et mesures 73
13.4 Registre des risques 73

14.1 Jalons 76
14.2 Livrables du projet 76
14.3 Grille d'évaluation 77

A.1 Traçabilité exigences → agents → vérification → livrables 78

Résumé exécutif

Le problème. L'application fil rouge *ECommerce* (.NET 10, Aspire 13, microservices Catalog et Ordering derrière une gateway YARP, interface Blazor) est volontairement livrée « nue » : base de données en mémoire, aucun test, aucun conteneur, aucun déploiement, aucune supervision. La moderniser jusqu'à un niveau « production » mobilise cinq expertises distinctes — qualité de code, migration de données et de frontend, tests, plateforme, observabilité — qu'il faut coordonner, contrôler et rendre reproductibles.

La solution. Un **système multi-agents orchestré**, construit sur Claude Code et packagé en un **plugin** unique, `ecom-modernizer`. Un **orchestrateur**, exécuté comme thread principal de la session, planifie le travail, délègue chaque phase au sous-agent propriétaire, évalue des *quality gates* déterministes et sollicite l'humain aux points irréversibles. Cinq **sous-agents** spécialisés disposent chacun de leurs skills, de leurs commandes et de leurs outils, connectés par le **protocole MCP** à onze serveurs (référentiel Code Health local, PostgreSQL, Next.js, Playwright, Kubernetes, GitHub, Grafana, Context7). Une couche de **hooks** garantit les règles qui ne doivent jamais dépendre du bon vouloir d'un modèle : périmètre d'écriture par agent, verrouillage du contexte Kubernetes, rapport obligatoire, boucle d'orchestration bornée.

Table 1 – Les six agents en un coup d'œil

Agent	Rôle reformulé	Serveurs MCP principaux
Orchestrateur	Chef de projet technique : planifie, délègue, contrôle les gates, arbitre, consolide. Ne code jamais.	grafana-ro, kubernetes-ro
CodeScene	Gardien de la santé du code : génère le référentiel Code Health local (sans clé d'API), audite et corrige les bloqueurs (.NET puis Next.js généré), garde anti-régression.	code-health (local), next-devtools
Migration	Architecte de modernisation : PostgreSQL via Aspire et EF Core, refonte Blazor → Next.js.	postgres-admin, next-devtools
Tests	Ingénieur qualité : tests unitaires, d'intégration et E2E, tri des échecs, rapport.	playwright, postgres-ro
DevOps	Ingénieur plateforme : images, cluster k3d, Helm, ouverture dans le navigateur, CI/CD.	kubernetes, github, playwright
Monitoring	Ingénieur SRE : stack Prometheus/Loki/Tempo/Grafana, tableaux de bord, SLO, alertes, tri d'incident.	grafana, kubernetes-ro

Décisions structurantes.

1. **Orchestration hybride** : un orchestrateur agentique (flexible, capable de dialoguer avec l'humain) adossé à une machine à états persistante, à des scripts de gates et, pour les phases massivement parallèles, à des *workflows* Claude Code déterministes.
2. **Moindre privilège par construction** : instances MCP séparées en lecture seule et en écriture, listes d'outils par agent, hooks de périmètre ; aucun sous-agent ne peut modifier ses propres garde-fous.

3. **One-shot à l'extérieur, boucles bornées à l'intérieur** : chaque délégation est un appel à contrat (entrées, critères, rapport), qui exécute en interne des boucles de convergence plafonnées ; seule la supervision tourne en continu.
4. **Activation à cinq niveaux** : installation du plugin, configuration utilisateur, profil projet, arguments de lancement, verrou dur par règle de permission et par hook.
5. **Analyse de code sans clé d'API** : la santé du code est mesurée par un **référentiel Code Health local**, généré dans le projet (règles versionnées, moteur open source, serveur MCP local), plutôt que par un service SaaS soumis à licence.

Résultats attendus. Zéro bloqueur critique résiduel et une Code Health ≥ 9 sur le code modifié ; persistance PostgreSQL avec migrations versionnées ; frontend Next.js iso-fonctionnel ; taux de réussite des tests ≥ 98 % et couverture ≥ 80 % (backend) ; déploiement k3d reproductible en moins de 10 minutes et chaîne CI/CD verte ; cinq tableaux de bord, trois SLO et six règles d'alerte au minimum ; un plugin validé (`claude plugin validate --strict`) et évalué (`claude plugin eval`).

Livrables. Ce document (LaTeX et PDF), le squelette de référence du plugin (6 agents, 17 commandes, 18 skills, 16 scripts de hooks, 3 utilitaires, 11 serveurs MCP, le modèle du référentiel Code Health local, 2 workflows, des cas d'évaluation), puis, au terme du projet, l'application modernisée, ses rapports générés, ses pipelines et ses tableaux de bord.

Première partie

Cahier des charges

1 Contexte et analyse de l'existant

1.1 L'application fil rouge

ECommerce est l'application support de la formation « Claude Code pour les Dev & DevOps ». Elle offre un catalogue de produits, un panier, la prise de commande et l'historique des commandes, via une interface web et une API REST. Elle est construite en **.NET 10 / ASP.NET Core** et orchestrée localement par **.NET Aspire 13** (SDK `Aspire.AppHost.Sdk/13.4.3`). Elle est livrée *volontairement nue* : pas de base réelle, pas de tests, pas de conteneurs, pas de manifestes, pas de CI. C'est précisément le terrain de jeu du système multi-agents.

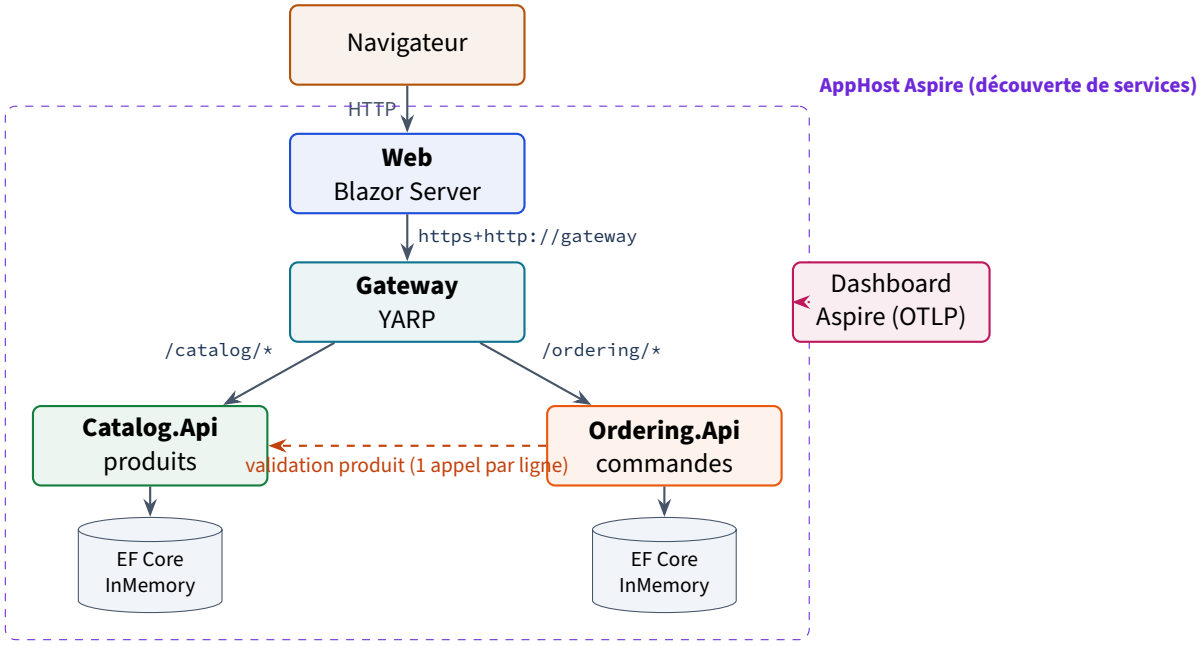


FIGURE 1.1 – Architecture actuelle (« as-is ») de l'application ECommerce

TABLE 1.1 – Projets de la solution `ECommerce.slnx`

Projet	Rôle
`ECommerce.AppHost`	Orchestrateur Aspire : déclare les ressources `catalog`, `ordering`, `gateway`, `web` et leurs dépendances (`WithReference`, `WaitFor`).
`ECommerce.Web`	Interface Blazor Server : pages `/` (accueil), `/products` (catalogue, panier, commande), `/orders` (historique).
`ECommerce.Gateway`	Reverse proxy YARP : routes `/catalog/{**catch-all}` et `/ordering/{**catch-all}`.
`ECommerce.Catalog.Api`	Minimal API produits, `CatalogDbContext` EF Core InMemory, 4 produits de démonstration.
`ECommerce.Ordering.Api`	Minimal API commandes ; valide chaque produit auprès de Catalog par HTTP synchrone.
`ECommerce.ServiceDefaults`	Socle commun : OpenTelemetry (OTLP si `OTEL_EXPORTER_OTLP_ENDPOINT`), santé, résilience, découverte de services.

TABLE 1.2 – Contrat de l'API REST exposée par la gateway

Méthode	Route	Comportement
GET	`/catalog/api/products`	Liste des produits
GET	`/catalog/api/products/{id}`	Un produit, 404 si absent
POST	`/catalog/api/products`	Création d'un produit (201)
GET	`/ordering/api/orders`	Liste des commandes avec leurs lignes
GET	`/ordering/api/orders/{id}`	Une commande, 404 si absente
POST	`/ordering/api/orders`	Création ; chaque produit est validé auprès de Catalog (400 si inconnu)

L'ensemble représente environ 560 lignes de C# et 360 lignes de Razor : une base volontairement compacte, qui permet de vérifier finement le travail de chaque agent.

1.2 Diagnostic de l'existant

Une revue préalable du code (qui sera refaite et outillée par l'agent **CodeScene**) fait déjà apparaître des **bloqueurs** réels. Ils serviront de jeu de référence pour valider le système : chaque bloqueur doit être détecté, classé, attribué au bon agent, puis corrigé ou arbitré.

TABLE 1.3 – Bloqueurs identifiés dans le code existant

Id	Localisation	Constat et impact	Sév.	Propriétaire
B-01	`ServiceDefaults/Extensions.cs:113`	`/health` et `/alive` ne sont exposés qu'en Development : en production les sondes Kubernetes échouent, les pods ne deviennent jamais prêts.	B0	CodeScene (code), DevOps (sondes)
B-02	`Catalog.Api/Program.cs:12,30` et `Ordering.Api/Program.cs:13,39`	`UseInMemoryDatabase` et `EnsureCreated()` : aucune persistance, incompatibilité avec les migrations EF Core.	B1	Migration
B-03	`CatalogDbContext.cs:21`	`HasData` avec identifiants explicites : sous PostgreSQL la séquence d'identité n'avance pas, d'où une violation de clé au premier `POST`.	B1	Migration
B-04	`OrderingEndpoints.cs:30`	`request.Items.Count` sans contrôle de nullité : une commande sans `items` provoque une `NullReferenceException` (HTTP 500).	B1	CodeScene
B-05	`OrderingEndpoints.cs:43--56`	Quantité non validée (zéro ou négative acceptée) et stock jamais décrémenté : incohérence métier.	B1	Humain G1
B-06	`OrderingEndpoints.cs:45`	Un appel HTTP synchrone vers Catalog par ligne (N+1) : latence proportionnelle à la taille du panier.	B2	CodeScene
B-07	`CatalogEndpoints.cs:23--29`	`CreateProductRequest` non validé (nom vide, prix négatif).	B2	CodeScene
B-08	`Catalog.Api/Program.cs:21` et `Ordering.Api/Program.cs:31`	OpenAPI exposé seulement en Development : pas de contrat exploitable pour générer le client TypeScript du futur frontend.	B1	Migration
B-09	`Web/Program.cs:29,32`	HSTS et redirection HTTPS derrière un ingress qui termine TLS : risque de boucle de redirection.	B2	CodeScene / DevOps
B-10	Dépôt	Ni tests, ni Dockerfile, ni manifestes, ni CI, ni supervision (choix pédagogique).	B1	Tests DevOps Monitoring

⚠ Un bloqueur n'est pas toujours un bug à corriger

B-05 modifie un comportement métier (réserver le stock au moment de la commande). Aucun agent ne doit trancher seul : la correction est soumise à la gate G1. Ce cas illustre la frontière entre l'automatisation et la décision humaine que le système doit respecter.

1.3 Problématique

Moderniser cette application demande d'enchaîner des travaux interdépendants : on ne déploie pas avant d'avoir testé, on ne teste pas un frontend qui n'existe pas encore, on ne supervise pas une application non déployée. Confier tout cela à un seul assistant conversationnel produit trois écueils bien connus :

- **saturation du contexte** : un seul fil accumule des milliers de lignes de logs, de diffs et de sorties d'outils, et perd en pertinence ;
- **absence de séparation des responsabilités** : le même agent écrit le code et le juge, modifie l'infrastructure et la supervise, sans contre-pouvoir ;
- **non-reproductibilité** : rien ne garantit qu'un second passage suive le même chemin, ni qu'une règle de sécurité soit respectée à chaque fois.

La question posée au projet est donc : *comment organiser plusieurs agents spécialisés, outillés par des protocoles standard, sous la conduite d'un orchestrateur, pour obtenir une modernisation contrôlée, traçable, reproductible et réutilisable ?*

2 Objectifs, périmètre et parties prenantes

2.1 Objectif général

Concevoir, réaliser et démontrer un **système multi-agents** capable de faire passer l'application ECommerce de son état initial à un état *modernisé, testé, déployé et supervisé*, sous la conduite d'un **agent orchestrateur**, avec des sous-agents spécialisés outillés par le protocole MCP, le tout **packagé en un plugin Claude Code** réutilisable et activable en totalité ou en partie.

2.2 Objectifs spécifiques et indicateurs

Table 2.1 – Objectifs mesurables du projet

Id	Objectif	Indicateur et cible	Vérifié par
O1	Assainir le code existant et le code généré	Référentiel Code Health local généré dans le projet, sans clé d'API ; 0 bloqueur B0/B1 ouvert ; Code Health moyenne $\geq$ 9,0 sur les fichiers modifiés ou créés ; aucune régression de score	CodeScene, gate audit/review
O2	Persister les données dans PostgreSQL	2 bases (`catalogdb`, `orderingdb`), migrations EF Core versionnées, application depuis une base vide, parité des 6 endpoints = 100 %	Migration, G2
O3	Refondre le frontend en Next.js	Parité fonctionnelle 100 % (catalogue, panier, commande, historique, 404) ; 0 erreur ESLint et TypeScript ; aucune violation d'accessibilité sérieuse	Migration, G3
O4	Prouver le bon fonctionnement	Taux de réussite $\geq$ 98 % ; couverture $\geq$ 80 % (backend) et $\geq$ 70 % (frontend) ; 7 parcours E2E critiques verts sur 3 exécutions ; rapport publié	Tests
O5	Déployer de façon reproductible	Cluster k3d et application déployés en moins de 10 minutes depuis un poste vierge ; pods prêts à 100 % ; `http://shop.localhost:8080` répond 200 ; pipeline CI vert	DevOps, G4
O6	Rendre l'application observable	5 tableaux de bord, 3 SLO, $\geq$ 6 règles d'alerte ; game day concluant (alerte déclenchée puis résolue)	Monitoring

Id	Objectif	Indicateur et cible	Vérifié par
O7	Industrialiser le système multi-agents	Plugin validé en mode strict ; activation totale et partielle démontrées ; évaluations du plugin $\geq$ 0,8 ; run complet traçable (journal, rapports, tags git)	Orchestrateur, jury

2.3 Périmètre

Table 2.2 – Périmètre du projet

Inclus	Exclus
• Orchestrateur et cinq sous-agents Claude Code • Skills, commandes, hooks, serveurs MCP, workflows • Référentiel Code Health local (règles, moteur, serveur MCP), sans clé d'API • Migration EF Core InMemory $\rightarrow$ PostgreSQL 17 • Refonte Blazor $\rightarrow$ Next.js 16 (App Router) • Tests unitaires, intégration, système, E2E • Images, cluster k3d local, chart Helm, CI/CD GitHub Actions • Stack Prometheus, Loki, Tempo, Alloy, Grafana ; SLO et alertes • Packaging en plugin et marketplace ; évaluations	• Paiement, authentification et gestion des clients • Déploiement sur un cloud public ou en production réelle • Migration de données réelles (seules des données de démonstration existent) • Haute disponibilité de PostgreSQL (option : opérateur CloudNativePG) • Refonte fonctionnelle (nouvelles fonctionnalités métier) • Recours à un service d'analyse SaaS soumis à clé ou licence (CodeScene Cloud) • Serveurs MCP tiers modifiés (seul le serveur du référentiel Code Health est développé)

2.4 Parties prenantes et rôles humains

Le système automatise l'exécution, pas la responsabilité. Trois rôles humains interviennent.

Table 2.3 – Rôles humains

Rôle	Responsabilités	Interventions
Product owner / encadrant	Fixe les objectifs et les seuils, arbitre les changements fonctionnels, réceptionne les livrables.	G1 G3, recette finale
Opérateur (tech lead)	Lance et pilote les runs, configure le plugin, valide les actions irréversibles, traite les escalades.	G0 G2 G4 G5
Équipe projet	Conçoit et met au point les agents, skills, hooks et serveurs ; analyse les rapports et les journaux.	Revue des rapports, évaluations
Jury	Évalue l'architecture, la démonstration de bout en bout et la qualité des livrables.	Soutenance

2.5 Hypothèses

- Le poste de travail dispose de Docker et d'au moins 16 Go de mémoire (8 Go avec le profil d'observabilité léger).
- L'équipe dispose d'un abonnement Claude (ou d'une clé API) et d'un compte GitHub ; aucune licence ni clé CodeScene n'est nécessaire (référentiel Code Health local).
- Le dépôt est versionné avec git ; si ce n'est pas le cas, `/ecom-modernizer:init` l'initialise (tag `ecom/baseline`).
- Les noms `*.localhost` se résolvent vers la boucle locale (comportement standard des navigateurs et de macOS/Linux).

3 Exigences

Les exigences sont priorisées selon la méthode MoSCoW : **M** (indispensable), **S** (souhaitable), **C** (optionnel). Chaque exigence porte un critère d'acceptation vérifiable ; la matrice de traçabilité de l'annexe A relie exigences, agents et gates.

3.1 Exigences fonctionnelles

TABLE 3.1 – Exigences fonctionnelles

Id	Exigence	P.	Critère d'acceptation
Orchestrateur Orchestrateur			
EF-ORC-01	Résoudre la sélection d'agents et de phases à partir des arguments, du profil projet et de la configuration utilisateur.	M	Plan affiché conforme à la sélection ; agents exclus signalés.
EF-ORC-02	Vérifier les prérequis avant tout run.	M	`ecom-doctor` exécuté ; run bloqué si un prérequis critique manque.
EF-ORC-03	Construire et faire valider le plan (DAG) par l'humain.	M	Gate GO enregistrée dans l'état.
EF-ORC-04	Déléguer chaque phase au sous-agent propriétaire avec un contrat de tâche.	M	Contrat conforme au modèle (première ligne `phase:`).
EF-ORC-05	Paralléliser les phases indépendantes.	S	`migrate-db` et `migrate-frontend` lancées simultanément.
EF-ORC-06	Évaluer une quality gate après chaque phase.	M	`ecom-gate` exécuté ; décision tracée.
EF-ORC-07	Router les échecs vers le bon agent, dans la limite d'un plafond de boucles.	M	Au plus 3 rétroactions par phase ; escalade ensuite.
EF-ORC-08	Persister l'état et permettre la reprise.	M	`/ecom-modernizer:resume` reprend après interruption.
EF-ORC-09	Produire un rapport final consolidé.	M	`docs/reports/<run_id>-final.md` présent et complet.
Agent CodeScene CodeScene			
EF-CS-00	Générer dans le projet le référentiel Code Health local (règles, moteur, serveur MCP, baseline), sans clé d'API ni service externe.	M	`quality/code-health/` versionné ; analyse et serveur MCP fonctionnels hors ligne.
EF-CS-01	Scanner tout le code .NET et Razor (passe audit).	M	Score de Code Health pour 100 % des fichiers sources.

Id	Exigence	P.	Critère d'acceptation
EF-CS-02	Classer les bloqueurs (sévérité, catégorie, propriétaire).	M	Tableau B0–B3 dans le rapport ; B-01 à B-10 retrouvés.
EF-CS-03	Corriger les bloqueurs de son périmètre par petites étapes vérifiées.	M	Build vert après chaque correction ; score non dégradé.
EF-CS-04	Transférer les bloqueurs relevant d'un autre agent.	M	Handoff `codescene-to-migration.json` produit.
EF-CS-05	Revoir le code Next.js généré et le code modifié (passe review).	M	`analyze_change_set` et `get_errors` exécutés ; 0 B0/B1.
EF-CS-06	Empêcher les régressions en continu (hooks) et avant commit.	S	Commit refusé si le moteur local (`safeguard`) détecte une régression.
EF-CS-07	Faire évoluer le référentiel (nouvelle règle de bloqueur, seuil) sous contrôle humain.	S	Modification de `rules.json` ou `waivers.json` validée par G1.
Agent Migration Migration			
EF-MIG-01	Introduire PostgreSQL via les intégrations Aspire.	M	Ressource `postgres`, bases `catalogdb` et `orderingdb`.
EF-MIG-02	Remplacer EF Core InMemory par Npgsql et générer les migrations.	M	Migrations `InitialCreate` par contexte ; `EnsureCreated` supprimé.
EF-MIG-03	Appliquer les migrations par un service dédié.	M	`ECommerce.MigrationService` ; API en `WaitForCompletion`.
EF-MIG-04	Corriger le seeding pour PostgreSQL.	M	Premier `POST /catalog/api/products` sans violation de clé.
EF-MIG-05	Générer le contrat OpenAPI et les types TypeScript.	M	Fichiers OpenAPI produits au build ; types générés.
EF-MIG-06	Créer l'application Next.js iso-fonctionnelle.	M	Matrice de parité à 100 %.
EF-MIG-07	Intégrer le storefront à l'AppHost.	M	Ressource `storefront` démarrée par Aspire.
EF-MIG-08	Retirer le projet Blazor après validation.	S	Suppression postérieure à G3.
Agent Tests Tests			
EF-TST-01	Générer les tests unitaires backend et frontend.	M	Projets de tests compilés ; cas minimaux du skill couverts.
EF-TST-02	Générer les tests d'intégration sur une vraie base.	M	Testcontainers PostgreSQL ; 6 endpoints couverts.
EF-TST-03	Générer et exécuter les parcours E2E avec Playwright.	M	7 parcours critiques ; exploration préalable via le MCP Playwright.

Id	Exigence	P.	Critère d'acceptation
EF-TST-04	Trier les échecs et mesurer l'instabilité.	M	Chaque échec classé (test, application, environnement, instable).
EF-TST-05	Réparer ses propres tests sans toucher au code applicatif.	M	Aucune écriture hors périmètre (hook).
EF-TST-06	Rédiger le rapport de tests.	M	Rapport Markdown + JUnit + couverture + rapport Playwright.
EF-TST-07	Rejouer les parcours critiques sur le cluster.	S	Smoke test sur `http://shop.localhost:8080`.
Agent DevOps DevOps			
EF-OPS-01	Construire les images de tous les services.	M	Images taguées par SHA dans le registre local.
EF-OPS-02	Créer (ou réutiliser) le cluster k3d.	M	Création idempotente ; contexte `k3d-ecommerce`.
EF-OPS-03	Déployer l'application via Helm, migrations incluses.	M	Release `ecommerce` ; job de migration réussi.
EF-OPS-04	Vérifier le déploiement et diagnostiquer les échecs.	M	Pods prêts ; diagnostic à partir des événements et logs.
EF-OPS-05	Ouvrir l'application dans le navigateur.	M	Navigateur ouvert ; capture d'écran dans le rapport.
EF-OPS-06	Mettre en place la CI.	M	`ci.yml` : build, tests, qualité, images, E2E sur k3d éphémère.
EF-OPS-07	Mettre en place la CD.	S	`cd.yml` : images GHCR, valeurs Helm, environnement protégé.
EF-OPS-08	Générer le chart depuis le modèle Aspire.	C	`aspire publish` avec `AddKubernetesEnvironment`.
Agent Monitoring Monitoring			
EF-MON-01	Installer la stack d'observabilité sur le cluster.	M	Prometheus, Loki, Tempo, Alloy, Grafana opérationnels.
EF-MON-02	Brancher la télémétrie des services et du storefront.	M	Métriques, logs et traces visibles par service.
EF-MON-03	Ajouter des métriques métier.	S	Compteurs de commandes et valeur des commandes exposés.
EF-MON-04	Créer les tableaux de bord via le MCP Grafana.	M	5 tableaux de bord dans le dossier « ECommerce ».
EF-MON-05	Définir SLO et alertes.	M	3 SLO, $\geq$ 6 règles d'alerte actives.
EF-MON-06	Surveiller périodiquement et trier les incidents.	M	`/loop 15m /ecom-modernizer:monitor-check` ; rapport d'incident.
EF-MON-07	Superviser le système multi-agents lui-même (AgentOps).	S	Tableau de bord tokens, coûts, échecs d'outils.

Id	Exigence	P.	Critère d'acceptation
Plugin `ecom-modernizer`			
EF-PLG-01	Regrouper agents, skills, commandes, hooks, MCP et workflows dans un plugin.	M	`claude plugin validate --strict` réussi.
EF-PLG-02	Distribuer le plugin par une marketplace.	M	Installation `ecom-modernizer@ecom-agents`.
EF-PLG-03	Demander les secrets à l'activation, sans les stocker en clair.	M	`userConfig` sensible (trousseau).
EF-PLG-04	Activer tous les sous-agents ou un sous-ensemble.	M	Profils et `--agents` ; refus motivé d'un agent désactivé.
EF-PLG-05	Proposer un mode non interactif pour la CI.	S	Profil `ci` ; gates en échec fermé.
EF-PLG-06	Fournir des évaluations automatisées du plugin.	S	`claude plugin eval` $\geq$ 0,8.

3.2 Exigences non fonctionnelles

Table 3.2 – Exigences non fonctionnelles

Id	Domaine	Exigence	P.
ENF-SEC-01	Sécurité	Moindre privilège : chaque agent ne dispose que des outils et serveurs MCP nécessaires ; instances MCP en lecture seule dès que possible.	M
ENF-SEC-02	Sécurité	Aucun secret dans le dépôt, les prompts ou les journaux ; secrets fournis par `userConfig` sensible ou Secrets Kubernetes.	M
ENF-SEC-03	Sécurité	Toute action irréversible (suppression de cluster, de namespace, de volume, push, fusion) exige une confirmation humaine.	M
ENF-SEC-04	Sécurité	Un agent ne peut modifier ni sa configuration, ni les hooks, ni l'état du run en dehors de l'utilitaire prévu.	M
ENF-IND-01	Indépendance	Aucune clé d'API ni service SaaS pour l'analyse de code : règles, moteur et serveur MCP exécutables hors ligne dans le projet.	M
ENF-FIA-01	Fiabilité	Les étapes sont idempotentes (cluster, déploiement, migrations, tableaux de bord).	M
ENF-FIA-02	Fiabilité	Toute boucle est bornée (itérations, absence de progrès, budget).	M
ENF-FIA-03	Fiabilité	Un run interrompu (fin de session, compaction) est repris sans perte.	M
ENF-TRA-01	Traçabilité	Chaque action significative est journalisée (`.ecom/journal.jsonl`) avec horodatage, agent et session.	M
ENF-TRA-02	Traçabilité	Chaque phase terminée est marquée par un tag git `ecom/<run_id>/<phase>`.	S

Id	Domaine	Exigence	P.
ENF-REP-01	Reproductibilité	Versions des outils et des serveurs MCP figées en CI ; configuration versionnée.	S
ENF-REP-02	Reproductibilité	Les phases massivement parallèles sont exécutables par workflow déterministe.	C
ENF-COU-01	Coûts	Modèle choisi par agent selon la difficulté (Opus pour orchestrer et migrer, Sonnet pour le reste) ; plafond `maxTurns` par agent.	S
ENF-PERF-01	Performance	Un run complet (profil full) s'exécute en moins de 4 heures sur le poste de référence, hors temps d'attente humaine.	S
ENF-MAI-01	Maintenabilité	Un agent = une responsabilité ; connaissances dans des skills chargés à la demande (fichier SKILL.md < 500 lignes).	M
ENF-POR-01	Portabilité	macOS et Linux ; Windows via WSL2 ; scripts compatibles bash 3.2.	S
ENF-OBS-01	Observabilité	Le système multi-agents est lui-même observable (télémétrie OpenTelemetry de Claude Code, journal).	S
ENF-ERG-01	Ergonomie	Messages et rapports en français ; une commande par intention ; état consultable à tout moment.	M

4 Contraintes et environnement

4.1 Contraintes techniques

- **Plateforme agentique** : Claude Code (CLI, extension VS Code ou application de bureau), version 2.1.300 ou ultérieure pour disposer des sous-agents imbriqués, des workflows, des hooks `mcp_tool`/`prompt`, du `userConfig` à options et de `claude plugin eval`.
- **Protocole d'outillage** : Model Context Protocol (révision courante 2026-07-28) ; serveurs existants réutilisés, lancés en `stdio` ou joints en Streamable HTTP.
- **Application** : .NET 10, ASP.NET Core Minimal API, Aspire 13.x, EF Core 10, YARP.
- **Cibles de modernisation** : PostgreSQL 17, Next.js 16 (App Router, TypeScript strict), Playwright.
- **Plateforme d'exécution** : Docker, k3d 5.x (k3s, Traefik), Helm $\geq$ 4.2 (exigé par la publication Kubernetes d'Aspire), GitHub Actions et GHCR.
- **Analyse de code** : référentiel Code Health local (Python 3.10+, lizard via `uv`, SDK MCP Python 2.x), sans clé d'API ni service externe.
- **Observabilité** : OpenTelemetry (déjà présent dans `ServiceDefaults`), Prometheus, Loki, Tempo, Grafana Alloy, Grafana.

4.2 Environnement de référence

Les versions ci-dessous sont celles relevées par `ecom-doctor` sur le poste de référence (macOS, Apple Silicon) au moment de la rédaction.

Table 4.1 – Versions de référence

Outil	Version	Usage
Claude Code	2.1.304	Plateforme agentique, plugin, hooks, workflows
.NET SDK	10.0.301	Build, tests, publication des conteneurs
Node.js / npm	24.21 / 11.19	Next.js, Vitest, Playwright, serveurs MCP via `npx`
uv (`uvx`)	0.9.30	Serveurs MCP Python (`mcp-grafana`, `postgres-mcp`)
Docker	29.8	Conteneurs, Testcontainers, k3d
k3d / kubectl	5.9.0 / 1.37.1	Cluster local, administration
Helm	4.2.3	Chart applicatif, stack d'observabilité
git / gh / jq	2.55 / 2.96 / 1.7.1	Versionnement, GitHub, scripts des hooks
lizard / SDK MCP Python	1.24.1 / 2.3.0	Moteur et serveur MCP du référentiel Code Health local (installés à la volée par `uv`)

Matériel recommandé : 4 cœurs, 16 Go de mémoire, 30 Go de disque libre. Avec 8 Go, utiliser le profil d'observabilité léger (`grafana/otel-lgtm`) et un cluster à un seul agent.

4.3 Comptes, jetons et secrets

TABLE 4.2 – Secrets nécessaires et mode de stockage

Secret	Usage	Stockage
Abonnement Claude ou clé API	Exécution des agents ; en CI : `CLAUDE_CODE_OAUTH_TOKEN` ou `ANTHROPIC_API_KEY`	Session locale ; secret GitHub Actions
`github_token`	Serveur MCP GitHub (dépôts, actions, pull requests)	`userConfig` sensible
`postgres_admin_uri`	MCP `postgres-admin` (développement uniquement)	`userConfig` sensible
`postgres_ro_uri`	MCP `postgres-ro` (compte en lecture seule)	`userConfig` sensible
Jeton de compte de service Grafana	MCP Grafana	Fichier `grafana-sa.token` dans `${CLAUDE_PLUGIN_DATA}`, créé par l'agent Monitoring
Mot de passe PostgreSQL du cluster	Application déployée	Secret Kubernetes généré par le chart

4.4 Contraintes organisationnelles

- **Données** : uniquement les produits de démonstration et des données synthétiques ; aucune donnée personnelle réelle.
- **Licences** : l'analyse de code n'utilise ni service CodeScene ni clé d'API ; le référentiel local repose sur lizard (licence MIT) et le SDK MCP Python (licence MIT). Grafana, Loki et Tempo sont utilisés localement (licence AGPLv3 sans redistribution).
- **Validation humaine** : les gates G0 à G5 ne peuvent pas être contournées en mode interactif ; en CI, seules celles listées dans `auto_approve_in_ci` sont automatiques, les autres mettent le run en pause.

4.5 Contraintes de coûts

Le coût d'un système multi-agents croît avec le nombre d'agents actifs et la taille de leurs contextes. Le projet impose donc :

- un **modèle par agent** adapté à la difficulté (tableau 7.1, chapitre 7) ;
- un **plafond de tours** (`maxTurns`) par sous-agent et des plafonds de boucles dans `.ecom/config.json` ;
- la **mesure** systématique des tokens et des coûts par phase (tableau de bord AgentOps, rapport final) ;

- l’usage de `/loop` plutôt que d’un agent permanent pour la surveillance (exécutions courtes et espacées).

Deuxième partie

Rapport de définition du projet (RDP)

5 Architecture du système multi-agents

5.1 Vue d'ensemble

Le système est organisé en cinq couches horizontales et une couche transverse de gouvernance (figure 5.1). L'humain interagit par des commandes et des gates ; l'orchestrateur pilote l'exécution à partir d'un état persistant ; les sous-agents réalisent le travail technique ; les serveurs MCP leur donnent accès aux systèmes cibles ; les hooks garantissent les règles.

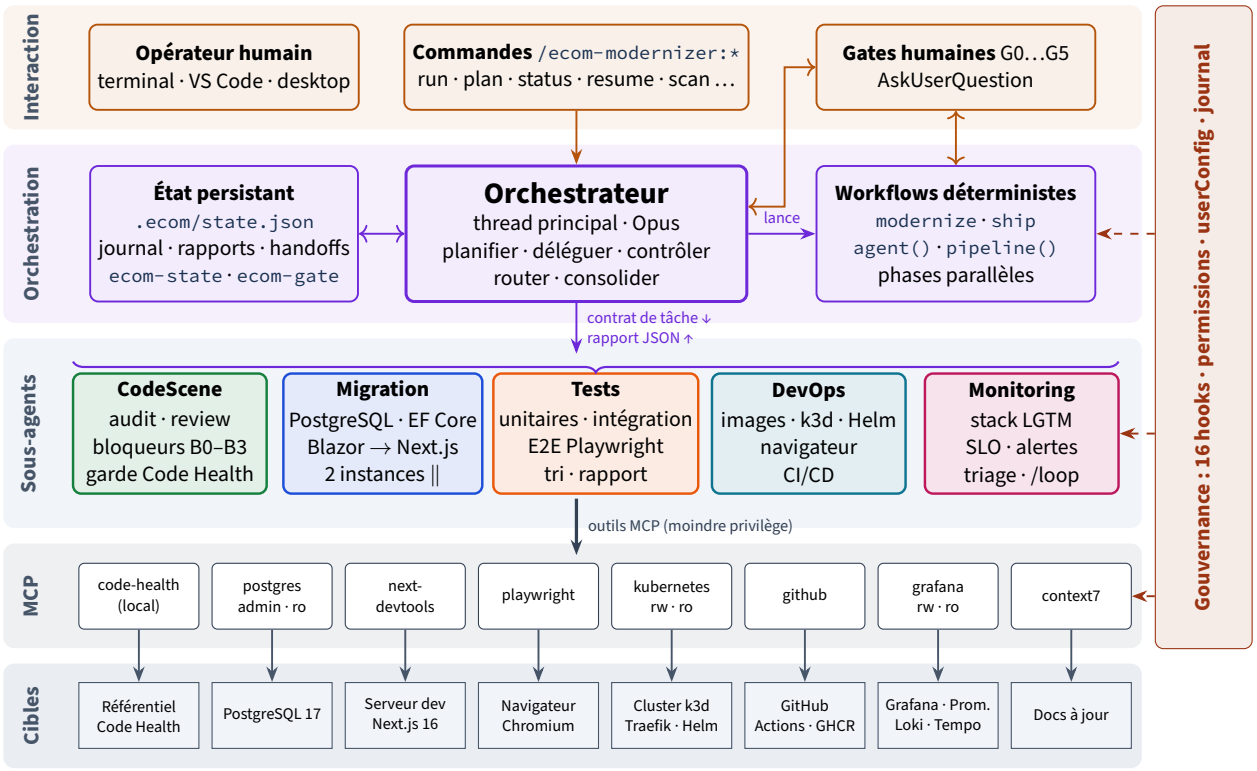


FIGURE 5.1 – Architecture en couches du système multi-agents `ecom-modernizer`

Interaction
L'opérateur lance des commandes `/ecom-modernizer:*` et répond aux gates G0–G5, posées par l'orchestrateur via l'outil `AskUserQuestion` (les sous-agents ne peuvent pas interroger l'utilisateur : c'est voulu).

Orchestration
L'orchestrateur s'exécute comme *thread principal* de la session. Il lit et écrit l'état du run au moyen de l'utilitaire `ecom-state`, évalue les gates avec `ecom-gate`, et peut confier une séquence de phases à un *workflow* déterministe.

Sous-agents
Cinq agents spécialisés, chacun avec son prompt système, sa liste d'outils, son modèle, ses skills préchargés et sa mémoire de projet. Chaque invocation dispose d'un contexte vierge et isolé.

MCP
Onze serveurs, dont trois dédoublés en instances lecture seule et lecture/écriture pour appliquer le

moindre privilège.

Cibles
Les systèmes réels : référentiel Code Health du projet, PostgreSQL, serveur de développement Next.js, navigateur, cluster k3d, GitHub, stack Grafana, documentation.

Gouvernance
Seize hooks, des règles de permission, la configuration utilisateur du plugin, un journal d'audit et des évaluations automatisées.

5.2 Principes d'architecture

TABLE 5.1 – Principes d'architecture et leur mise en œuvre

N°	Principe	Mise en œuvre
P1	Une responsabilité par agent	Cinq domaines disjoints ; périmètres d'écriture contrôlés par `guard-paths`.
P2	Moindre privilège	Liste `tools` par agent, instances MCP en lecture seule, hook `guard-mcp`, aucun agent ne touche à `.claude/`.
P3	Contrats explicites	Contrat de tâche en entrée, rapport JSON validé en sortie, handoffs typés entre agents.
P4	Orchestration pilotée par l'état	Machine à états persistée dans `.ecom/state.json` ; l'orchestrateur ne s'appuie pas sur sa seule mémoire conversationnelle.
P5	Garanties déterministes	Ce qui doit toujours être vrai est codé en hooks et en scripts (gates), pas seulement écrit dans un prompt.
P6	Humain aux points irréversibles	Gates G0 à G5 ; décision `ask` imposée par hook pour les suppressions, push et fusions.
P7	Idempotence et reprise	Étapes rejouables, checkpoints git par phase, reprise après interruption ou compaction.
P8	Boucles bornées	Plafonds d'itérations, détection d'absence de progrès, plafond de relances du hook Stop.
P9	Observabilité de bout en bout	Télémétrie de l'application *et* du système d'agents (tokens, coûts, échecs d'outils).
P10	Flexibilité là où elle sert, déterminisme ailleurs	Orchestrateur agentique pour décider et dialoguer ; workflows scriptés pour les phases répétables et parallèles.

5.3 Briques Claude Code mobilisées

TABLE 5.2 – Briques de la plateforme et leur rôle dans le système

Brique	Nature	Rôle dans le système	Emplacement
Sous-agent	Assistant à contexte isolé, prompt système, outils, modèle, skills préchargés	Les cinq agents spécialisés ; l'orchestrateur en thread principal (`--agent`)	`agents/*.md`
Skill	Connaissances et procédures chargées à la demande (divulgation progressive)	Savoir-faire de chaque agent (migration EF Core, parité Blazor/Next.js, SLO…)	`skills/*/SKILL.md`
Commande	Point d'entrée nommé invoqué par l'utilisateur	Lancer un run, un agent, consulter l'état, reprendre	`commands/*.md`
Hook	Programme exécuté à un événement du cycle de vie	Garde-fous, quality gates, journal, injection de contexte, boucle bornée	`hooks/hooks.json`, `scripts/`
Serveur MCP	Fournisseur d'outils externe (protocole MCP)	Accès à CodeScene, PostgreSQL, Next.js, navigateur, cluster, GitHub, Grafana	`.mcp.json`
Workflow	Script JavaScript orchestrant des sous-agents	Exécution déterministe et parallèle de séquences de phases	`workflows/*.js`
Monitor	Processus de fond dont chaque ligne de sortie notifie Claude	Alerte Grafana → tri d'incident (boucle événementielle)	`monitors/`
Exécutables	Fichiers ajoutés au PATH du shell des agents	`ecom-state`, `ecom-gate`, `ecom-doctor`	`bin/`
`userConfig`	Valeurs demandées à l'activation (secrets au trousseau)	Jetons, URI, profil par défaut, agents activés	`plugin.json`
Mémoire d'agent	Répertoire persistant `MEMORY.md` par agent	Capitaliser d'un run à l'autre (smells récurrents, particularités du cluster)	`.claude/agent-memory/`
Référentiel Code Health	Règles, moteur et serveur MCP générés dans le projet	Mesure locale de la santé du code et détection des bloqueurs, sans clé d'API	`quality/code-health/`
Évaluations	Cas de test et correcteurs pour le plugin	Mesurer le routage, l'activation, le respect du contrat	`evals/`
Plugin et marketplace	Unité de distribution versionnée	Installation, activation, mise à jour	`.claude-plugin/`

⚖ Pourquoi des sous-agents plutôt qu'un agent unique ou une équipe d'agents ?

Les sous-agents offrent l'**isolation de contexte** (chaque phase démarre avec un contexte propre, seul le rapport remonte), la **spécialisation** (prompt, outils, modèle) et un **coût maîtrisé**. Les *agent teams* de Claude Code (coéquipiers indépendants, liste de tâches partagée, messagerie) restent

expérimentales, sans reprise de session pour les coéquipiers et plus coûteuses : elles sont réservées à une variante (section 12.5). Les *workflows* complètent les sous-agents quand l'orchestration doit être codifiée et rejouable.

5.4 Architecture cible de l'application

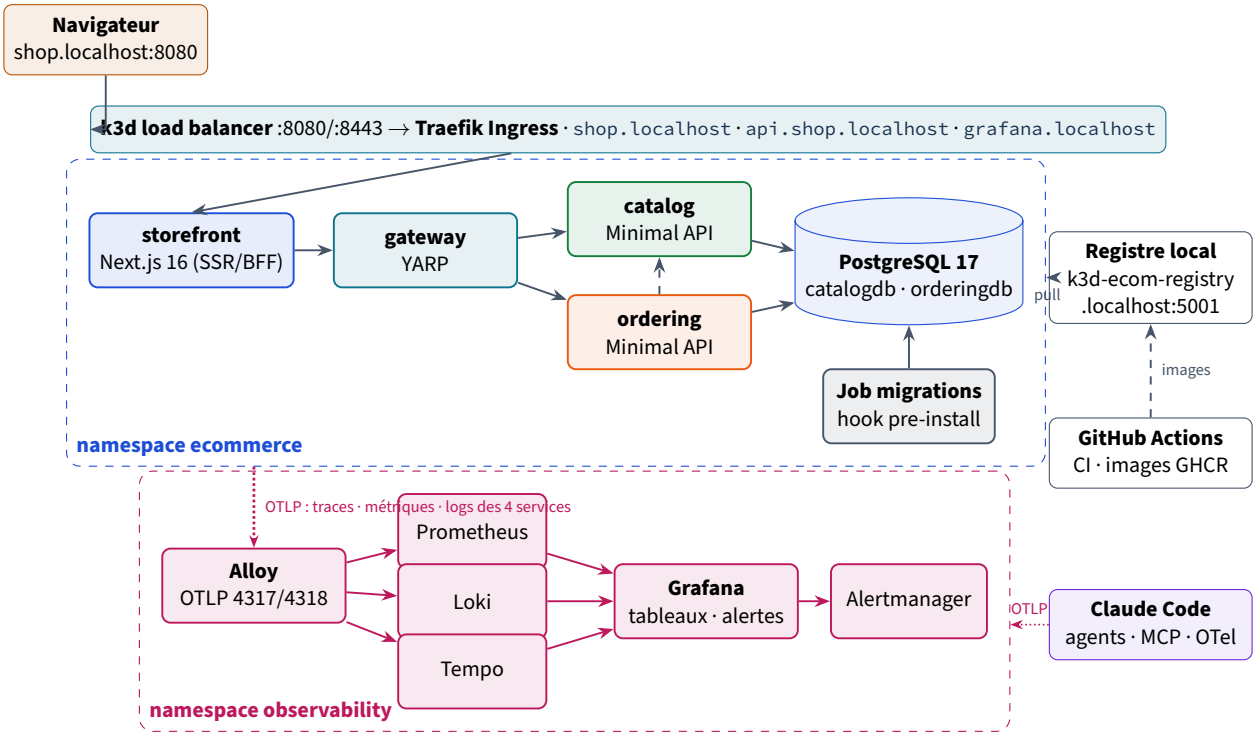


FIGURE 5.2 – Architecture cible de l'application modernisée, déployée sur k3d

Les évolutions par rapport à l'existant (figure 1.1) sont :

- un **storefront Next.js** rendu côté serveur, qui appelle la gateway *depuis le serveur* (rôle de BFF) : pas de CORS à ouvrir, aucun secret côté navigateur ;
- **PostgreSQL 17** avec une base par service et un **job de migration** exécuté avant chaque installation ou mise à niveau ;
- un **ingress Traefik** unique sur le load balancer k3d (`shop.localhost`, `grafana.localhost`) ;
- une **stack d'observabilité** dans son propre namespace, alimentée en OTLP par Alloy ; les services exportent déjà en OTLP dès que `OTEL_EXPORTER_OTLP_ENDPOINT` est défini ;
- un **registre local** pour les images construites, une **CI GitHub Actions** pour l'intégration continue, et la télémétrie de **Claude Code** lui-même pour le tableau de bord AgentOps.

5.5 Organisation du dépôt cible

```
ecommerce-app-agents/
├── ECommerce.slnx
├── src/
│   ├── ECommerce.AppHost/ .....+ PostgreSQL, storefront
│   ├── ECommerce.Catalog.Api/ ...+ Npgsql, migrations
│   ├── ECommerce.Ordering.Api/ + validation, télémétrie
│   ├── ECommerce.MigrationService/ ..... nouveau
│   ├── ECommerce.Gateway/
│   ├── ECommerce.ServiceDefaults/ ..+ sondes en prod
│   ├── ECommerce.Storefront/ ........nouveau (Next.js)
│   └── ECommerce.Web/ ...................retiré après G3
├── tests/ ................. unitaires, intégration, système
├── quality/code-health/ .......référentiel Code Health
│   (généré)
├── deploy/
│   ├── k3d/cluster.yaml
│   ├── helm/ecommerce/
│   └── observability/
├── .github/workflows/ ................... ci.yml, cd.yml
├── .ecom/ ..................état du système multi-agents
├── docs/reports/ ......................rapports générés
└── .claude/settings.json ..........plugin, permissions
```

Le dépôt reste la **source unique de vérité** : code, tests, manifestes, pipelines, rapports et état du système multi-agents y sont versionnés (à l'exception de `.ecom/runtime/`, `.ecom/cache/` et `.ecom/checkpoints/`, ignorés par git).
Le projet Blazor est conservé jusqu'à la validation de parité G3 : c'est la stratégie *strangler*, qui permet de comparer les deux interfaces pendant la transition.

5.6 Communication entre agents

Les sous-agents **ne dialoguent pas directement** entre eux : la topologie est en étoile (*hub and spoke*) autour de l'orchestrateur, complétée par un **tableau noir** partagé sur disque, le répertoire `.ecom/`. Ce choix rend chaque échange explicite, persistant et auditable.

TABLE 5.3 – Artefacts partagés du répertoire .ecom

Artefact	Contenu	Écrit par
`config.json`	Profils, préconditions, gates, seuils, plafonds de boucles, paramètres du cluster	Humain (`init`)
`state.json`	Run courant : statut, phases, gates, dérogations, compteurs, checkpoints	`ecom-state` uniquement
`journal.jsonl`	Événements horodatés (lancements, refus, blocages, fins, échecs d'outils)	Hooks, `ecom-state`
`reports/<agent>/*.json`	Rapports de fin de tâche conformes au schéma	Sous-agents
`handoffs/*.json`	Transferts typés entre agents (bloqueurs, endpoints, ressources AppHost…)	Sous-agents
`checkpoints/`	Copies de l'état avant compaction	Hook `PreCompact`
`runtime/`, `cache/`	Marqueurs de lancement, compteurs, scores Code Health	Hooks

Exemple de handoff produit par l'agent Migration à destination de l'agent Tests :

```
{
  "from": "migration", "to": "tests", "run_id": "run-20261021-1430",
  "endpoints": [
    {"method": "POST", "route": "/ordering/api/orders", "changed": "validation items/quantité (B-04, B-05)"},
    {"method": "POST", "route": "/catalog/api/products", "changed": "validation nom/prix (B-07)"}
  ],
  "frontend_routes": ["/", "/products", "/orders", "/not-found"],
  "database": {"catalogdb": ["Products"], "orderingdb": ["Orders", "OrderItems"]},
  "seed": {"products": 4},
  "start": "dotnet run --project src/ECommerce.AppHost"
}
```

Un échange *synchrone* reste possible lorsqu'une correction est nécessaire : l'orchestrateur reprend l'instance d'agent concernée avec `SendMessage` (identifiant ou nom de l'agent), qui conserve tout son historique. C'est le mécanisme des boucles de rétroaction décrit au chapitre 11.

6 L'orchestrateur

6.1 Ce qu'est exactement l'orchestrateur

Définition

L'orchestrateur est un **agent Claude Code exécuté comme thread principal de la session** (`claude --agent ecom-modernizer:orchestrator`). Il possède un prompt système de chef de projet technique, un modèle de raisonnement (Opus), une liste d'outils restreinte — il ne peut lancer *que* les cinq sous-agents du plugin — et un protocole adossé à une **machine à états persistante**. Son travail consiste à **planifier, déléguer, contrôler, arbitrer et consolider**. Il ne produit jamais lui-même de code, de test, de manifeste ou de tableau de bord.

Trois idées fausses sont à écarter :

- **ce n'est pas un serveur ni un démon** : il n'existe que le temps d'une session Claude Code; sa continuité est assurée par l'état sur disque (`.ecom/state.json`), pas par un processus permanent;
- **ce n'est pas un super-agent qui fait tout** : s'il codait, il saturerait son contexte et perdrait la séparation des responsabilités; le hook `guard-bash` lui refuse d'ailleurs `dotnet`, `npm`, `kubectl`, `helm`;
- **ce n'est pas un script figé** : il interprète les rapports, choisit le bon agent pour corriger, formule les questions aux humains. Lorsque l'enchaînement est parfaitement connu, il peut en revanche déléguer à un *workflow* scripté.

Table 6.1 – Quatre manières d'orchestrer et leur place dans le projet

Approche	Qui décide la suite	Où vivent les résultats	Usage dans le projet
Orchestrateur agentique (sous-agents)	Le modèle, tour par tour	Rapports sur disque, résumés en contexte	**Mode principal** : runs interactifs, gates humaines, routage des échecs
Workflow dynamique	Le script	Variables du script	Phases répétables et parallèles (`modernize`, `ship`), relance après pause
Agent teams (expérimental)	Le lead et les coéquipiers	Liste de tâches partagée	Variante pour l'exploration parallèle (revue croisée, débogage par hypothèses)
Pipeline CI	La définition YAML	Artefacts CI	Exécution non interactive des profils `ci` et des contrôles de PR

6.2 Responsabilités

TABLE 6.2 – Responsabilités de l'orchestrateur

N°	Responsabilité	Comment	Outils et artefacts
R1	Charger le contexte	Lit l'état, la configuration du projet et les arguments de la commande.	`ecom-state summary`, `config.json`
R2	Résoudre la sélection	Agents = arguments > profil > configuration utilisateur ; retire les phases orphelines et vérifie leurs préconditions.	Skill `pipeline-orchestration`
R3	Vérifier les prérequis	Outils, démon Docker, SDK, serveurs MCP.	`ecom-doctor --json`
R4	Planifier	Construit le DAG, identifie le parallélisme et les gates, fait valider le plan.	`AskUserQuestion` G0
R5	Déléguer	Rédige le contrat de tâche, lance le sous-agent propriétaire (en parallèle si possible).	Outil `Agent`, modèle de contrat
R6	Contrôler	Lit le rapport JSON, évalue la gate de la phase.	`ecom-gate`, rapports
R7	Router et arbitrer	Phase suivante, rétroaction vers l'instance fautive, gate humaine ou escalade.	`SendMessage`, table de routage
R8	Tracer et reprendre	Met à jour l'état, pose les checkpoints git, reprend après interruption.	`ecom-state`, tags `ecom/<run>/<phase>`
R9	Consolider	Rédige le rapport final (indicateurs, gates, coûts, risques).	Skill `run-reporting`

Ce que l'orchestrateur ne fait pas — et ce qui l'en empêche

Modifier `src/`, `tests/`, `deploy/`	Hook `guard-paths` : écriture limitée à `.ecom/` et `docs/reports/`
Lancer des builds, des tests, des commandes de cluster	Hook `guard-bash` : `dotnet`, `npm`, `kubectl`, `helm`, `k3d`… refusés
Lancer un agent hors plugin ou désactivé	Liste `Agent(...)` dans `tools` + hook `gate-agents`
Modifier l'état à la main	`state.json` protégé ; seul `ecom-state` écrit
Décider à la place de l'humain	Gates G0–G5 ; décision `ask` imposée sur l'irréversible

6.3 Définition et activation

L'en-tête de l'agent fixe son identité, ses outils et son modèle (fichier complet en annexe) :

agents/orchestrator.md (en-tête)

```
---
name: orchestrator
description: Orchestrateur du pipeline ecom-modernizer. Planifie le run, délègue chaque phase au sous-agent
     ↪ propriétaire (codescene, migration, tests, devops, monitoring), évalue les quality gates, conduit les
     ↪ validations humaines et consolide le rapport final. À lancer comme thread principal (claude --agent ecom-
     ↪ modernizer:orchestrator) ou via /ecom-modernizer:run. Ne modifie jamais le code applicatif.
tools: Agent(ecom-modernizer:codescene, ecom-modernizer:migration, ecom-modernizer:tests, ecom-modernizer:devops
     ↪ , ecom-modernizer:monitoring), Read, Grep, Glob, Write, Edit, Bash, Skill, TodoWrite, AskUserQuestion,
     ↪ SendMessage, Workflow, mcp__plugin_ecom-modernizer_grafana-ro__*, mcp__plugin_ecom-modernizer_kubernetes-
     ↪ ro__*
model: opus
effort: high
memory: project
color: purple
skills:
  - ecom-modernizer:pipeline-orchestration
  - ecom-modernizer:agent-report-contract
  - ecom-modernizer:run-reporting
---
```

⚠ Pourquoi le thread principal ?

Seul le thread principal peut interroger l'utilisateur (`AskUserQuestion` est retiré de tout sous-agent), lancer un workflow (l'outil `Workflow` est lui aussi retiré des sous-agents) et restreindre la liste des agents lançables : la syntaxe `Agent(type1, type2)` n'est appliquée qu'à un agent exécuté avec `--agent`. Un orchestrateur lancé comme simple sous-agent perdrait donc ces trois capacités.

Quatre modes d'activation sont prévus :

1. **Session dédiée (recommandé)** : `claude --agent ecom-modernizer:orchestrator`, puis `/ecom-modernizer:run --profile full`. Le thread principal *est* l'orchestrateur, avec ses restrictions d'outils.
2. **Commande depuis une session ordinaire** : `/ecom-modernizer:run`; la commande charge le skill `pipeline-orchestration` qui décrit le même protocole. Plus souple, mais sans restriction d'outils : les hooks restent la garantie.
3. **Workflow** : `/ecom-modernizer:modernize` puis `/ecom-modernizer:ship`; orchestration codifiée, les gates humaines se placent *entre* les deux workflows (aucune saisie possible pendant un workflow).
4. **Non interactif (CI)** : `claude -p "/ecom-modernizer:run --profile ci" --agent ecom-modernizer:orchestrator` ou l'action GitHub `anthropics/claude-code-action@v1`; les gates non auto-approuvées mettent le run en pause (échec fermé).

Une variante consiste à placer `{"agent": "orchestrator"}` dans le `settings.json` du plugin : toute session où le plugin est actif démarrerait alors avec l'orchestrateur. Nous l'écartons par défaut, pour ne pas détourner les sessions ordinaires du dépôt.

6.4 Machine à états du run

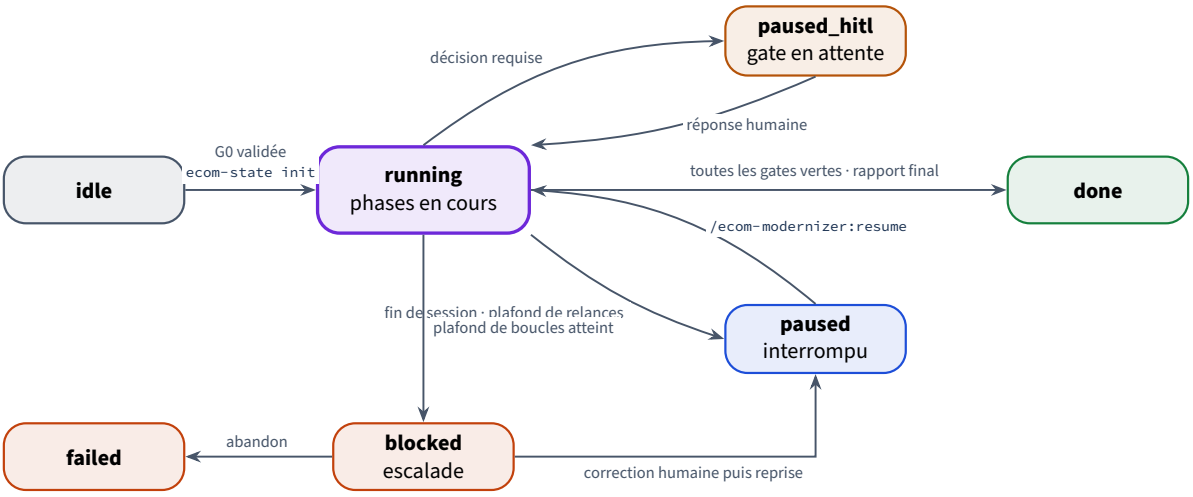


FIGURE 6.1 – Machine à états d'un run (statuts persistés dans `.ecom/state.json`)

Chaque phase possède en outre son propre statut (`pending`, `running`, `done`, `failed`, `skipped`), son nombre de tentatives et ses horodatages. La transition `running` → `paused` est déclenchée automatiquement par le hook `SessionEnd` ou par le plafond de relances du hook `Stop` : un run n'est jamais perdu, il est repris par `/ecom-modernizer:resume`.

6.5 Plan d'exécution et parallélisme

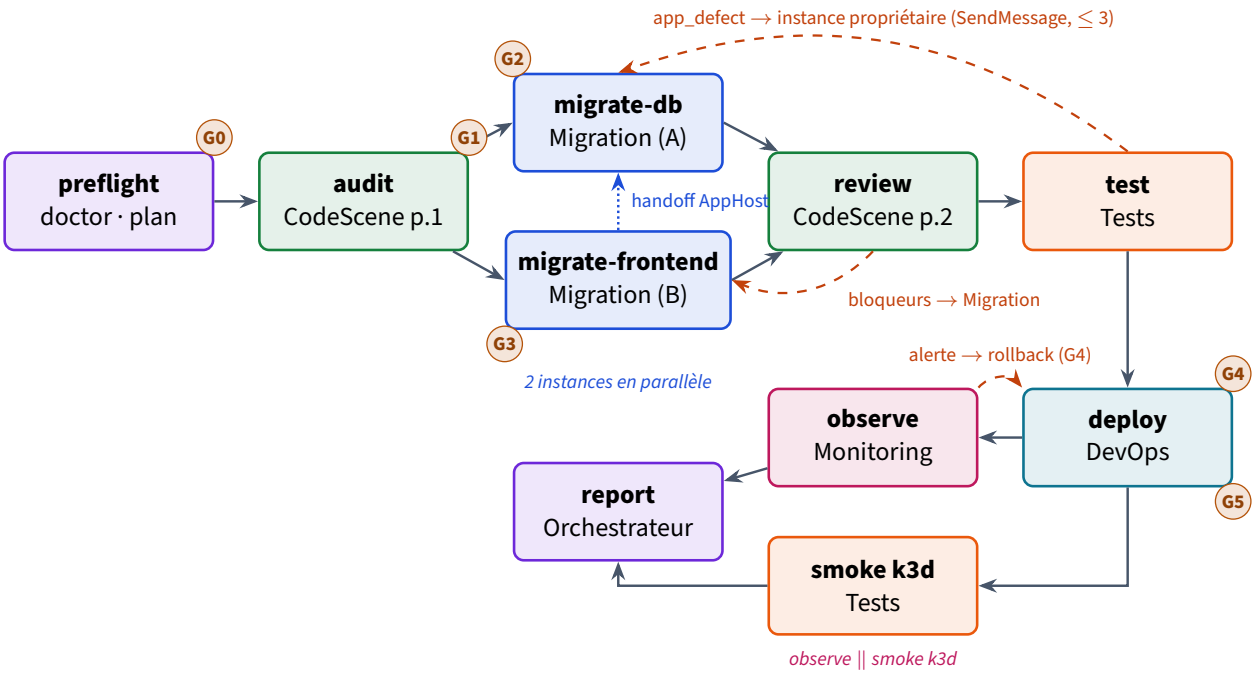


FIGURE 6.2 – DAG des phases du profil full, gates humaines et boucles de rétroaction

Deux points de parallélisme sont exploités :

- **migrate-db** ‖ **migrate-frontend** : deux instances de l'agent Migration travaillent sur des périmètres disjoints (`src/ECommerce.*.Api`, `src/ECommerce.MigrationService` d'un côté, `src/ECommerce.Storefront` de l'autre). Le seul fichier commun, `AppHost.cs`, appartient à l'instance A ; l'instance B décrit la ressource `storefront` dans un handoff que l'instance A applique.
- **observe** ‖ **smoke k3d** : dès que le déploiement est vert, la supervision s'installe pendant que les parcours critiques sont rejoués sur le cluster.

6.6 Contrat de tâche

Chaque délégation suit le même modèle. La première ligne `phase:` est lue par le hook `gate-agents`, qui refuse le lancement si une phase préalable n'est pas terminée ou dérogée.

templates/task-contract.md

```
phase: <audit|migrate-db|migrate-frontend|review|test|deploy|observe>
task: <identifiant de la tâche>
run_id: <run-AAAAMMJJ-HHMM>

## Contexte
<état du dépôt, phase précédente, décisions humaines déjà prises (gates), liens vers les rapports utiles>

## Objectif
<résultat attendu en une phrase vérifiable>

## Entrées
- Handoffs : <.ecom/handoffs/*.json>
- Rapports amont : <.ecom/reports/...>
- Références : <fichiers, tags git, URL locales>

## Contraintes
- Périmètre d'écriture : <dossiers> (contrôlé par guard-paths)
- Interdits : <actions exclues>
- Budget : <maxTurns / nombre d'itérations de la boucle>

## Critères d'acceptation (lus par ecom-gate)
- <métrique> <opérateur> <seuil>
- ...

## Livrables
- Rapport : .ecom/reports/<agent>/<run_id>-<task>.json (skill agent-report-contract)
- Synthèse : docs/reports/<run_id>-<agent>-<task>.md
- Handoffs éventuels : .ecom/handoffs/<agent>-to-<agent>.json

## Mode d'exécution
<one-shot | boucle bornée : condition d'arrêt, plafond d'itérations, critère d'absence de progrès>
```

6.7 Quality gates

Une gate de qualité est évaluée par `ecom-gate <phase> <rapport>` à partir des métriques du rapport et des seuils de `.ecom/config.json`. Le résultat est un JSON `{phase, pass, reasons[]}` et un code de sortie (0 vert, 1 rouge). La décision ne dépend donc pas de l'appréciation du modèle.

TABLE 6.3 – Critères des quality gates par phase

Phase	Critères (seuils par défaut)
audit, review	0 bloqueur B0 et B1 ouvert ; aucune régression de Code Health ; en review, Code Health moyenne $\geq$ 9,0
migrate-db	Build vert ; parité des endpoints = 1,0
migrate-frontend	Build Next.js vert ; parité des pages = 1,0 ; 0 erreur ESLint ; 0 erreur TypeScript
test	Taux de réussite $\geq$ 0,98 ; couverture backend $\geq$ 0,80 et frontend $\geq$ 0,70 ; instabilité $\leq$ 0,02 ; parcours E2E critiques verts ; aucun défaut critique ou majeur ouvert
deploy	Pods prêts = pods attendus ; ingress HTTP 200 ; durée $\leq$ 10 minutes
observe	Sources de données saines ; $\geq$ 5 tableaux de bord ; $\geq$ 6 règles d'alerte ; game day concluant
toutes	Statut du rapport = `success`

6.8 Routage des échecs

TABLE 6.4 – Table de routage des échecs

Signal	Responsable	Action de l'orchestrateur	Plafond
Gate audit ou review rouge	CodeScene	`SendMessage` à la même instance avec les bloqueurs restants	3
Build ou migration en échec	Migration (instance concernée)	`SendMessage` avec la sortie d'erreur	3
Défaut applicatif (`app_defect`)	propriétaire présumé	Transmettre la preuve, puis relancer Tests sur le périmètre	3
Défaut de test (`test_defect`)	Tests	Auto-réparation (déjà bornée dans l'agent)	3
Défaut d'environnement, CrashLoop, ingress KO	DevOps	Diagnostic à partir des événements et des logs, redéploiement	3
Alerte après déploiement	Monitoring puis DevOps	Tri d'incident ; rollback Helm après G4	1
Régression de Code Health après correctif	CodeScene	Revue ciblée du diff	2
Plafond atteint	Humain	`ecom-state status blocked` et synthèse de l'escalade	—

6.9 Gates humaines

TABLE 6.5 – Gates humaines

Gate	Moment	Question type et options	En CI
G0	Avant tout run	« Valider ce plan (agents, phases, durée estimée) ? » — Lancer / Modifier la sélection / Essai à blanc	Auto-approuvée
G1	Correctif métier détecté	« Le stock doit-il être réservé à la commande (B-05) ? » — Implémenter / Reporter / Refuser	Pause
G2	Fin de migrate-db	« Valider le schéma et les migrations ? » — Valider / Demander une correction	Pause
G3	Fin de migrate-frontend	« Valider la parité de l'UI (captures jointes) ? » — Valider et retirer Blazor / Valider en gardant Blazor / Corriger	Pause
G4	Avant déploiement, rollback ou suppression	« Déployer la version `<sha>` sur k3d-ecommerce ? » — Déployer / Reporter	Pause
G5	Avant push, PR ou fusion	« Pousser la branche et ouvrir la pull request ? » — Pousser / Garder en local	Pause

Les décisions sont enregistrées (`ecom-state gate G2 passed "motif"`) et reprises dans le rapport final. Indépendamment de l'orchestrateur, le hook `guard-bash` impose une confirmation pour `git push`, `k3d cluster delete`, `helm uninstall` ou `gh pr merge` : la gate est donc garantie même si un agent tentait l'action de lui-même.

7 Les sous-agents

Chaque sous-agent est décrit selon la même grille : mission reformulée, moment d'intervention, entrées et sorties, outils, skills, commandes, hooks applicables, mode d'exécution, critères de succès et garde-fous. Les prompts système complets figurent dans le squelette du plugin (`agents/*.md`).

7.1 Choix des modèles et budgets

TABLE 7.1 – Modèle, effort et budget par agent

Agent	Modèle	Effort	maxTurns	Justification
Orchestrateur	opus	high	—	Arbitrages, lecture de rapports, dialogue humain ; contexte léger car tout le détail reste chez les sous-agents.
CodeScene	sonnet	high	120	Refactorings ciblés et nombreux appels d'outils courts.
Migration	opus	high	200	Refonte multi-fichiers, conception du schéma et de l'architecture Next.js.
Tests	sonnet	high	150	Génération volumineuse et itérative de tests.
DevOps	sonnet	high	150	Procédures connues, diagnostic guidé par les événements et les logs.
Monitoring	sonnet	medium	120	Requêtes et configurations ; le mode `check` peut être lancé en `haiku` via le paramètre `model` de l'outil Agent.

Tous les sous-agents ont `memory: project` : leur fichier `.claude/agent-memory/<agent>/MEMORY.md` capitalise d'un run à l'autre (conventions découvertes, pièges rencontrés, décisions humaines), sans alourdir les prompts.

7.2 Agent CodeScene — gardien de la santé du code

Mission

Générer et tenir à jour le **référentiel Code Health local** du projet (sans clé d'API), mesurer la santé du code, **détecter, classer et corriger les bloqueurs** qui relèvent de la qualité, du déploiement, de la sécurité ou du frontend, **avant et après la refonte** — y compris le code Next.js généré par l'agent Migration — et empêcher toute régression de Code Health.

TABLE 7.2 – Fiche signalétique de l'agent CodeScene

Phases	`audit` (passe 1, sur l'existant) et `review` (passe 2, sur le code modifié et sur `src/ECommerce.Storefront`) ; garde continue via hooks
Entrées	Code source, seuils de `config.json`, tag `ecom/<run>/audit` pour la passe 2
Sorties	Référentiel `quality/code-health/` (règles, baseline, historique), `.ecom/reports/codescene/<run>-<passe>.json`, synthèse Markdown, handoff `codescene-to-migration.json`
Outils natifs	Read, Grep, Glob, Edit, Write, Bash, Skill, TodoWrite
Serveurs MCP	`code-health`, serveur local du référentiel (`project_code_health`, `code_health_score`, `code_health_review`, `list_blockers`, `analyze_change_set`, `pre_commit_code_health_safeguard`, `list_hotspots`, `rules_config_list_thresholds`) ; `next-devtools` (`nextjs_index`, `nextjs_call get_errors`) ; `context7`
Skills préchargés	`code-health-referential`, `blocker-taxonomy`, `code-health-refactoring`, `nextjs-code-health`, `agent-report-contract`
Commande	`/ecom-modernizer:scan [--init] [--scope backend|frontend|all] [--fix] [--baseline <ref>] [--pass audit|review] [--hotspots]`
Hooks applicables	`guard-paths` (écriture dans `src/` et `quality/code-health/`), `guard-bash` (pas de cluster ni de push), `code-health-guard` (note du fichier après chaque édition, moteur local), `precommit-safeguard` (`safeguard` avant commit), `subagent-stop` (rapport obligatoire)
Mode d'exécution	One-shot par passe ; boucle de correction bornée (3 itérations par fichier, arrêt après 2 itérations sans progrès)
Gate	0 B0/B1 ouvert ; aucune régression ; Code Health moyenne $\geq$ 9,0 en review
Interdits	`kubectl`, `helm`, `k3d`, `docker push`, `git push` ; corriger un bloqueur MIGRATION ; appliquer un correctif métier sans G1

7.2.1 Taxonomie et procédure

Le skill `blocker-taxonomy` fixe une échelle commune à tous les agents : **B0** empêche le build, le démarrage, le déploiement ou expose une faille critique ; **B1** bloque la migration ou une fonctionnalité ; **B2** est une dette majeure (Code Health < 7) ; **B3** une amélioration. Chaque bloqueur reçoit une catégorie (QUALITY, MIGRATION, DEPLOY, SECURITY, FRONTEND, TEST, OBSERVABILITY, FUNCTIONAL) qui détermine son propriétaire. La procédure est la suivante :

1. **Référentiel** : génération de `quality/code-health/` au premier passage (`scan --init`), baseline des notes, puis mise à jour éventuelle des règles sous G1.
2. **Inventaire** : `project_code_health` sur le périmètre, `code_health_review` pour les fichiers sous le seuil (Complex Method, Deep Nested Complexity, Complex Logic, Large Method, Excess Function Arguments, Large File, Code Duplication) et `list_blockers` pour les règles de bloqueurs du projet.
3. **Revue complémentaire** des bloqueurs que la Code Health ne mesure pas (sondes, migrations, sécurité, Next.js) à l'aide du catalogue du skill.
4. **Classement et attribution** ; transfert des bloqueurs MIGRATION ; `decisions_required` pour les changements métier (B-05).

5. **Correction itérative** : un smell à la fois, build et tests après chaque changement, re-mesure ; annulation de toute modification qui fait baisser le score.
6. **Contrôle final** : `pre_commit_code_health_safeguard` sur l'ensemble des changements.

7.2.2 Le cas du code Next.js généré

La passe `review` intervient *après* l'agent Migration : un code généré par un modèle doit être relu par un agent distinct, outillé différemment. Pour Next.js, l'agent combine trois sources : la Code Health des composants (TypeScript/TSX), les erreurs *à chaud* du serveur de développement obtenues par `nextjs_call get_errors` (build, exécution, typage, hydratation) et les règles du skill `nextjs-code-health` (frontière serveur/client, secrets en `NEXT_PUBLIC_*`, `error.tsx`, accessibilité, `output: 'standalone'`).

7.2.3 Le référentiel Code Health du projet (sans clé d'API)

⚖ Exigence : analyser le code sans clé d'API

Les outils officiels de CodeScene (serveur MCP, CLI) exigent un jeton d'accès, y compris pour l'analyse locale. Le projet impose au contraire une analyse **sans clé d'API ni service externe**. L'agent CodeScene **génère donc, à l'intérieur du dépôt, un référentiel Code Health** : des règles versionnées, un moteur d'analyse open source, un serveur MCP local et une baseline. La démarche Code Health (notes de 1 à 10, code smells, garde anti-régression, hotspots) est conservée ; seul le moteur change, et il appartient au projet.

```
quality/code-health/
├── REFERENTIEL.md ....modèle et gouvernance
├── rules.json ........smells, seuils, bloqueurs
├── code_health.py .............moteur et CLI
├── server.py ................serveur MCP local
├── baseline.json ..........notes de référence
├── waivers.json ..............dérogations (G1)
└── history/ ................rapports successifs
```

Génération : au premier `/ecom-modernizer:scan --init`, l'agent copie le modèle livré par le plugin (`templates/code-health/`), mesure l'existant et écrit la baseline. Le serveur MCP du plugin bascule alors automatiquement sur le référentiel du projet.
Moteur : métriques de **lizard** (C#, TypeScript, TSX, JavaScript) — complexités cyclomatique et cognitive, imbrication, lignes utiles, paramètres — complétées par une détection de duplication et par les règles de bloqueurs propres au projet. Exécution par `uv run --script`, sans installation préalable.

TABLE 7.3 – Modèle de notation du référentiel (extrait de rules.json)

Code smell	Métrique	Alerte	Critique	Poids
Complex Method	complexité cyclomatique	10	15	1,0
Deep, Nested Complexity	imbrication maximale	4	5	1,0
Complex Logic	complexité cognitive	15	25	0,8
Large Method	lignes utiles (TSX : 90/150)	70	120	0,8
Excess Function Arguments	paramètres	5	7	0,5
Large File	lignes utiles du fichier	600	1000	1,0
Code Duplication	blocs de 6 lignes identiques	1 bloc	—	0,7 (plafond 2)

La note d'un fichier vaut $\max(1,\ 10 - \sum \text{pénalités})$, une alerte coûtant $0{,}5\times$ poids et une alerte critique $1{,}0\times$ poids ; les bandes sont *healthy* (≥ 9), *problematic* (4 à 8,9) et *unhealthy* (< 4). La formule est publique, versionnée et modifiable sous contrôle humain (G1).

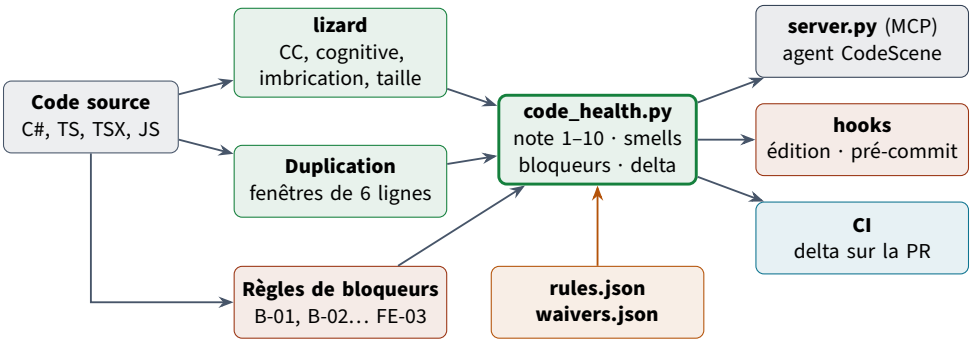


FIGURE 7.1 – Chaîne d'analyse du référentiel Code Health local

Résultats sur l'existant. Le moteur a été exécuté sur le code actuel de l'application, sans clé d'API : 16 fichiers analysés, Code Health moyenne **9,95**, minimum **9,5** (`ReconnectModal.razor.js` : Deep Nested Complexity, imbrication 4). Les règles de bloqueurs du référentiel détectent **11 occurrences** — B0 : 1 (B-01) ; B1 : 8 (B-02 et B-02b sur les deux API, B-03, B-04, B-08 sur les deux API) ; B2 : 2 (B-06, B-09) — conformes au diagnostic du tableau 1.3. Les bloqueurs B-05, B-07 et B-10, qui demandent une compréhension métier ou une vue d'ensemble, restent du ressort de la revue de l'agent. La santé élevée du code existant confirme que ses défauts sont des bloqueurs de migration et de déploiement plutôt que de la dette de conception : le référentiel servira surtout à *garder* ce niveau pendant la refonte.

Garde anti-régression. Sur une copie du dépôt, l'ajout d'une méthode à conditions imbriquées et à six paramètres fait passer la note du fichier de 10,0 à 7,3 avec quatre nouveaux smells (Complex Method, Deep Nested Complexity, Complex Logic, Excess Function Arguments) : `safeguard` refuse le commit (code 1) et le hook `code-health-guard` renvoie immédiatement l'écart au modèle (code 2) pour qu'il corrige avant de poursuivre.

templates/code-health/rules.json (smell Complex Method)

```
  "smells": {
    "complex_method": {
      "label": "Complex Method", "scope": "function", "metric": "cyclomatic_complexity",
      "warning": 10, "critical": 15, "weight": 1.0,
      "advice": "Extraire des méthodes nommées par responsabilité ; remplacer les conditions par des retours
      ↪ anticipés."
    },
```

templates/code-health/rules.json (règle de bloqueur B-01)

```
    {
      "id": "B-01", "severity": "B0", "category": "DEPLOY", "owner": "codescene",
      "title": "Sondes /health et /alive exposées uniquement en Development",
      "glob": "src/**/*.cs", "multiline": true,
      "pattern": "IsDevelopment\\(\\)\\)\\s*\\{[^}]*MapHealthChecks",
      "advice": "Exposer les sondes en production (filtrage réseau ou port dédié) pour Kubernetes."
    },
```

agents/codescene.md (en-tête)

```
---
name: codescene
description: Gardien de la santé du code. Génère et maintient le référentiel Code Health local du projet (
     ↪ quality/code-health, sans clé d'API), scanne tout le code (.NET et Next.js), détecte et classe les
     ↪ bloqueurs (qualité, migration, déploiement, sécurité, frontend), corrige ceux de son périmètre et empê
     ↪ che toute régression. À utiliser pour l'audit initial, la revue post-refonte du code Next.js généré, et
     ↪ avant chaque commit sensible.
tools: Read, Grep, Glob, Edit, Write, Bash, Skill, TodoWrite, mcp__plugin_ecom-modernizer_code-health__*,
     ↪ mcp__plugin_ecom-modernizer_next-devtools__*, mcp__plugin_ecom-modernizer_context7__*
model: sonnet
effort: high
maxTurns: 120
memory: project
color: green
skills:
  - ecom-modernizer:code-health-referential
  - ecom-modernizer:blocker-taxonomy
  - ecom-modernizer:code-health-refactoring
  - ecom-modernizer:nextjs-code-health
  - ecom-modernizer:agent-report-contract
---
```

7.3 Agent Migration — architecte de modernisation

Mission

Conduire les deux chantiers de modernisation : **(1) migrer la persistance** d'EF Core InMemory vers PostgreSQL au moyen des intégrations Aspire, de migrations versionnées et d'un service de migration ; **(2) refondre le frontend** Blazor en une application Next.js iso-fonctionnelle, branchée sur la gateway.

TABLE 7.4 – Fiche signalétique de l'agent Migration

Phases	`migrate-db` (instance A) et `migrate-frontend` (instance B), en parallèle ; reprises pour correction (`SendMessage`)
Entrées	Handoff `codescene-to-migration.json`, code existant, contrat API

Sorties	Code migré, migrations EF, projet `ECommerce.MigrationService`, projet `ECommerce.Storefront`, rapports, handoff `migration-to-tests.json`, captures pour G3
Outils natifs	Read, Grep, Glob, Edit, Write, Bash, Skill, TodoWrite
Serveurs MCP	`postgres-admin` (`list_objects`, `get_object_details`, `execute_sql`, `explain_query`) ; `next-devtools` (`nextjs_docs`, `nextjs_index`, `nextjs_call`) ; `context7` ; `playwright` (captures)
Skills préchargés	`efcore-postgresql-migration`, `blazor-to-nextjs`, `agent-report-contract`
Commandes	`/ecom-modernizer:migrate-db`, `/ecom-modernizer:migrate-frontend`
Hooks applicables	`guard-paths` (`src/`, `docs/`), `guard-bash` (pas de cluster ni de push), `guard-mcp` (seul agent autorisé sur `postgres-admin` ; DROP/TRUNCATE soumis à confirmation), `post-edit-format` (Prettier, ESLint, `dotnet format`), `code-health-guard`
Mode d'exécution	One-shot par tâche ; boucle build/correction bornée (5 tentatives) ; reprise ciblée sur rétroaction
Gates	migrate-db : build vert, parité des endpoints = 1 ; migrate-frontend : build, parité des pages = 1, 0 erreur ESLint et TypeScript ; puis G2 et G3
Interdits	Cluster, push, suppression du projet Blazor avant G3, édition d'`AppHost.cs` par l'instance B

7.3.1 Tâche 1 — persistance PostgreSQL

1. **AppHost** : ressource PostgreSQL avec volume, deux bases, service de migration ; les API attendent la fin des migrations.
2. **Services** : `AddNpgsqlDbContext<T>` remplace `UseInMemoryDatabase` ; l'intégration Aspire apporte le contrôle de santé `CanConnectAsync`, les traces et les métriques Npgsql.
3. **Migrations** : `InitialCreate` par contexte ; suppression d'`EnsureCreated` (B-02).
4. **Seeding** : `UseAsyncSeeding` idempotent à la place de `HasData` à identifiants explicites (B-03).
5. **Vérification** par le MCP `postgres-admin` : tables, colonnes, index, comptages, plans d'exécution.

AppHost.cs — extrait cible

```
var postgres   = builder.AddPostgres("postgres").WithDataVolume().WithPgWeb();
var catalogDb  = postgres.AddDatabase("catalogdb");
var orderingDb = postgres.AddDatabase("orderingdb");

var migrations = builder.AddProject<Projects.ECommerce_MigrationService>("migrations")
    .WithReference(catalogDb).WithReference(orderingDb)
    .WaitFor(catalogDb).WaitFor(orderingDb);

var catalog = builder.AddProject<Projects.ECommerce_Catalog_Api>("catalog")
    .WithReference(catalogDb).WaitForCompletion(migrations);
// Catalog.Api/Program.cs : builder.AddNpgsqlDbContext<CatalogDbContext>("catalogdb");
```

7.3.2 Tâche 2 — refonte Blazor vers Next.js

TABLE 7.5 – Matrice de parité Blazor → Next.js

Blazor (existant)	Next.js (cible)
`Pages/Home.razor` (/)	`app/page.tsx`, Server Component
`Pages/Products.razor` : tableau, panier, commande	`app/products/page.tsx` + `components/cart.tsx` (Client Component) + Server Action `placeOrder`
`Pages/Orders.razor` + bouton Refresh	`app/orders/page.tsx` (rendu dynamique) + `revalidatePath('/orders')`
`NotFound.razor`, `Error.razor`	`app/not-found.tsx`, `app/error.tsx`
`MainLayout`, `NavMenu`	`app/layout.tsx`, `components/nav.tsx`
`CatalogApiClient`, `OrderingApiClient`	`lib/api.ts` (client typé généré depuis OpenAPI)
Session Blazor Server (SignalR)	Rendu serveur sans état ; panier côté client

Le contrat est généré au build (B-08) puis transformé en types TypeScript par `openapi-typescript`. Le storefront appelle la gateway **côté serveur** : l'URL est injectée par Aspire (variable `services__gateway__http__0`) ou par le chart Helm, jamais exposée au navigateur. L'intégration à l'AppHost utilise l'API JavaScript d'Aspire 13 (marquée expérimentale), qui lance `next dev` en local et génère à la publication un Dockerfile multi-étapes en sortie `standalone` :

AppHost.cs — ressource storefront (handoff appliqué par l'instance A)

```
#pragma warning disable ASPIREJAVASCRIPT001
builder.AddNextJsApp("storefront", "../ECommerce.Storefront")
    .WithReference(gateway).WaitFor(gateway)
    .WithExternalHttpEndpoints();
```

7.4 Agent Tests — ingénieur qualité

Mission

Prouver que l'application modernisée fonctionne : générer et exécuter les tests unitaires, d'intégration, système et end-to-end, **trier** chaque échec, réparer ses propres tests et **rédiger le rapport de tests**. L'agent ne corrige jamais le code applicatif : il le signale.

TABLE 7.6 – Fiche signalétique de l'agent Tests

Phases	`test` (local, après la revue) ; smoke sur k3d après déploiement ; relances après correctifs
Entrées	Handoff `migration-to-tests.json`, cible (`local` ou `k3d`), `BASE_URL`
Sorties	Projets `tests/*`, tests Vitest et Playwright, rapport Markdown, JUnit, couverture, rapport HTML Playwright, traces, `defects[]`
Outils natifs	Read, Grep, Glob, Edit, Write, Bash, Skill, TodoWrite

Serveurs MCP	`playwright` (`browser_navigate`, `browser_snapshot`, `browser_click`, `browser_fill_form`, `browser_take_screenshot`, `browser_generate_locator`, `browser_verify_*`) ; `postgres-ro` (`execute_sql` en lecture) ; `next-devtools` ; `kubernetes-ro` ; `context7`
Skills préchargés	`dotnet-testing`, `frontend-e2e-testing`, `test-report`, `agent-report-contract`
Commande	`/ecom-modernizer: test [unit|integration|e2e|all] [--target local|k3d]`
Hooks applicables	`guard-paths` (`tests/`, fichiers `*.test.ts(x)`, `*.spec.ts`, configurations de test), `guard-bash` (cluster en lecture seule), `guard-mcp` (pas d'accès aux instances en écriture)
Mode d'exécution	Boucle bornée « générer → exécuter → trier → réparer » (3 tours), puis rapport one-shot
Gate	Réussite ≥ 98 % ; couverture ≥ 80/70 % ; instabilité ≤ 2 % ; parcours critiques verts ; aucun défaut critique ou majeur

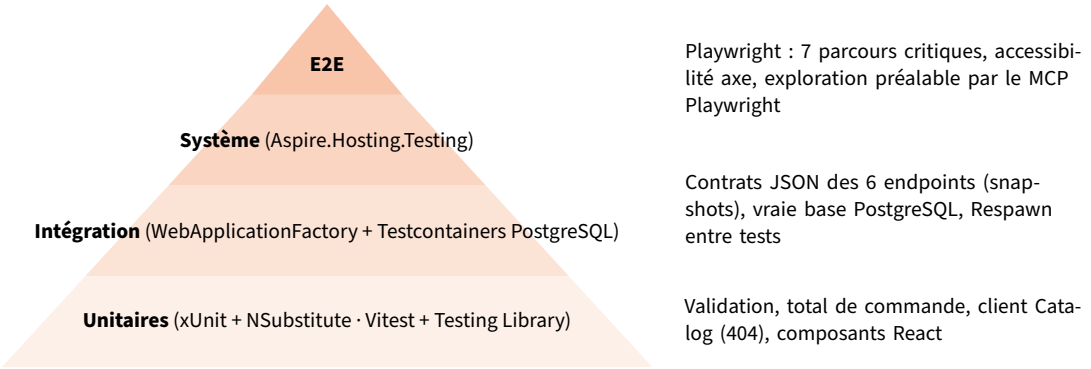


FIGURE 7.2 – Pyramide de tests générée par l'agent Tests

7.4.1 Playwright : le MCP pour explorer, les tests pour prouver

Le MCP Playwright sert à **explorer** l'application comme le ferait un testeur : l'agent navigue, lit l'arbre d'accessibilité (`browser_snapshot`), choisit des localisateurs robustes (rôles, libellés) et capture des preuves. Les parcours retenus sont ensuite écrits en **tests Playwright versionnés**, exécutables sans agent en CI. Les agents de test fournis par Playwright (`npx playwright init-agents --loop=claude` : planificateur, générateur, réparateur) peuvent accélérer cette étape ; l'agent Tests reste propriétaire du résultat.

7.4.2 Tri des échecs

Chaque test en échec est relancé trois fois. Un échec systématique dû à l'application devient un `app_defect` (avec preuve et propriétaire présumé, transmis à l'orchestrateur) ; un échec dû au test lui-même est réparé (`test_defect`) ; un échec intermittent est mis en quarantaine (`flaky`) ; un problème de port, de conteneur ou de cluster est un `env_defect` pour l'agent DevOps.

7.5 Agent DevOps — ingénieur plateforme

Mission

Rendre l'application **déployable, déployée et livrable en continu** : construire les images, créer le cluster k3d et son registre, déployer l'application refondue par Helm, vérifier le déploiement, **ouvrir l'application dans le navigateur** et mettre en place la chaîne **CI/CD**.

TABLE 7.7 – Fiche signalétique de l'agent DevOps

Phases	`deploy` (après la gate de tests) ; tâche `cicd` ; rollback sur décision G4 ; actions de game day à la demande
Entrées	Code validé, rapport de tests, `templates/k3d-cluster.yaml`
Sorties	Images, `deploy/k3d/`, `deploy/helm/ecommerce/`, `.github/workflows/`, rapport de déploiement, capture d'écran, handoff `devops-to-monitoring.json`
Outils natifs	Read, Grep, Glob, Edit, Write, Bash, Skill, TodoWrite
Serveurs MCP	`kubernetes` en écriture (`pods_list_in_namespace`, `pods_log`, `events_list`, `resources_create_or_update`, `helm_install`, `nodes_top`) ; `github` (`actions_list`, `actions_get`, `get_job_logs`, `actions_run_trigger`) ; `playwright` ; `grafana-ro` ; `context7`
Skills préchargés	`containerization`, `k3d-helm-deploy`, `github-actions-cicd`, `agent-report-contract`
Commandes	`/ecom-modernizer:deploy`, `/ecom-modernizer:open-app`, `/ecom-modernizer:ci`, `/ecom-modernizer:teardown`
Hooks applicables	`guard-bash` (contexte `k3d-ecommerce` obligatoire ; `ask` sur suppression de cluster, de namespace, de release, sur push et fusion), `guard-mcp` (seul agent sur l'instance Kubernetes en écriture), `guard-paths` (`deploy/`, `.github/`, Dockerfiles, AppHost)
Mode d'exécution	One-shot ; vérification du rollout bornée (3 tentatives) avec diagnostic et redéploiement
Gate	Pods prêts à 100 % ; `http://shop.localhost:8080` en HTTP 200 ; déploiement $\leq$ 10 min ; CI verte

7.5.1 Cluster et déploiement

Le cluster est décrit par un fichier de configuration k3d versionné (un serveur, deux agents, ports 8080/8443 sur le load balancer, registre local). Sa création est idempotente :

templates/k3d-cluster.yaml

```
# Cluster k3d de l'application ECommerce : k3d cluster create --config deploy/k3d/cluster.yaml
apiVersion: k3d.io/v1alpha5
kind: Simple
metadata:
  name: ecommerce
servers: 1
agents: 2
ports:
  - port: 8080:80          # Traefik HTTP : shop.localhost, grafana.localhost
    nodeFilters:
      - loadbalancer
```

```
  - port: 8443:443
    nodeFilters:
      - loadbalancer
registries:
  create:
    name: ecom-registry.localhost
    host: "0.0.0.0"
    hostPort: "5001"
options:
  k3d:
    wait: true
    timeout: "180s"
  kubeconfig:
    updateDefaultKubeconfig: true
    switchCurrentContext: true
```

Le chart `deploy/helm/ecommerce` contient PostgreSQL (StatefulSet et volume persistant), le job de migration (hook Helm `pre-install,pre-upgrade`), les quatre services applicatifs avec leurs sondes `/alive` et `/health` (rendues disponibles en production par la correction B-01), la découverte de services hors Aspire (`services__catalog__http__0=http://catalog:8080`) et l'export OTLP vers Alloy. Variante : faire générer le chart par Aspire (`AddKubernetesEnvironment`, `aspire publish`, Helm $\geq$ 4.2), puis le versionner et l'enrichir.

7.5.2 Ouverture dans le navigateur

Une fois le déploiement vérifié, l'agent ouvre l'application dans le navigateur de l'utilisateur (`open http://shop.localhost:8080` sous macOS, `xdg-open` sous Linux), puis rejoue un parcours de fumée avec le MCP Playwright en mode headless et dépose une capture dans le rapport. La commande `/ecom-modernizer:open-app` permet de rouvrir l'application à tout moment.

7.5.3 Chaîne CI/CD

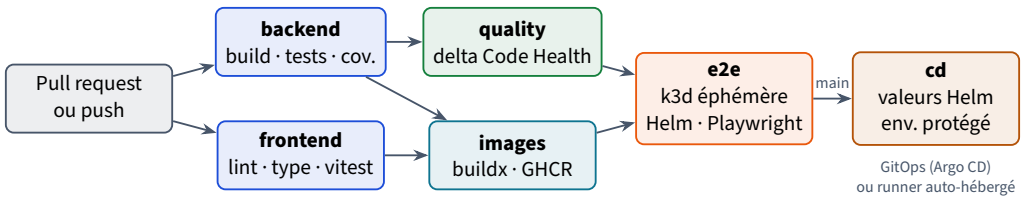


FIGURE 7.3 – Pipeline CI/CD généré par l'agent DevOps

Un runner hébergé par GitHub ne peut pas joindre un cluster k3d local : la CI crée donc un **cluster k3d éphémère** dans le runner pour les tests E2E, et la CD s'appuie sur un modèle *pull* (Argo CD dans le cluster local) ou sur un runner auto-hébergé. Extrait de `ci.yml` :

```
jobs:
  e2e:
    needs: [quality, images]
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: AbsaOSS/k3d-action@v2
        with: { cluster-name: ecommerce, args: --config deploy/k3d/cluster.yaml }
      - uses: azure/setup-helm@v5
      - run: helm upgrade --install ecommerce deploy/helm/ecommerce -n ecommerce --create-namespace --wait --
    ↪ timeout 10m
            --set global.imageTag=${{ github.sha }}
```

```
      - uses: actions/setup-node@v7
        with: { node-version: 22 }
      - run: npm ci && npx playwright install --with-deps chromium && BASE_URL=http://shop.localhost:8080 npx
    ↪ playwright test --grep @critical
        working-directory: src/ECommerce.Storefront
      - uses: actions/upload-artifact@v7
        if: always()
        with: { name: playwright-report, path: src/ECommerce.Storefront/playwright-report }
```

7.6 Agent Monitoring — ingénieur observabilité (SRE)

Mission

Rendre l'application **observable** et vérifier qu'elle tient ses **objectifs de niveau de service** : installer la stack, brancher la télémétrie, créer via le MCP Grafana les tableaux de bord, SLO et alertes, surveiller en continu et **trier les incidents**; superviser aussi le système multi-agents (AgentOps).

TABLE 7.8 – Fiche signalétique de l'agent Monitoring

Phases	`observe` (`setup`, one-shot, après le déploiement); `check` (récurrent); `triage` (sur alerte)
Entrées	Handoff `devops-to-monitoring.json`, cluster, Grafana
Sorties	`deploy/observability/`, instrumentation (`src/**/Telemetry/`, `instrumentation.ts`), tableaux de bord, règles d'alerte, rapports d'observabilité et d'incident
Outils natifs	Read, Grep, Glob, Edit, Write, Bash, Skill, TodoWrite
Serveurs MCP	`grafana` en écriture (`search_dashboards`, `update_dashboard`, `get_dashboard_summary`, `list_datasources`, `check_datasources_health`, `query_prometheus`, `query_loki_logs`, `query_loki_patterns`, `search_tempo_traces`, `get_tempo_trace`, `alerting_rules_read`, `alerting_rules_write`, `create_annotation`, `generate_deeplink`, `get_panel_image`); `kubernetes-ro`; `postgres-ro` (`analyze_db_health`, `get_top_queries`); `playwright`; `context7`
Skills préchargés	`observability-stack`, `grafana-dashboards-slo`, `incident-triage`, `agent-report-contract`
Commandes	`/ecom-modernizer:monitor setup|check|report|triage`, `/ecom-modernizer:monitor-check` (pour `/loop`)
Hooks applicables	`guard-bash` (Helm et kubectl en écriture uniquement dans `observability`), `guard-mcp` (seul agent sur l'instance Grafana en écriture), `guard-paths`
Mode d'exécution	`setup` one-shot; `check` récurrent (`/loop 15m`); `triage` événementiel (monitor `grafana-alerts`)
Gate	Sources saines; $\geq$ 5 tableaux de bord; $\geq$ 6 règles d'alerte; game day concluant

TABLE 7.9 – Tableaux de bord, SLO et alertes créés par l'agent Monitoring

Élément	Contenu
Vue métier	Commandes par minute, chiffre d'affaires par heure, panier moyen, échecs de validation Catalog
Services RED	Débit, taux d'erreurs 5xx, latence p50/p95/p99 par service (`http_server_request_duration_seconds`)
PostgreSQL	Connexions, transactions par seconde, requêtes les plus lentes
Kubernetes	Redémarrages, CPU et mémoire par pod, pods non prêts
AgentOps	Tokens et coûts (`claude_code.token.usage`, `claude_code.cost.usage`) par modèle et `query_source`, refus des hooks et échecs d'outils (journal collecté dans Loki)
SLO	Disponibilité du storefront 99,5 % sur 30 jours ; p95 de `POST /ordering/api/orders` < 800 ms ; succès des commandes $\geq$ 99 %
Alertes	HighErrorRate, HighLatencyP95, PodCrashLooping, CatalogDependencyFailures, SLOBurnRateFast (14,4× sur 1 h et 5 min), SLOBurnRateSlow (6× sur 6 h et 30 min)

Game day : prouver que la supervision fonctionne

L'agent Monitoring ne se contente pas de créer des alertes : il démontre qu'elles se déclenchent. Sur sa demande, l'orchestrateur fait réduire le service `catalog` à zéro réplica par l'agent DevOps (séparation des responsabilités), puis l'agent vérifie que l'alerte HighErrorRate passe à l'état *firing*, que le tri d'incident identifie la cause (événements Kubernetes, traces en erreur), et que l'alerte se résout après le retour à deux réplicas.

Télémétrie de Claude Code et confidentialité

L'export OpenTelemetry de Claude Code s'active par variables d'environnement (`CLAUDE_CODE_ENABLE_TELEMETRY=1`, `OTEL_METRICS_EXPORTER=otlp`, `OTEL_EXPORTER_OTLP_ENDPOINT`…) définies dans le shell ou les paramètres utilisateur : Claude Code ignore ces variables lorsqu'elles proviennent du `.claude/settings.json` d'un dépôt. Les noms d'agents personnalisés apparaissent sous la valeur générique `custom` dans l'attribut `agent.name` : la ventilation par agent s'obtient donc en croisant la télémétrie avec le journal `.ecom/journal.jsonl`.

8 Serveurs MCP

8.1 Le protocole MCP en bref

Le **Model Context Protocol** (MCP) standardise la manière dont une application d'IA — l'*hôte*, ici Claude Code — accède à des outils et à des données externes. L'hôte crée un *client* MCP par serveur ; chaque *serveur* expose des capacités typées. Les messages suivent JSON-RPC 2.0.

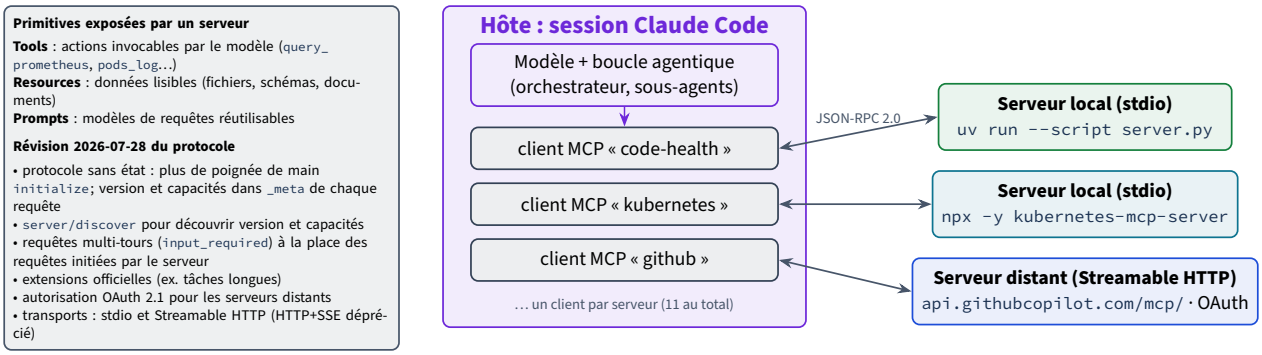


FIGURE 8.1 – Architecture MCP : un hôte, un client par serveur, des serveurs locaux ou distants

8.2 Intégration dans Claude Code et dans le plugin

- **Déclaration** : les serveurs du plugin sont déclarés dans `.mcp.json` à la racine du plugin. Ils apparaissent dans `/mcp` sous le nom `plugin:ecom-modernizer:<serveur>`.
- **Nommage des outils** : `mcp__plugin_ecom-modernizer_<serveur>__<outil>`. C'est ce nom qu'utilisent les listes `tools` des agents (`mcp__plugin_ecom-modernizer_grafana__*` accorde tout un serveur), les règles de permission et les matchers de hooks.
- **Variables** : `${user_config.<option>}` injecte une valeur de configuration (y compris un secret stocké au trousseau) dans `env`, `args` ou `headers` ; `${CLAUDE_PLUGIN_ROOT}` désigne les fichiers livrés avec le plugin ; `${CLAUDE_PLUGIN_DATA}` un répertoire persistant entre les mises à jour.
- **Chargement différé** : lorsque les définitions d'outils pèsent trop dans le contexte, Claude Code ne charge que leurs noms et récupère leur schéma à la demande, ce qui rend soutenable un plugin à onze serveurs.

⚖ Moindre privilège sans « mcpServers » par agent

Pour des raisons de sécurité, un sous-agent *livré dans un plugin* ne peut pas déclarer ses propres serveurs MCP (champ `mcpServers` ignoré), ni ses hooks, ni son mode de permission. Le moindre privilège est donc obtenu par trois mécanismes combinés : (1) des **instances séparées** en lecture seule (`postgres-ro`, `kubernetes-ro`, `grafana-ro`) et en écriture ; (2) la liste `tools` de chaque agent, qui n'accorde que les serveurs nécessaires ; (3) le hook `guard-mcp`, qui refuse l'usage d'une instance en écriture par un agent non propriétaire et impose une confirmation pour les outils destructeurs. La variante « agents éjectés » du chapitre 12 permet, au prix de la distribution, de déclarer des serveurs

propres à chaque agent.

8.3 Catalogue des serveurs

Pourquoi pas le serveur MCP officiel de CodeScene ?

Le serveur MCP et la CLI de CodeScene exigent un jeton d'accès (`CS_ACCESS_TOKEN`), y compris pour l'analyse locale ; la variante « standalone » requiert elle aussi une licence. Le projet impose une analyse **sans clé d'API** : la démarche Code Health est donc portée par un référentiel local généré dans le dépôt, dont le serveur MCP reprend les noms d'outils de CodeScene. Les prompts et skills de l'agent restent ainsi indépendants du moteur.

code-health — référentiel Code Health local (sans clé)

Implémentation	Serveur développé pour le projet (`server.py`, SDK MCP Python 2.x) au-dessus du moteur `code_health.py` ; généré par l'agent CodeScene dans `quality/code-health/`, modèle livré dans le plugin.
Lancement	`${CLAUDE_PLUGIN_ROOT}/scripts/code-health-mcp.sh` : lance `uv run --script quality/code-health/server.py`, ou le modèle du plugin tant que le référentiel n'est pas généré.
Configuration	Aucune clé : règles dans `rules.json`, dérogations dans `waivers.json`, baseline dans `baseline.json` ; dépendances (lizard, SDK MCP) résolues par `uv`.
Outils utilisés	`project_code_health`, `code_health_score`, `code_health_review`, `list_blockers`, `analyze_change_set`, `pre_commit_code_health_safeguard`, `list_hotspots`, `rules_config_list_thresholds`, `explain_code_health` — noms alignés sur ceux du serveur CodeScene.
Consommateurs	CodeScene (principal) ; la CLI du même moteur sert aux hooks et à la CI.
Sécurité	Outils en lecture seule ; chemins limités à la racine du projet ; exécution 100 % locale.

postgres-admin et postgres-ro — PostgreSQL

Implémentation	Postgres MCP Pro (`crystaldba/postgres-mcp`).
Lancement	`uvx postgres-mcp --access-mode=unrestricted` (admin) et `--access-mode=restricted` (lecture seule : transactions en lecture, temps d'exécution limité).
Configuration	`DATABASE_URI` ← `${user_config.postgres_admin_uri}` ou `postgres_ro_uri`. Analyse d'index : extensions `pg_stat_statements` et `hypopg`.
Outils utilisés	`list_schemas`, `list_objects`, `get_object_details`, `execute_sql`, `explain_query`, `get_top_queries`, `analyze_workload_indexes`, `analyze_query_indexes`, `analyze_db_health`.
Consommateurs	Migration (admin, développement) ; Tests et Monitoring (lecture seule).
Sécurité	`guard-mcp` : admin réservé à Migration, DROP/TRUNCATE soumis à confirmation ; jamais d'URI d'administration vers le cluster.

next-devtools — Next.js

Implémentation	`next-devtools-mcp` (Vercel), passerelle vers le point de terminaison MCP intégré au serveur de développement Next.js 16 (`/_next/mcp`).
Lancement	`npx -y next-devtools-mcp@latest` (stdio) ; Node.js ≥ 20.19.
Configuration	`NEXT_TELEMETRY_DISABLED=1`.
Outils utilisés	`nextjs_index` (découverte des serveurs de dev), `nextjs_call` (outils d'exécution : `get_errors`, `get_logs`, `get_page_metadata`, `get_project_metadata`, `get_server_action_by_id`), `nextjs_docs` (documentation de la version installée).
Consommateurs	Migration (refonte), CodeScene (revue), Tests (diagnostic E2E).
Sécurité	Lecture seule sur un serveur de développement local.

playwright — navigateur

Implémentation	Playwright MCP (`@playwright/mcp`, Microsoft) : pilotage par l'arbre d'accessibilité, sans modèle de vision.
Lancement	`npx -y @playwright/mcp@latest --headless --isolated --caps testing, devtools --output-dir ${CLAUDE_PLUGIN_DATA}/playwright`.
Configuration	Durcissement possible : `--allowed-origins` limité à `shop.localhost` et `localhost`.
Outils utilisés	`browser_navigate`, `browser_snapshot`, `browser_click`, `browser_type`, `browser_fill_form`, `browser_select_option`, `browser_wait_for`, `browser_take_screenshot`, `browser_console_messages`, `browser_network_requests`, `browser_generate_locator`, `browser_verify_text_visible`, `browser_start_tracing`.
Consommateurs	Tests (principal), DevOps (smoke et capture), Migration (captures de parité), Monitoring (contrôle synthétique).
Sécurité	Profil isolé ; contenu des pages traité comme donnée non fiable (risque d'injection de prompt).

kubernetes et kubernetes-ro — cluster k3d

Implémentation	`kubernetes-mcp-server` (projet `containers`), client natif de l'API Kubernetes.
Lancement	`npx -y kubernetes-mcp-server@latest --config ${CLAUDE_PLUGIN_ROOT}/config/kubernetes-rw.toml` (ou `kubernetes-ro.toml`).
Configuration	Fichiers TOML : `read_only = true|false`, `toolsets = ["core", "config", "helm"]` ; kubeconfig courant.
Outils utilisés	`namespaces_list`, `pods_list_in_namespace`, `pods_get`, `pods_log`, `pods_top`, `nodes_top`, `events_list`, `resources_list`, `resources_get`, `resources_create_or_update`, `resources_scale`, `helm_install`, `helm_list`, `helm_uninstall`.
Consommateurs	DevOps (écriture) ; Monitoring, Tests, Orchestrateur (lecture seule).
Sécurité	Contexte `k3d-ecommerce` vérifié par hook ; suppressions soumises à confirmation ; namespace `kube-system` interdit.

github — dépôt et CI

Implémentation	Serveur MCP officiel de GitHub (distant ; variante locale `ghcr.io/github/github-mcp-server`).
Lancement	Streamable HTTP : `https://api.githubcopilot.com/mcp/`.
Configuration	En-tête `Authorization: Bearer ${user_config.github_token}` (PAT à granularité fine) ; jeux d'outils `repos`, `pull_requests`, `actions`, `issues` ; option `--read-only` en local.
Outils utilisés	`actions_list` (workflows et runs), `actions_get` (détail d'un run, artefacts), `actions_run_trigger`, `get_job_logs`, `create_pull_request`, `pull_request_read`, `list_pull_requests`, `merge_pull_request`.
Consommateurs	DevOps (CI/CD) ; Tests et CodeScene (commentaires de PR) ; Monitoring (tickets d'incident).
Sécurité	Fusion soumise à G5 ; déclenchement de workflow réservé à DevOps.

grafana et grafana-ro — observabilité

Implémentation	Serveur MCP officiel de Grafana (`mcp-grafana`).
Lancement	`uvx mcp-grafana` (stdio) ; `--disable-write` pour l'instance en lecture seule.
Configuration	`GRAFANA_URL` ← `${user_config.grafana_url}` ; `GRAFANA_SERVICE_ACCOUNT_TOKEN_FILE` → fichier écrit par l'agent Monitoring après création du compte de service (relu à chaque requête, ce qui permet de démarrer le serveur avant que Grafana n'existe).
Outils utilisés	`search_dashboards`, `get_dashboard_by_uid`, `get_dashboard_summary`, `update_dashboard`, `list_datasources`, `check_datasources_health`, `query_prometheus`, `list_prometheus_metric_names`, `query_prometheus_histogram`, `query_loki_logs`, `query_loki_patterns`, `search_tempo_traces`, `get_tempo_trace`, `alerting_rules_read`, `alerting_rules_write`, `create_annotation`, `generate_deeplink`, `get_panel_image`.
Consommateurs	Monitoring (écriture) ; DevOps et Orchestrateur (lecture seule).
Sécurité	Compte de service au rôle Editor limité au dossier « ECommerce » ; suppressions soumises à confirmation.

context7 — documentation à jour

Implémentation	Context7 (`@upstash/context7-mcp`).
Lancement	`npx -y @upstash/context7-mcp` (stdio) ou serveur distant.
Configuration	Aucune.
Outils utilisés	`resolve-library-id`, `query-docs`.
Consommateurs	Tous les agents techniques (EF Core et Npgsql, Aspire, Next.js, Playwright, Helm, Grafana…).
Sécurité	Lecture seule ; limite les erreurs dues à des API obsolètes.

8.4 Matrice agents × serveurs

TABLE 8.1 – Accès des agents aux serveurs MCP (✎ écriture, 👁 lecture, — aucun)

Serveur	Orch.	CodeScene	Migration	Tests	DevOps	Monitoring
code-health	—	👁	—	—	—	—
postgres-admin	—	—	✎	—	—	—
postgres-ro	—	—	—	👁	—	👁
next-devtools	—	👁	👁	👁	—	—
playwright	—	—	👁	✎	👁	👁
kubernetes	—	—	—	—	✎	—
kubernetes-ro	👁	—	—	👁	—	👁
github	—	—	—	—	✎	—
grafana	—	—	—	—	—	✎
grafana-ro	👁	—	—	—	👁	—
context7	—	👁	👁	👁	👁	👁

Pour Playwright, « écriture » signifie que l'agent Tests remplit des formulaires et passe des commandes de test ; les autres agents se limitent à la navigation et aux captures.

8.5 Configuration du plugin

.mcp.json

```
{
  "mcpServers": {
    "code-health": {
      "command": "${CLAUDE_PLUGIN_ROOT}/scripts/code-health-mcp.sh"
    },
    "postgres-admin": {
      "command": "uvx",
      "args": ["postgres-mcp", "--access-mode=unrestricted"],
      "env": { "DATABASE_URI": "${user_config.postgres_admin_uri}" }
    },
    "postgres-ro": {
      "command": "uvx",
      "args": ["postgres-mcp", "--access-mode=restricted"],
      "env": { "DATABASE_URI": "${user_config.postgres_ro_uri}" }
    },
    "next-devtools": {
      "command": "npx",
      "args": ["-y", "next-devtools-mcp@latest"],
      "env": { "NEXT_TELEMETRY_DISABLED": "1" }
    },
    "playwright": {
      "command": "npx",
      "args": ["-y", "@playwright/mcp@latest", "--headless", "--isolated",
               "--caps", "testing,devtools",
               "--output-dir", "${CLAUDE_PLUGIN_DATA}/playwright"]
    },
    "kubernetes": {
      "command": "npx",
```

```
      "args": ["-y", "kubernetes-mcp-server@latest",
               "--config", "${CLAUDE_PLUGIN_ROOT}/config/kubernetes-rw.toml"]
    },
    "kubernetes-ro": {
      "command": "npx",
      "args": ["-y", "kubernetes-mcp-server@latest",
               "--config", "${CLAUDE_PLUGIN_ROOT}/config/kubernetes-ro.toml"]
    },
    "github": {
      "type": "http",
      "url": "https://api.githubcopilot.com/mcp/",
      "headers": { "Authorization": "Bearer ${user_config.github_token}" }
    },
    "grafana": {
      "command": "uvx",
      "args": ["mcp-grafana"],
      "env": {
        "GRAFANA_URL": "${user_config.grafana_url}",
        "GRAFANA_SERVICE_ACCOUNT_TOKEN_FILE": "${CLAUDE_PLUGIN_DATA}/grafana-sa.token"
      }
    },
    "grafana-ro": {
      "command": "uvx",
      "args": ["mcp-grafana", "--disable-write"],
      "env": {
        "GRAFANA_URL": "${user_config.grafana_url}",
        "GRAFANA_SERVICE_ACCOUNT_TOKEN_FILE": "${CLAUDE_PLUGIN_DATA}/grafana-sa.token"
      }
    },
    "context7": {
      "command": "npx",
      "args": ["-y", "@upstash/context7-mcp"]
    }
  }
}
```

8.6 Sécurité des serveurs MCP

- **Injection de prompt indirecte** : le contenu d'une page web, d'un log Loki ou d'un commentaire de PR peut contenir des instructions malveillantes. Les agents traitent les résultats d'outils comme des *données* ; les actions sensibles restent bornées par les hooks, quelle que soit l'instruction reçue.
- **Chaîne d'approvisionnement** : en CI, les versions des serveurs lancés par `npx` ou `uvx` sont figées (`@playwright/mcp@<version>` plutôt que `@latest`).
- **Secrets** : uniquement par `userConfig` sensible ou fichier de jeton hors dépôt ; jamais dans `.mcp.json` ni dans un prompt (le hook `prompt-guard` bloque un jeton collé dans la conversation).
- **Réseau** : serveurs locaux en stdio ; seul le serveur GitHub est distant, en HTTPS avec authentification.

9 Hooks, skills, commandes et utilitaires

9.1 Hooks : la couche de garanties

Principe

Les prompts suggèrent, les hooks garantissent. Tout ce qui doit être vrai *à chaque fois* — périmètre d'écriture, contexte Kubernetes, confirmation de l'irréversible, rapport de fin de tâche, plafond de boucles — est codé dans des hooks déterministes. Un agent ne peut ni les contourner, ni les modifier (`.claude/` et les fichiers du plugin sont hors de son périmètre).

Les sous-agents livrés par un plugin ignorent le champ `hooks` de leur en-tête : les hooks sont donc déclarés au niveau du plugin (`hooks/hooks.json`) et s'appliquent à toute la session, sous-agents compris. Les règles *propres à un agent* sont obtenues en lisant le champ `agent_type` de l'entrée JSON (par exemple `ecom-modernizer:tests`), présent dans tous les événements émis à l'intérieur d'un sous-agent ou d'une session lancée avec `--agent`.

9.1.1 Contrat d'entrée et de sortie

TABLE 9.1 – Contrat d'un hook de type commande

Élément	Description
Entrée (stdin)	JSON : `session_id`, `cwd`, `hook_event_name`, `permission_mode`, `agent_id` et `agent_type` (dans un sous-agent), `tool_name`, `tool_input` (événements d'outil), `source` (SessionStart), `stop_hook_active`, `background_tasks`, `last_assistant_message` (Stop)
Code 0	Succès ; la sortie standard est lue comme JSON si c'est un objet unique
Code 2	Blocage : l'action est empêchée (PreToolUse) ou le retour est transmis au modèle (PostToolUse) ; stderr sert de motif
Autre code	Erreur non bloquante (l'action se poursuit)
PreToolUse	`hookSpecificOutput.permissionDecision` : `allow`, `deny`, `ask` ou `defer` (priorité : deny > defer > ask > allow), avec `permissionDecisionReason` ; `updatedInput` pour réécrire l'appel
Stop, SubagentStop	`{"decision": "block", "reason": "..."}` : l'agent ne s'arrête pas et reçoit la raison comme prochaine instruction
SessionStart, SubagentStart	`hookSpecificOutput.additionalContext` : texte injecté dans le contexte
Tous	`systemMessage` (avertissement à l'utilisateur), `continue: false` (arrêt complet)

9.1.2 Catalogue des hooks du plugin

TABLE 9.2 – Les 16 hooks du plugin `ecom-modernizer`

Id	Événement	Script	Rôle	Effet
H01	SessionStart (`startup|resume|clear|compact`)	`session-bootstrap.sh`	Crée `.ecom/`, détecte les prérequis manquants, réinjecte l'état du run (notamment après compaction)	Contexte
H02	UserPromptSubmit	`prompt-guard.sh`	Refuse un message contenant un jeton ou une clé privée	Blocage
H03	PreToolUse `Agent`	`gate-agents.sh`	Verrou d'activation des sous-agents ; préconditions de phase (`phase:` du contrat)	deny
H04	PreToolUse `Bash`	`guard-bash.sh`	Interdits globaux, confirmations de l'irréversible, règles par agent, contexte `k3d-ecommerce`	deny / ask
H05	PreToolUse `Bash`, `if: Bash(git commit *)`	`precommit-safeguard.sh`	Refuse un commit qui dégrade la Code Health des fichiers indexés (moteur local, `safeguard`)	deny
H06	PreToolUse `Write|Edit|NotebookEdit`	`guard-paths.sh`	Fichiers protégés (secrets, `.claude/`, `state.json`) et périmètre d'écriture par agent	deny / ask
H07	PreToolUse `mcp__plugin_ecom-modernizer_.*`	`guard-mcp.sh`	Instances MCP en écriture réservées à leur propriétaire ; outils destructeurs confirmés	deny / ask
H08	PostToolUse `Write|Edit`	`post-edit-format.sh`	Prettier, ESLint (Next.js), `dotnet format` (C#)	Retour (code 2)
H09	PostToolUse `Write|Edit`	`code-health-guard.sh`	Note du fichier par le moteur local ; signale une baisse ou un passage sous le seuil	Retour (code 2)
H10	PostToolUseFailure	`tool-failure.sh` (async)	Journalise les échecs d'outils (AgentOps)	Journal
H11	SubagentStart	`subagent-start.sh`	Marqueur de lancement, injection du cadre (run, phase, rapport attendu, règles)	Contexte
H12	SubagentStop	`subagent-stop.sh`	Exige un rapport JSON valide (2 relances max), enregistre le rapport dans l'état	block
H13	Stop	`orchestrator-stop.sh`	Boucle d'orchestration bornée : relance tant que le run est actif, sans travail en fond ni absence de progrès	block
H14	PreCompact	`checkpoint.sh`	Copie de l'état avant compaction	Journal
H15	Notification	`notify.sh` (async)	Notification de bureau quand une validation humaine est attendue	—
H16	SessionEnd	`session-end.sh`	Met un run interrompu en pause pour une reprise propre	État

9.1.3 Cycle de vie d'une délégation

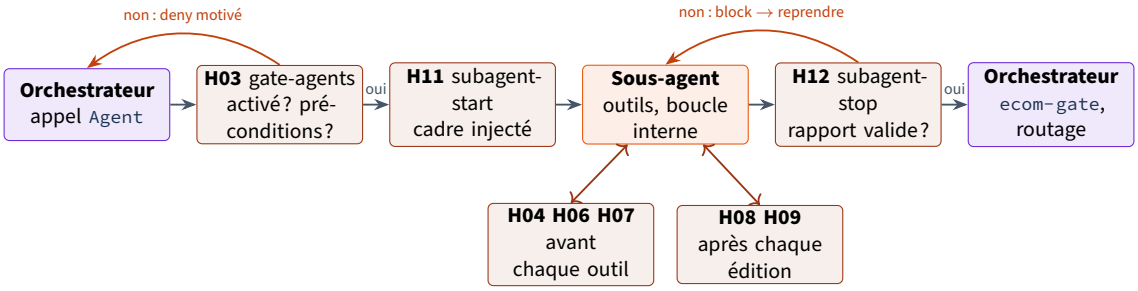


FIGURE 9.1 – Hooks traversés lors d'une délégation de l'orchestrateur à un sous-agent

9.1.4 Vérification des hooks

Les scripts ont été exécutés hors de Claude Code avec des entrées JSON simulées, dans un dépôt factice. Les résultats obtenus sont conformes aux attentes ; ils constituent la base des tests automatisés des hooks (chapitre 13).

TABLE 9.3 – Cas de test des hooks et résultats observés

Hook	Cas	Résultat
prompt-guard	Message contenant un jeton `ghp_...`	block ✔
gate-agents	`devops` lancé alors que seuls `codescene,tests` sont activés	deny ✔
gate-agents	`codescene` activé	autorisé ✔
gate-agents	Orchestrateur lancé comme sous-agent	deny ✔
gate-agents	Phase `test` alors que `review` est `pending`	deny motivé ✔
guard-bash	`rm -rf /` ; `curl ... | bash` ; `git push --force`	deny ✔
guard-bash	`git push origin feature/x` ; `k3d cluster delete ecommerce`	ask ✔
guard-bash	`kubectl get pods --context minikube`	deny ✔
guard-bash	Tests : `helm install` ; Orchestrateur : `dotnet build` ; Monitoring : Helm hors `observability`	deny ✔
guard-bash	DevOps : `docker build`	autorisé ✔
guard-paths	Tests écrit dans `src/.../Program.cs` ; DevOps écrit dans `src/.../Program.cs`	deny ✔
guard-paths	Tests écrit dans `tests/...` ; DevOps dans `deploy/helm/...` ; Monitoring dans `src/.../Telemetry/...`	autorisé ✔
guard-paths	CodeScene écrit `.env` ; Orchestrateur écrit `.ecom/state.json`	deny ✔
guard-mcp	Tests utilise `kubernetes` (écriture)	deny ✔
guard-mcp	DevOps `resources_delete` ; Migration `DROP DATABASE`	ask ✔
subagent-stop	Fin de tâche sans rapport, puis avec rapport valide	block, puis autorisé ✔

Hook	Cas	Résultat
orchestrator-stop	Run actif sans tâche de fond ; sous-agent en fond ; absence de progrès ; gate en attente	block ; arrêt ; message ; arrêt ✔
ecom-gate	Rapport de tests conforme ; couverture 0,6 et défaut majeur	vert ; rouge (2 motifs) ✔
code-health-guard	Fichier sain (10,0) puis ajout d'une méthode complexe (8,3)	code 0, puis code 2 ✔
precommit-safeguard	Commit d'un fichier dégradé (4 nouveaux smells)	deny motivé ✔

9.1.5 Variantes de hooks

Claude Code propose cinq types de hooks : `command`, `http`, `mcp_tool`, `prompt` et `agent` (expérimental). Le plugin s'appuie sur des hooks `command`, déterministes et testables ; les autres types sont des extensions possibles :

- **`mcp_tool`** : appeler directement un outil MCP, par exemple `code_health_review` sur le fichier modifié. Le résultat textuel est interprété comme la sortie standard d'un hook commande : utile pour l'audit, mais un blocage fiable suppose un objet JSON de décision, d'où le choix d'un script autour de la CLI du moteur local.
- **`prompt`** : faire juger un rapport par un petit modèle (`{"ok": false, "reason": "..."}`), par exemple « le rapport de tests contient-il une preuve pour chaque défaut ? » en complément de la validation structurelle de H12.
- **`/goal`** : variante interactive de H13 ; une condition (« le run est à l'état done ») évaluée par un modèle après chaque tour. H13 est préféré car son critère est déterministe (`state.json`).

Exemple de hook mcp_tool (variante non retenue par défaut)

```
{ "type": "mcp_tool", "server": "plugin:ecom-modernizer:code-health",
  "tool": "code_health_review", "input": { "file_path": "${tool_input.file_path}" } }
```

9.2 Skills : les savoir-faire

Un skill est un répertoire contenant un `SKILL.md` (en-tête YAML et instructions) et, au besoin, des fichiers d'appui (références, modèles, scripts). Seule sa description est présente en permanence dans le contexte ; son contenu est chargé lorsqu'il est utile (*divulgation progressive*). Deux usages coexistent :

- **préchargement** : la liste `skills` de l'en-tête d'un agent injecte le contenu complet au démarrage du sous-agent — c'est le « manuel de métier » de l'agent ;
- **chargement à la demande** : tout agent peut invoquer un skill du plugin par l'outil `Skill`.

Les skills de connaissance portent `user-invocable: false` (absents du menu `/`) ; aucun ne porte `disable-model-invocation: true`, qui empêcherait leur préchargement.

TABLE 9.4 – Catalogue des 18 skills

Skill	Agent(s)	Contenu
`pipeline-orchestration`	Orchestrateur	Machine à états, DAG et parallélisme, contrat de tâche, table de routage, gates humaines, mode CI
`run-reporting`	Orchestrateur	Modèle et règles du rapport final consolidé
`agent-report-contract`	tous	Schéma du rapport JSON, statuts, défauts, décisions requises, handoffs
`code-health-referential`	CodeScene	Génération et usage du référentiel local : structure, modèle de notation, outils MCP, CLI, évolution des règles sous G1
`blocker-taxonomy`	CodeScene	Sévérités B0–B3, catégories, propriétaires, catalogue des bloqueurs connus (B-01 à B-10) et typiques de Next.js
`code-health-refactoring`	CodeScene	Boucle de refactoring guidée par la Code Health, correspondance smells → patrons C#
`nextjs-code-health`	CodeScene	Diagnostic à chaud (next-devtools), règles App Router, sécurité, accessibilité, typage
`efcore-postgresql-migration`	Migration	AppHost PostgreSQL, Npgsql EF Core, migrations, service de migration, seeding, vérifications
`blazor-to-nextjs`	Migration	Matrice de parité, contrat OpenAPI, structure du projet, intégration Aspire, stratégie strangler
`dotnet-testing`	Tests	Projets de tests, cas minimaux, Testcontainers, Respawn, Aspire.Hosting.Testing, couverture
`frontend-e2e-testing`	Tests	Exploration MCP, 7 parcours critiques, conventions Playwright, agents Playwright
`test-report`	Tests	Modèle du rapport, tri des échecs, fiche de défaut
`containerization`	DevOps	Publication conteneur .NET, Dockerfile Next.js standalone, tags et registre
`k3d-helm-deploy`	DevOps	Cluster k3d, chart Helm, sondes, découverte de services, variante `aspire publish`
`github-actions-cicd`	DevOps	Jobs CI, CD GitOps ou runner auto-hébergé, suivi par MCP, action Claude Code
`observability-stack`	Monitoring	Installation Helm, télémétrie OTLP, métriques métier, compte de service Grafana
`grafana-dashboards-slo`	Monitoring	5 tableaux de bord, PromQL de référence, SLO, alertes, game day
`incident-triage`	Monitoring	Runbook en 7 étapes : qualifier, localiser, expliquer, contextualiser, corréler, recommander, rapporter

skills/efcore-postgresql-migration/SKILL.md

```
---
name: efcore-postgresql-migration
description: Procédure de migration EF Core InMemory vers PostgreSQL dans une solution .NET Aspire (intégrations
    ↪  Aspire.Hosting.PostgreSQL et Aspire.Npgsql.EntityFrameworkCore.PostgreSQL, migrations, service de
    ↪ migration, seeding, vérification). À utiliser pour la tâche migrate-db.
user-invocable: false
---

# AppHost

```csharp
var postgres = builder.AddPostgres("postgres").WithDataVolume().WithPgWeb();
var catalogDb = postgres.AddDatabase("catalogdb");
var orderingDb = postgres.AddDatabase("orderingdb");
var migrations = builder.AddProject<Projects.ECommerce_MigrationService>("migrations")
    .WithReference(catalogDb).WithReference(orderingDb)
    .WaitFor(catalogDb).WaitFor(orderingDb);
var catalog = builder.AddProject<Projects.ECommerce_Catalog_Api>("catalog")
    .WithReference(catalogDb).WaitForCompletion(migrations);
```

Services

Remplacer `UseInMemoryDatabase(...)` par `builder.AddNpgsqlDbContext<CatalogDbContext>("catalogdb");`
(santé `CanConnectAsync`, traces et métriques Npgsql fournies par l'intégration). Supprimer `EnsureCreated()`.

Migrations et seeding

- `dotnet ef migrations add InitialCreate --project src/ECommerce.Catalog.Api` (idem Ordering).
- Service `ECommerce.MigrationService` : `Database.MigrateAsync()` puis arrêt ; en Kubernetes, Job Helm `pre-
    ↪ install,pre-upgrade`.
- Seeding : `UseAsyncSeeding` (EF Core 9+) idempotent ; éviter `HasData` avec identifiants explicites
  (la séquence d'identité n'avance pas → violation de clé au premier `POST`).
- `DateTimeOffset` → `timestamptz` : n'écrire que des valeurs UTC.

Vérification (MCP postgres-admin)

`list_objects` (tables attendues), `get_object_details` (colonnes, index, FK `OrderId`),
`execute_sql` (comptages du seed), `explain_query` sur `GET /api/orders` (Include des lignes).
```

9.3 Commandes : les points d'entrée

Les commandes sont des fichiers Markdown du dossier `commands/`, invoqués sous la forme `/ecom-modernizer:<nom>`. Leur en-tête précise la description, l'indication d'arguments, les outils pré-autorisés et, pour celles qui ont des effets de bord, `disable-model-invocation: true` (seul l'humain peut les lancer). Elles exploitent `$ARGUMENTS` et l'injection de contexte dynamique (`!‘commande‘`), qui exécute par exemple `ecom-state summary` avant que le modèle ne lise la commande.

TABLE 9.5 – Catalogue des 17 commandes

Commande	Agent	Effet	Qui
`init`	Orchestrateur	Prépare le dépôt : git, `.ecom/config.json`, `.gitignore`, `.claude/settings.json` proposé	H
`doctor`	Orchestrateur	Prérequis et état des serveurs MCP	H, M
`plan`	Orchestrateur	Plan d'exécution sans exécution (DAG, gates, durées)	H, M

Commande	Agent	Effet	Qui
`run`	Orchestrateur	Lance un run (profil, `--agents`, `--skip`, `--from`, `--until`, `--dry-run`)	H
`status`	Orchestrateur	État du run, journal récent, gates en attente	H, M
`resume`	Orchestrateur	Reprise d'un run en pause ou interrompu	H
`report`	Orchestrateur	Rapport final consolidé	H, M
`scan`	CodeScene	Référentiel, audit et correction (`--init`, `--scope`, `--fix`, `--baseline`, `--pass`, `--hotspots`)	H
`migrate-db`	Migration	Migration PostgreSQL	H
`migrate-frontend`	Migration	Refonte Next.js	H
`test`	Tests	Campagne de tests (`unit|integration|e2e|all`, `--target`)	H
`deploy`	DevOps	Images, cluster, déploiement (`--recreate`, `--strategy`)	H
`open-app`	DevOps	Ouvre l'application dans le navigateur	H, M
`ci`	DevOps	Génère ou suit la CI/CD (`--generate`, `--status`)	H
`teardown`	DevOps	Supprime le cluster (double confirmation)	H
`monitor`	Monitoring	`setup|check|report|triage`	H
`monitor-check`	Monitoring	Contrôle de santé rapide, conçu pour `/loop`	H, M

H : invocable par l'humain ; M : invocable par le modèle.

⚠ Une commande lancée par /loop doit rester invocable par le modèle

`/loop 15m /ecom-modernizer:monitor-check` ne fonctionne que si `monitor-check` *n'a pas* `disable-model-invocation: true` : une exécution planifiée ne lance que les skills et commandes que Claude a le droit d'invoquer lui-même. C'est pourquoi `monitor-check` est séparée de `monitor`.

commands/run.md

```
---
description: Lance le pipeline multi-agents (orchestrateur) pour un profil ou un sous-ensemble d'agents
argument-hint: "[--profile full|quality|modernize|ship|observe|ci] [--agents a,b] [--skip a,b] [--from phase]
    ↪ [--until phase] [--dry-run]"
disable-model-invocation: true
allowed-tools: Read Grep Glob Bash(ecom-state *) Bash(ecom-doctor *) Bash(ecom-gate *) Bash(git status *) Bash(
    ↪ git tag *)
---

# Lancement d'un run ecom-modernizer

État actuel :
!`"${CLAUDE_PLUGIN_ROOT}/bin/ecom-state" summary`

Arguments reçus : `$ARGUMENTS`

Tu agis comme **orchestrateur** (si la session n'a pas été lancée avec
`claude --agent ecom-modernizer:orchestrator`, applique strictement le skill
`ecom-modernizer:pipeline-orchestration`, qui décrit le même protocole).

1. Résous la sélection : `--agents`/`--skip` > `--profile` > profil par défaut de `.ecom/config.json`
   (créé par `/ecom-modernizer:init`) > `userConfig.default_profile`. Calcule les phases retenues,
```

```
   en respectant `--from`/`--until`.
2. Vérifie les prérequis avec `ecom-doctor --json`.
3. Présente le plan (DAG, parallélisme, gates humaines, agents désactivés et conséquences).
   Avec `--dry-run`, arrête-toi ici.
4. Demande la gate **G0** (validation du plan), puis `ecom-state init <run_id> <profil> <agents> <phases>`
   avec `run_id = run-<AAAAMMJJ-HHMM>`.
5. Déroule les phases selon le protocole (délégation, contrôle `ecom-gate`, routage, checkpoints).
```

9.4 Utilitaires, workflows et monitor

TABLE 9.6 – Utilitaires du dossier bin (dans le PATH des agents)

Utilitaire	Rôle et sous-commandes
`ecom-state`	Unique point d'écriture de `.ecom/state.json` (verrou et écriture atomique) : `init`, `status`, `phase`, `gate`, `report`, `waive`, `loop`, `checkpoint` (tag git), `get`, `summary`, `active`
`ecom-gate`	Évalue la quality gate d'une phase à partir du rapport et des seuils ; JSON `{phase, pass, reasons}` et code de sortie
`ecom-doctor`	Vérifie git, jq, .NET, Node, Docker (et son démon), k3d, kubectl, Helm, uv, gh ; sortie tabulaire ou `--json`

Workflows. `workflows/modernize.js` enchaîne l'audit, les deux migrations en parallèle (`pipeline()` sur deux tâches), l'application du handoff AppHost et la revue ; `workflows/ship.js` enchaîne les tests, le déploiement, puis en parallèle le smoke test sur le cluster et l'installation de la supervision. Chaque appel `agent()` exige un résultat conforme à un schéma JSON (`status`, `summary`, `report_path`). La spécialisation passe par les skills que l'agent du workflow charge en premier. Ils s'invoquent comme des commandes (`/ecom-modernizer:modernize`) et se reprennent après une pause sans rejouer les agents terminés.

Monitor. `monitors/monitors.json` déclare `grafana-alerts`, un processus de fond démarré au premier `monitor-check` : il interroge l'API d'alertes de Grafana chaque minute et imprime une ligne par nouvelle alerte. Chaque ligne parvient à Claude comme une notification, qui déclenche le tri d'incident sans attendre la prochaine itération de `/loop` (fonction expérimentale, sessions interactives uniquement).

10 Scénario de référence et déroulement détaillé

Ce chapitre décrit *qui fait quoi et à quel moment* sur un run complet (scénario S1, profil `full`), puis les scénarios alternatifs. Les durées sont indicatives (poste de référence, hors temps d'attente humaine).

10.1 Vue d'ensemble du scénario S1 « modernisation complète »

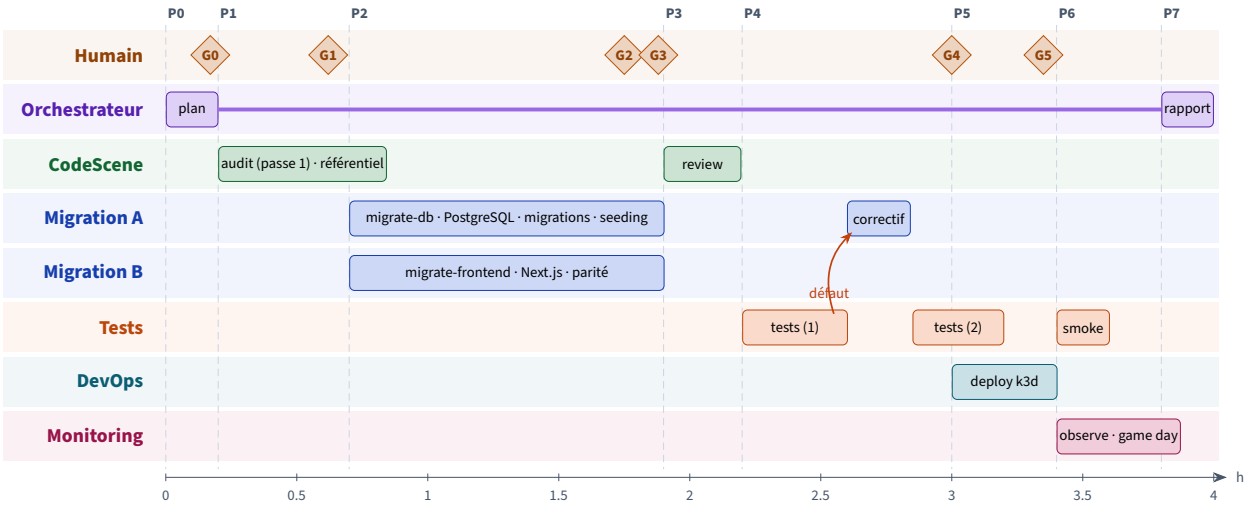


FIGURE 10.1 – Chronogramme du scénario S1 : couloirs par agent, phases P0 à P7, gates et rétroaction

TABLE 10.1 – Phases du scénario S1

Ph.	Objet	Acteur(s)	Résultat attendu	Gate
P0	Préparation et plan	Humain Orchestrateur	Plugin configuré, dépôt initialisé, prérequis vérifiés, plan validé	G0
P1	Audit et assainissement	CodeScene	Référentiel Code Health généré, bloqueurs classés, B0/B1 du périmètre corrigés, handoff vers Migration	G1
P2	Modernisation	Migration ×2	PostgreSQL et migrations ; storefront Next.js iso-fonctionnel	G2 G3
P3	Revue post-refonte	CodeScene	Code Next.js et code modifié revus, aucune régression	—
P4	Validation	Tests (+ Migration)	Campagne complète, défauts corrigés par rétroaction, rapport de tests	—
P5	Industrialisation	DevOps	Images, cluster k3d, déploiement Helm, application ouverte, CI/CD	G4 G5

Ph.	Objet	Acteur(s)	Résultat attendu	Gate
P6	Observabilité	Monitoring ‖ Tests	Stack, tableaux de bord, SLO, alertes, game day ; smoke sur le cluster	—
P7	Clôture	Orchestrateur	Rapport final, checkpoint `ecom/<run>/report`	—

10.2 Déroulement phase par phase

P0 — Préparation et plan

1. Humain installe le plugin (`claude plugin install ecom-modernizer@ecom-agents --scope project`) et renseigne la configuration (`/plugin configure`) : jeton GitHub, URI PostgreSQL. Aucune clé n'est demandée pour l'analyse de code.
2. Humain ouvre une session dédiée : `claude --agent ecom-modernizer:orchestrator`, puis `/ecom-modernizer:init` (dépôt git, tag `ecom/baseline`, `.ecom/config.json`, `.claude/settings.json` proposé).
3. Humain lance `/ecom-modernizer:run --profile full`. Le hook `SessionStart` a déjà injecté l'état et les prérequis manquants ; Orchestrateur exécute `ecom-doctor --json`, construit le DAG et présente le plan.
4. G0 : Humain valide. Orchestrateur exécute `ecom-state init run-20261021-1430 full ...` et pose le tag de départ.

P1 — Audit et assainissement (CodeScene, passe 1)

1. Orchestrateur délègue la phase `audit` (contrat de tâche) ; `gate-agents` vérifie l'activation, `SubagentStart` injecte le cadre.
2. CodeScene génère le référentiel `quality/code-health/`, mesure l'existant (moyenne 9,95) et applique les règles de bloqueurs : 11 occurrences (B-01 à B-09) ; sa revue ajoute B-05, B-07 et B-10.
3. CodeScene corrige les bloqueurs de son périmètre : sondes de santé disponibles en production (B-01), validation des requêtes (B-04, B-07), redirection HTTPS paramétrable (B-09). Chaque édition passe par `code-health-guard`.
4. Les bloqueurs MIGRATION (B-02, B-03, B-08) partent dans `codescene-to-migration.json` ; B-05 (réservation du stock) est une décision métier.
5. G1 : Humain décide pour B-05 (par exemple « implémenter la validation de quantité, reporter la réservation de stock ») ; la décision est tracée. Commit, refus éventuel par `precommit-safeguard`, tag `ecom/<run>/audit`.

P2 — Modernisation (Migration, deux instances en parallèle)

1. Orchestrateur lance simultanément l'instance A (`migrate-db`) et l'instance B (`migrate-frontend`).
2. **Instance A** : PostgreSQL dans l'AppHost, `AddNpgsqlDbContext`, migrations `InitialCreate`, `ECommerce.MigrationService`, `UseAsyncSeeding` ; vérification par le MCP `postgres-admin`.

3. **Instance B** : OpenAPI au build, types TypeScript, `src/ECommerce.Storefront`, matrice de parité, captures ; la ressource `storefront` est décrite dans `frontend-apphost.json`.
4. À la fin de B, **Orchestrateur** transmet le handoff à A (`SendMessage`), qui l'applique à `AppHost.cs`.
5. `ecom-gate migrate-db` et `ecom-gate migrate-frontend` ; G2 (schéma) puis G3 (parité de l'UI).

P3 — Revue post-refonte (CodeScene, passe 2)

CodeScene exécute `analyze_change_set` depuis `ecom/<run>/audit`, revoit l'intégralité du storefront (`project_code_health`, `nextjs_call get_errors`, règles FE-01 à FE-03) et corrige les écarts de son périmètre (par exemple une page entière marquée `'use client'`). Gate : aucune régression, moyenne $\geq$ 9,0.

P4 — Validation (Tests, avec rétroaction)

Tests génère les tests, explore l'application avec le MCP Playwright, exécute la campagne et trie les échecs. Dans l'exemple de la figure 10.2, le parcours P4 révèle qu'une commande de quantité nulle est encore acceptée côté storefront : le défaut est attribué à l'instance B, corrigé, puis le parcours est rejoué.

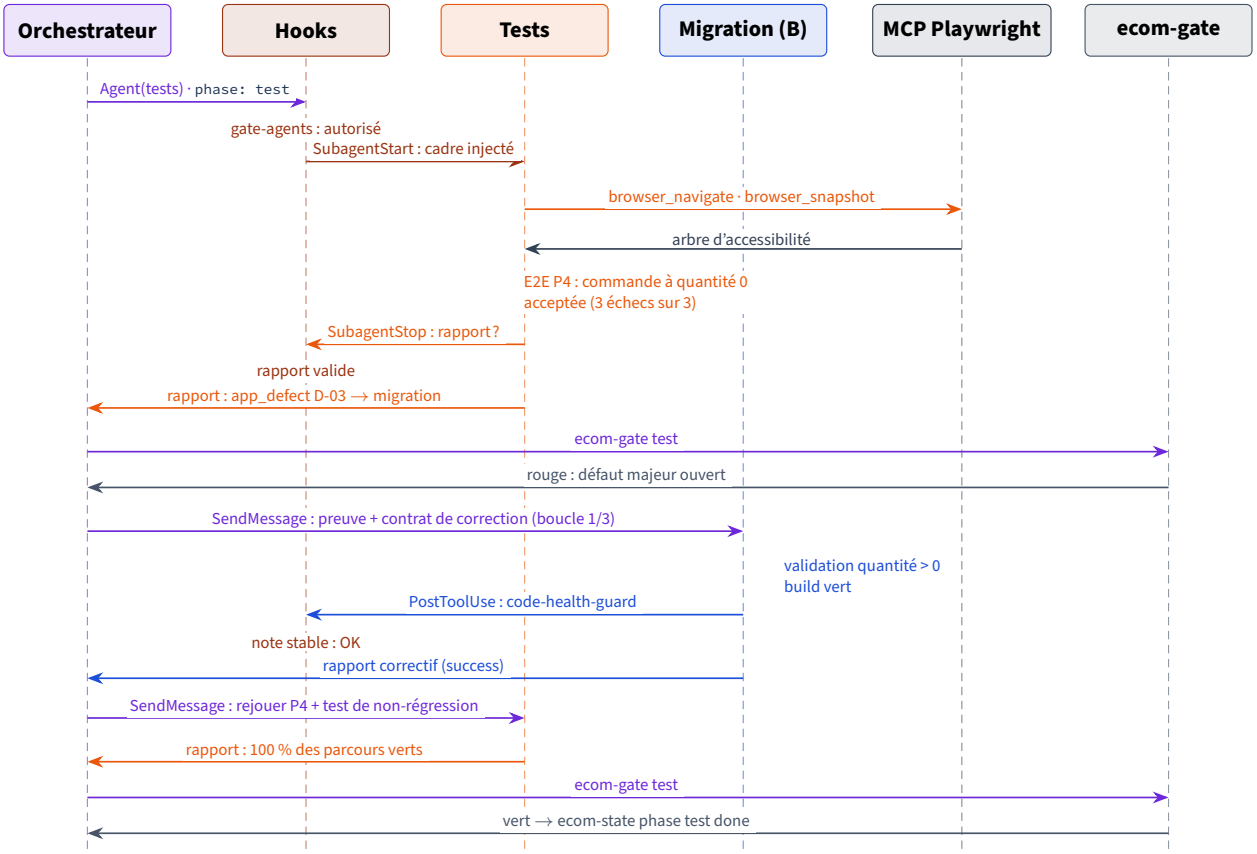


FIGURE 10.2 – Séquence d'une boucle de rétroaction : test en échec, correctif, nouvelle exécution

P5 — Industrialisation (DevOps)

1. G4 : autorisation de déployer la version `<sha>` sur `k3d-ecommerce`.

2. DevOps construit les images, crée le cluster (idempotent), pousse dans le registre local, installe le chart (job de migration, puis services), vérifie le rollout avec le MCP Kubernetes.
3. DevOps ouvre `http://shop.localhost:8080` dans le navigateur de l'utilisateur, rejoue un parcours de fumée (MCP Playwright) et dépose une capture.
4. DevOps génère `ci.yml` et `cd.yml` sur une branche ; G5 avant le push et l'ouverture de la pull request ; suivi des runs avec le MCP GitHub jusqu'au vert.

P6 — Observabilité (Monitoring, en parallèle du smoke test)

Monitoring installe la stack, branche la télémétrie, crée les tableaux de bord, SLO et alertes par le MCP Grafana, pose une annotation de déploiement, puis mène le game day (catalog à zéro réplica via DevOps, alerte déclenchée puis résolue). En parallèle, Tests rejoue les parcours critiques sur le cluster. Humain peut lancer `/loop 15m /ecom-modernizer:monitor-check` pour la surveillance continue.

P7 — Clôture

Orchestrateur rédige `docs/reports/<run>-final.md` : synthèse, indicateurs de chaque gate, décisions humaines, défauts résiduels, coûts (tokens par phase), risques ; puis `ecom-state status done` et le tag final.

10.3 Qui fait quoi et à quel moment

TABLE 10.2 – Matrice RACI (R réalise, A approuve, C consulté, I informé)

Activité	Hum.	Orch.	CS	Mig.	Tst	Ops	Mon.
Plan d'exécution (P0)	A	R	I	I	I	I	I
Référentiel Code Health, audit (P1)	I	A	R	C	I	I	—
Décision métier B-05	A	R	C	C	—	—	—
Migration PostgreSQL (P2)	A	C	C	R	C	—	—
Refonte Next.js (P2)	A	C	C	R	C	—	—
Revue post-refonte (P3)	I	A	R	C	I	—	—
Tests et rapport (P4)	I	A	—	C	R	C	—
Correctifs de défauts (P4)	I	A	C	R	C	—	—
Déploiement k3d (P5)	A	C	—	—	C	R	I
CI/CD et pull request (P5)	A	C	C	—	C	R	—
Observabilité, game day (P6)	I	A	—	—	C	C	R
Tri d'incident et rollback	A	R	—	C	—	R	R
Rapport final (P7)	A	R	C	C	C	C	C

10.4 Scénarios alternatifs

TABLE 10.3 – Scénarios alternatifs

Scénario	Lancement	Déroulement
S2 — Qualité seule	`/ecom-modernizer:run --profile quality`	CodeScene puis Tests sur le code existant ; utile avant toute modernisation ou en revue de dette
S3 — Livraison	`--profile ship`	DevOps, Tests (smoke), Monitoring sur une application déjà migrée ; précondition « tests verts » vérifiée par dérogation explicite si la phase n'a pas eu lieu dans ce run
S4 — Incident	monitor `grafana-alerts` ou `/loop`	Alerte → Monitoring (triage, rapport d'incident) → G4 → DevOps (rollback Helm) → Tests (smoke)
S5 — Pull request	GitHub Actions + `claude-code-action`	Profil `ci` non interactif : delta Code Health et tests sur les fichiers modifiés, commentaires sur la PR, gates en échec fermé
S6 — Reprise	`/ecom-modernizer:resume`	Après une fin de session ou une compaction : état relu, dernière phase terminée retrouvée par son tag, gate en attente reposée
S7 — Sous-ensemble	`--agents codescene,tests`	Seules les phases de ces agents sont planifiées ; toute délégation à un autre agent est refusée par `gate-agents`

11 Modèle d'exécution : one-shot ou boucle ?

Réponse courte

Vu de l'orchestrateur, chaque sous-agent est appelé en **one-shot** : un contrat de tâche en entrée, un rapport en sortie. À l'intérieur, chaque sous-agent exécute des **boucles de convergence bornées** (corriger jusqu'au vert, dans une limite d'itérations). L'orchestrateur, lui, tourne en **boucle d'orchestration** jusqu'à l'état `done` ou une gate humaine, sous plafond. Seule la supervision tourne en **boucle récurrente**. Rien ne boucle sans condition d'arrêt, plafond et détection d'absence de progrès.

11.1 Quatre niveaux de boucles

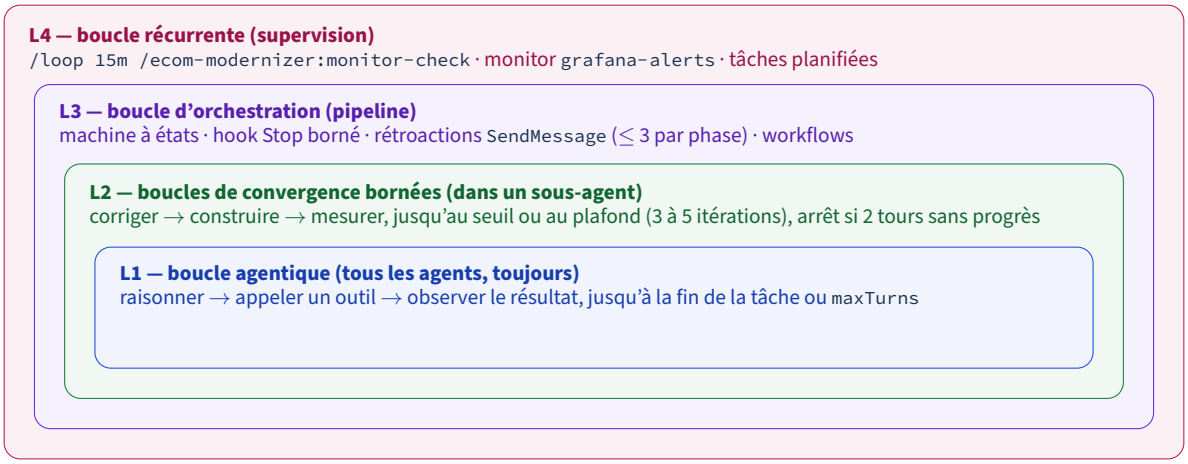


FIGURE 11.1 – Les quatre niveaux de boucles du système

L1 — boucle agentique
Inhérente à tout agent Claude : tant que la tâche n'est pas terminée, le modèle choisit un outil, observe le résultat et décide de la suite. Elle est bornée par `maxTurns` (sortie marquée partielle, reprise possible).

L2 — boucles de convergence
Propres à chaque sous-agent : correction jusqu'au vert (CodeScene, Migration), génération-exécution-réparation (Tests), vérification du rollout (DevOps). Elles portent un prédicat de succès, un plafond et un critère d'absence de progrès.

L3 — boucle d'orchestration
Portée par l'orchestrateur : tant que le run est actif, lancer la phase prête suivante, évaluer la gate, renvoyer les corrections. Le hook `Stop` empêche un arrêt prématuré ; les plafonds empêchent l'emballement.

L4 — boucle récurrente
La supervision réexécute un contrôle court à intervalle régulier ou sur événement.

11.2 Mode d'exécution par agent

TABLE 11.1 – Mode d'exécution de chaque agent

Agent	Invocation	Boucle interne	Arrêt	Reprise
Orchestrateur	Session longue	L3 : phases, gates, rétroactions	État `done`, gate en attente, blocage, plafond de 30 relances	`resume`
CodeScene	One-shot par passe	L2 : corriger → construire → noter	Fichier ≥ seuil et 0 B0/B1, ou 3 itérations, ou 2 sans progrès	SendMessage
Migration	One-shot par tâche (2 instances)	L2 : construire → corriger	Build vert et parité, ou 5 tentatives	SendMessage
Tests	One-shot par campagne	L2 : générer → exécuter → trier → réparer	Tous les tests triés, ou 3 tours de réparation	SendMessage
DevOps	One-shot par déploiement	L2 : déployer → vérifier → diagnostiquer	Pods prêts et HTTP 200, ou 3 tentatives	SendMessage
Monitoring (setup)	One-shot	L2 : configurer → vérifier	Critères de la gate observe	SendMessage
Monitoring (check)	Récurrent (L4)	aucune	Contrôle de 5 lignes ; tri si anomalie	`/loop`
Monitoring (triage)	Événementiel	L1	Hypothèse prouvée ou escalade	—

11.3 Mécanismes de la plateforme

TABLE 11.2 – Mécanismes de boucle disponibles et leur usage

Mécanisme	Fonctionnement	Usage dans le projet
`maxTurns`	Nombre maximal de tours d'un sous-agent ; au-delà, sortie partielle reprenable	Plafond de L1 (120 à 200 selon l'agent)
Hook `SubagentStop`	`decision: block` : la raison devient la prochaine instruction du sous-agent	Rapport obligatoire (2 relances maximum)
Hook `Stop`	Empêche l'arrêt du thread principal ; Claude Code plafonne nativement à 8 relances consécutives sans appel d'outil	Boucle L3 avec plafond de 30 relances par run et détection d'absence de progrès (empreinte de l'état)
`SendMessage`	Reprend une instance de sous-agent avec tout son historique	Rétroactions ciblées (correctifs, compléments)

Mécanisme	Fonctionnement	Usage dans le projet
`/loop` et `CronCreate`	Prompt ou commande réexécuté à intervalle fixe ou choisi par Claude ; tâches de session, expirant après 7 jours	`monitor-check` toutes les 15 minutes
Monitor de plugin	Processus de fond ; chaque ligne imprimée notifie Claude (sessions interactives)	Alertes Grafana → tri immédiat
Workflow	Script JavaScript : boucles, parallélisme, schémas de sortie ; reprise sans rejouer les agents terminés	Phases répétables `modernize` et `ship`
`/goal`	Condition évaluée par un modèle après chaque tour	Variante interactive de L3 (non retenue par défaut)

scripts/orchestrator-stop.sh — garde-fous de la boucle L3 (extrait)

```
BG="$(j '.background_tasks | length')"                    # un sous-agent travaille : on attend sa notification
[ -n "$BG" ] && [ "$BG" != "0" ] && exit 0
case "$STATUS" in paused_hitl|blocked|done|failed|paused) exit 0 ;; esac
if [ "$(j '.stop_hook_active')" = "true" ] && [ "$HASH" = "$(cat "$HASH_FILE")" ]; then
  ...  # aucune progression depuis la dernière relance : on rend la main à l'humain
fi
[ "$COUNT" -ge "$MAX" ] && ecom-state status paused        # plafond de relances atteint
block_stop "Run ... toujours actif (phase : ...). Applique le protocole."
```

11.4 Garde-fous contre l'emballement

- **Plafonds explicites** dans `.ecom/config.json` (`loops.*`) et `maxTurns` par agent.
- **Absence de progrès** : deux itérations sans amélioration du critère (note, tests verts, pods prêts) arrêtent une boucle L2 ; une empreinte inchangée de `state.json` arrête la relance L3.
- **Idempotence** : relancer une étape ne casse rien (création de cluster conditionnelle, `helm upgrade --install`, migrations EF, `update_dashboard` par identifiant).
- **Budget** : tokens et coûts suivis par phase (AgentOps) ; l'orchestrateur peut abaisser le modèle d'un contrôle (`haiku` pour `monitor-check`).
- **Échec fermé** : en CI, une gate non auto-approuvée met le run en pause au lieu de boucler.

11.5 Pourquoi ne pas tout mettre en boucle

Une boucle permanente sur chaque agent multiplierait les coûts, rendrait le comportement non déterministe et diluerait la responsabilité (qui a changé quoi, et quand ?). Le choix « one-shot à l'extérieur, boucles bornées à l'intérieur » garde trois propriétés : chaque délégation est **auditable** (un contrat, un rapport, un tag git), **reprenable** (état persistant) et **chiffrable** (budget par phase). La récurrence est réservée à ce qui est intrinsèquement continu : la surveillance d'une application en fonctionnement.

12 Packaging en plugin et utilisation

12.1 Structure du plugin

Tout le système — orchestrateur, sous-agents, skills, commandes, hooks, serveurs MCP, workflows, utilitaires, modèles et évaluations — est livré dans un plugin Claude Code unique, `ecom-modernizer`, publié dans la marketplace `ecom-agents`. Le squelette de référence passe `claude plugin validate --strict`.

```
ecom-agents-marketplace/
├── .claude-plugin/marketplace.json
└── plugins/ecom-modernizer/
    ├── .claude-plugin/plugin.json .... manifeste, userConfig
    ├── agents/ ..... 6 agents
    ├── commands/ ..... 17 commandes
    ├── skills/ ..... 18 skills
    ├── hooks/hooks.json ..... 16 hooks
    ├── scripts/ ..... scripts des hooks, lanceur MCP
    ├── .mcp.json ..... 11 serveurs MCP
    ├── config/ ..... profils, schéma, TOML
    ├── bin/ ..... ecom-state, ecom-gate, ecom-doctor
    ├── templates/ ..... contrat, rapports, k3d
    │   └── code-health/ ..... modèle du référentiel
    ├── workflows/ ..... modernize, ship
    ├── monitors/monitors.json
    └── evals/ ..... cas d'évaluation, mocks
```

Noms exposés : les agents deviennent `ecom-modernizer:<agent>`, les commandes `/ecom-modernizer:<commande>`, les workflows `/ecom-modernizer:modernize` et `/ecom-modernizer:ship`, les outils MCP `mcp__plugin_ecom-modernizer_<serveur>__<outil>`.
Variables : `${CLAUDE_PLUGIN_ROOT}` (fichiers du plugin), `${CLAUDE_PLUGIN_DATA}` (données persistantes : jeton Grafana, sorties Playwright), `${CLAUDE_PROJECT_DIR}` (dépôt).
Exécutables : le dossier `bin/` est ajouté au PATH du shell des agents tant que le plugin est actif.

12.2 Manifeste et configuration utilisateur

.claude-plugin/plugin.json

```
{
  "name": "ecom-modernizer",
  "displayName": "E-Commerce Modernizer",
  "version": "1.0.0",
  "description": "Système multi-agents orchestré : audit Code Health (référentiel local, sans clé), migration
    ↪ PostgreSQL et Next.js, tests, déploiement k3d + CI/CD, supervision Grafana",
  "author": {
    "name": "Équipe Capstone ECommerce"
  },
  "license": "MIT",
  "keywords": ["multi-agents", "orchestration", "codescene", "postgresql", "nextjs", "playwright", "k3d", "
    ↪ grafana", "mcp"],
  "userConfig": {
    "default_profile": {
      "type": "string",
      "title": "Profil par défaut",
      "description": "Profil utilisé par /ecom-modernizer:run quand aucun --profile n'est passé",
      "options": ["full", "quality", "modernize", "ship", "observe", "ci"],
      "default": "full"
    },
```

```
    "enabled_agents": {
      "type": "string",
      "multiple": true,
      "title": "Sous-agents activés",
      "description": "Sous-ensemble autorisé parmi codescene, migration, tests, devops, monitoring (vérifié par
     ↪ le hook gate-agents)",
      "default": ["codescene", "migration", "tests", "devops", "monitoring"]
    },
    "github_token": {
      "type": "string",
      "title": "Jeton GitHub",
      "description": "PAT fine-grained (repo, actions, pull requests) pour le serveur MCP GitHub",
      "sensitive": true
    },
    "postgres_admin_uri": {
      "type": "string",
      "title": "URI PostgreSQL (administration, dev uniquement)",
      "description": "Ex. postgresql://postgres:***@localhost:5432/catalogdb — utilisée par l'agent Migration",
      "sensitive": true
    },
    "postgres_ro_uri": {
      "type": "string",
      "title": "URI PostgreSQL (lecture seule)",
      "description": "Compte en lecture seule utilisé par les agents Tests et Monitoring",
      "sensitive": true
    },
    "grafana_url": {
      "type": "string",
      "title": "URL Grafana",
      "description": "URL de Grafana exposée par l'ingress du cluster k3d",
      "default": "http://grafana.localhost:8080"
    },
    "kube_context": {
      "type": "string",
      "title": "Contexte Kubernetes attendu",
      "description": "Seul contexte kubectl/helm autorisé par le hook guard-bash",
      "default": "k3d-ecommerce"
    }
  }
}
```

Le bloc `userConfig` est présenté à l'activation du plugin (`/plugin configure`) : les valeurs sensibles (jeton GitHub, URI PostgreSQL) sont stockées dans le trousseau du système, jamais dans `settings.json` ; elles sont injectées dans `.mcp.json` par `${user_config.<option>}` et dans les hooks par les variables `CLAUDE_PLUGIN_OPTION_<OPTION>`. **Aucune clé n'est demandée pour l'analyse de code**, assurée par le référentiel Code Health local.

12.3 Marketplace et distribution

.claude-plugin/marketplace.json

```
{
  "name": "ecom-agents",
  "description": "Marketplace du Capstone : système multi-agents orchestré pour moderniser, tester, déployer et
     ↪ superviser l'application ECommerce",
  "owner": {
    "name": "Équipe Capstone ECommerce"
  },
  "plugins": [
    {
      "name": "ecom-modernizer",
      "source": "./plugins/ecom-modernizer",
      "description": "Orchestrateur + 5 sous-agents (CodeScene, Migration, Tests, DevOps, Monitoring) avec
     ↪ skills, commandes, hooks et serveurs MCP"
```

```
    }
  ]
}
```

La marketplace est un dépôt git (ou un dossier local). Une nouvelle version du plugin se publie en incrémentant `version` dans `plugin.json` ; les utilisateurs la reçoivent par `claude plugin update`.

12.4 Installation

```
# Développement local, sans marketplace
claude --plugin-dir "./ecom-agents-marketplace/plugins/ecom-modernizer"

# Vérification structurelle (CI du plugin)
claude plugin validate --strict ./ecom-agents-marketplace/plugins/ecom-modernizer

# Installation depuis la marketplace (dossier local ou dépôt git)
claude plugin marketplace add ./ecom-agents-marketplace        # ou : claude plugin marketplace add <org>/ecom-
    ↪ agents-marketplace
claude plugin install ecom-modernizer@ecom-agents --scope project
# puis, dans une session : /plugin configure ecom-modernizer@ecom-agents
```

Pour qu'une équipe dispose automatiquement du plugin en ouvrant le dépôt, le `.claude/settings.json` du projet déclare la marketplace et active le plugin (chaque membre l'installe au premier lancement, après avoir accepté la confiance du dossier) :

```
{
  "extraKnownMarketplaces": {
    "ecom-agents": { "source": { "source": "github", "repo": "<org>/ecom-agents-marketplace" } }
  },
  "enabledPlugins": { "ecom-modernizer@ecom-agents": true },
  "permissions": {
    "allow": ["Bash(dotnet build *)", "Bash(dotnet test *)", "Bash(npm run *)", "Bash(ecom-state *)",
              "mcp__plugin_ecom-modernizer_code-health__*", "mcp__plugin_ecom-modernizer_context7__*"],
    "ask":   ["Bash(git push *)", "Bash(k3d cluster delete *)", "Bash(helm uninstall *)"]
  }
}
```

12.5 Modes d'utilisation

TABLE 12.1 – Modes d'utilisation du plugin

Mode	Lancement	Quand l'utiliser
Session dédiée	`claude --agent ecom-modernizer:orchestrator` puis `/ecom-modernizer:run`	Mode nominal : restrictions d'outils de l'orchestrateur, gates interactives
Commande	`/ecom-modernizer:run` dans une session ordinaire	Usage ponctuel ; protocole chargé par le skill `pipeline-orchestration`
Agent isolé	`/ecom-modernizer:scan`, `:test`, `:deploy`… ou `@agent-ecom-modernizer:tests`	Travail ciblé sans run complet ; hooks toujours actifs

Mode	Lancement	Quand l'utiliser
Workflow	`/ecom-modernizer:modernize`, puis `/ecom-modernizer:ship`	Exécution codifiée et parallèle ; gates entre les deux workflows
Non interactif	`claude -p "/ecom-modernizer:run --profile ci" --agent ecom-modernizer:orchestrator --output-format stream-json`	Scripts, CI ; gates en échec fermé
GitHub Actions	`anthropics/claude-code-action@v1` avec `plugin_marketplaces` et `plugins`	Revue automatique des pull requests (profil `ci`)
Agent teams (variante)	`CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1` ; coéquipiers créés à partir des définitions d'agents	Exploration parallèle (revue croisée, débogage par hypothèses) ; expérimental, sans reprise des coéquipiers

.github/workflows/claude-pr.yml — revue des pull requests par le plugin (extrait)

```yaml
on: { pull_request: { types: [opened, synchronize] } }
jobs:
  review:
    runs-on: ubuntu-latest
    permissions: { contents: read, pull-requests: write, id-token: write }
    steps:
      - uses: actions/checkout@v7
      - uses: anthropics/claude-code-action@v1
        with:
          anthropic_api_key: ${{ secrets.ANTHROPIC_API_KEY }}
          plugin_marketplaces: "https://github.com/<org>/ecom-agents-marketplace.git"
          plugins: "ecom-modernizer@ecom-agents"
          prompt: "/ecom-modernizer:run --profile ci"
          claude_args: "--agent ecom-modernizer:orchestrator --max-turns 60"
```

12.6 Activer tous les sous-agents ou un sous-ensemble

L'activation se règle à cinq niveaux, du plus large au plus fin. Les niveaux 1 à 4 *sélectionnent* ; le niveau 5 *verrouille* : un agent exclu ne peut pas être lancé, même si un modèle le tentait.

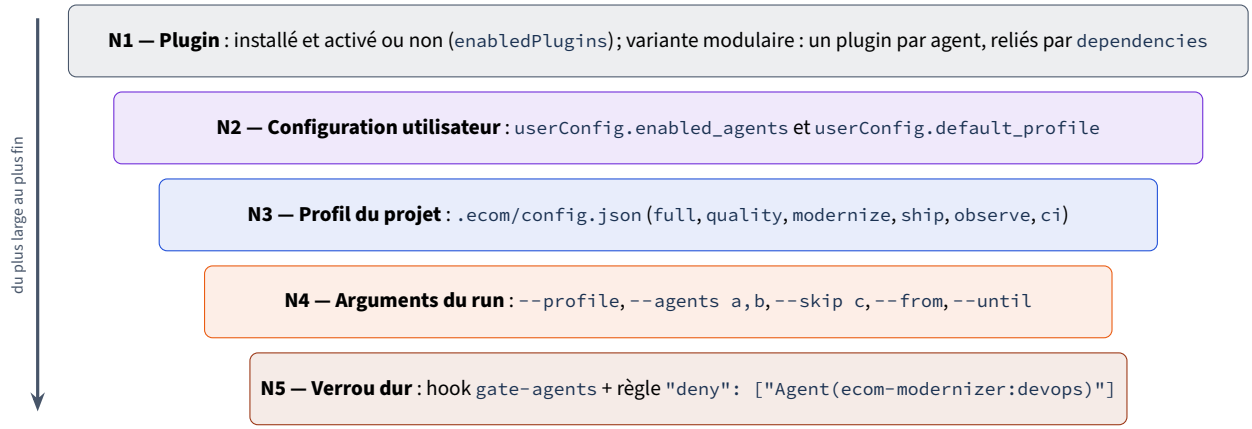


FIGURE 12.1 – Cinq niveaux d'activation des sous-agents

TABLE 12.2 – Profils prédéfinis

Profil	Agents	Phases
`full`	les cinq	preflight, audit, migrate-db, migrate-frontend, review, test, deploy, observe, report
`quality`	CodeScene, Tests	preflight, audit, test, report
`modernize`	CodeScene, Migration, Tests	preflight, audit, migrate-db, migrate-frontend, review, test, report
`ship`	DevOps, Tests, Monitoring	preflight, deploy, test (smoke), observe, report
`observe`	Monitoring	preflight, observe, report
`ci`	CodeScene, Tests	audit (fichiers modifiés), test ; non interactif

```
/ecom-modernizer:run                                  # profil par défaut (full) : les cinq sous-agents
/ecom-modernizer:run --profile quality                # CodeScene + Tests
/ecom-modernizer:run --agents codescene,migration     # sous-ensemble explicite
/ecom-modernizer:run --profile full --skip monitoring # tout sauf la supervision
/ecom-modernizer:run --from deploy --until observe    # reprendre une portion du pipeline
/ecom-modernizer:plan --agents devops,monitoring      # vérifier un plan sans l'exécuter
```

Préconditions d'un sous-ensemble. Lorsqu'un agent est exclu, sa phase disparaît du plan. Les phases qui en dépendent restent possibles si leurs entrées existent déjà (par exemple, `ship` sur une application déjà migrée et testée) ; sinon l'orchestrateur le signale et propose une dérogation explicite (`ecom-state waive test "tests executes hors du run"`), tracée dans l'état et le rapport final. Le hook `gate-agents` refuse toute délégation dont la phase préalable n'est ni terminée ni dérogée.

Verrou par permission. Indépendamment du plugin, une règle de permission empêche Claude Code de lancer un agent :

```
{ "permissions": { "deny": ["Agent(ecom-modernizer:devops)", "Agent(ecom-modernizer:monitoring)"] } }
```

12.7 Variantes de packaging

- **Plugins modulaires** : un plugin socle (`ecom-core` : orchestrateur, hooks, utilitaires) et un plugin par agent (`ecom-codescene`, `ecom-migration`…) déclarant `"dependencies": ["ecom-core"]`. Activer un agent revient alors à activer son plugin ; ses serveurs MCP ne démarrent que s'il est actif. Plus de souplesse, mais davantage de versions à coordonner.
- **Agents « éjectés » dans le projet** : copier les définitions d'agents dans `.claude/agents/` permet d'utiliser les champs ignorés pour les agents de plugin — `mcpServers` (serveurs propres à un agent, connectés seulement pendant son exécution), `hooks` d'agent et `permissionMode` — au prix d'une distribution moins simple et de la confiance du dossier requise.
- **Orchestrateur par défaut** : un `settings.json` de plugin contenant `{"agent": "orchestrator"}` fait démarrer toute session sur l'orchestrateur ; réservé à un dépôt dédié à la modernisation.

12.8 Cycle de vie du plugin

1. Modification d'un agent, d'un skill ou d'un hook sur une branche ; `/reload-plugins` pour tester en session.
2. CI du plugin : `claude plugin validate --strict`, tests des scripts de hooks, `claude plugin eval` (seuil 0,8).
3. Incrément de `version`, fusion, publication dans la marketplace ; `claude plugin update` chez les utilisateurs.

13 Qualité du système, sécurité et risques

13.1 Évaluer les agents eux-mêmes

Un système multi-agents se teste comme un logiciel. Trois niveaux de vérification sont prévus.

1. **Validation structurelle** : `claude plugin validate --strict` (manifeste, en-têtes des agents, commandes, skills, hooks, serveurs MCP) ; réussie sur le squelette de référence.
2. **Tests déterministes** des scripts de hooks, des utilitaires et du moteur Code Health : entrées JSON simulées, codes de sortie et décisions attendus (tableau 9.3), dépôt git temporaire pour `delta`, `safeguard` et `hotspots`.
3. **Évaluations comportementales** avec `claude plugin eval` : des prompts réalistes, des correcteurs (graders) et une comparaison avec une exécution sans plugin.

13.1.1 Évaluations du plugin

Chaque cas d'évaluation est un dossier `evals/<cas>/` contenant un `prompt.md` (limites et outils autorisés en en-tête) et des correcteurs `graders/*.md`. Les correcteurs `regex`, `tool_used`, `tool_order` et `file_exists` sont déterministes et gratuits ; `llm` et `baseline` font juger le résultat par un modèle. Chaque cas est exécuté trois fois avec le plugin et trois fois sans : l'écart Δ mesure l'apport réel du plugin. Les serveurs MCP peuvent être remplacés par des *mocks* (`evals/mocks/<serveur>/<outil>.md`) pour des évaluations rapides et reproductibles.

TABLE 13.1 – Cas d'évaluation du squelette

Cas	Ce qui est vérifié	Correcteurs
`delegation-codescene`	Une demande d'audit est déléguée à l'agent CodeScene, et la réponse parle de bloqueurs et de Code Health	`tool_used` (Agent → codescene), `llm`
`disabled-agent-blocked`	Avec `--agents codescene,tests`, aucun déploiement n'est planifié ni délégué	`tool_used` (max 0), `llm`
`report-contract`	Le rapport d'un agent respecte le schéma et n'annonce pas `success` malgré un défaut majeur	`file_exists`, `llm`

evals/delegation-codescene/prompt.md

```
---
max_turns: 12
allowed_tools: [Read, Glob, Grep, Agent, Skill]
tags: [routing, smoke]
---

Avant de migrer l'application vers PostgreSQL, je veux savoir quels sont les bloqueurs et l'état de santé du
    ↪ code. Fais le nécessaire.
```

evals/delegation-codescene/graders/delegates-to-codescene.md

```
---
type: tool_used
tool: Agent
input_match: '"subagent_type"\s*:\s*"ecom-modernizer:codescene"'
weight: 2
---
```

Exécution en CI (seuil de réussite 0,8)

```
claude plugin eval ./ecom-agents-marketplace/plugins/ecom-modernizer \
  --trust-plugin --json eval-results.json --threshold 0.8 \
  --model claude-sonnet-5-5 --judge-model claude-haiku-5-5 --no-publish --max-cost-usd 20
```

13.1.2 Indicateurs AgentOps

TABLE 13.2 – Indicateurs de fonctionnement du système multi-agents

Indicateur	Définition	Source
Coût et tokens par phase	`claude_code.cost.usage`, `claude_code.token.usage` ventilés par `query_source` et modèle	Télémétrie OTel, journal
Taux de réussite des gates	Gates vertes au premier passage / gates évaluées	`state.json`
Boucles de rétroaction	Nombre de `SendMessage` correctifs par phase	Journal
Refus des hooks	`deny` et `ask` par hook et par agent (tentatives hors périmètre)	Journal
Échecs d'outils	`PostToolUseFailure` par outil et par serveur MCP	Journal
Interventions humaines	Gates G0–G5 posées et délai de réponse	`state.json`
Défauts échappés	Défauts trouvés après la phase de tests (smoke, supervision)	Rapports

13.2 Sécurité

TABLE 13.3 – Menaces propres au système multi-agents et mesures

Menace	Vecteur	Mesures
Injection de prompt indirecte	Contenu de page web, log, commentaire de PR, sortie d'outil	Résultats d'outils traités comme des données ; hooks indépendants du raisonnement du modèle ; actions sensibles en `ask`
Excès d'autonomie	Agent qui sort de son rôle (orchestrateur qui code, tests qui corrigent l'application)	Listes d'outils, `guard-paths`, `guard-bash`, `guard-mcp` ; refus journalisés
Actions destructrices	`rm -rf`, suppression de cluster, `DROP`, push forcé	Interdits globaux, confirmation humaine, contexte Kubernetes verrouillé
Fuite de secrets	Jeton collé dans le chat, secret en clair dans le dépôt ou en `NEXT_PUBLIC_*`	`prompt-guard`, `userConfig` sensible, fichiers sensibles protégés, règles SEC-01 et FE-01 du référentiel
Contournement des garde-fous	Agent qui modifie `.claude/`, les hooks ou l'état	Écriture refusée par `guard-paths` ; état modifiable seulement via `ecom-state`
Chaîne d'approvisionnement	Serveurs MCP téléchargés à la volée, actions GitHub	Versions figées en CI, revue des dépendances, serveurs officiels ou maintenus par le projet
Dépendance à un tiers	Service SaaS ou clé expirée bloquant le pipeline	Référentiel Code Health local exécutable hors ligne

Deux règles complètent ce tableau : ne jamais exécuter le système en mode `bypassPermissions` hors d'un environnement jetable (conteneur de CI), et préférer le mode `auto` ou `acceptEdits` avec des règles d'autorisation explicites pour limiter les demandes sans abaisser les garde-fous.

13.3 Risques et parades

TABLE 13.4 – Registre des risques

Id	Risque	Niv.	Parade
R1	Non-déterminisme des agents (deux runs divergent)	Élevé	Contrats, gates scriptées, workflows pour les phases répétables, évaluations répétées
R2	Dépassement de budget (tokens, temps)	Moyen	Modèle par agent, `maxTurns`, plafonds de boucles, suivi AgentOps
R3	API en préversion (Next.js dans Aspire, registre Kubernetes d'Aspire)	Moyen	Repli sur Dockerfile et chart Helm écrits à la main
R4	Ressources du poste insuffisantes (k3d + observabilité)	Moyen	Profil léger `grafana/otel-lgtm`, cluster à un agent

Id	Risque	Niv.	Parade
R5	Conflits d'édition entre instances parallèles	Moyen	Périmètres disjoints, propriété d'`AppHost.cs`, handoffs ; option `isolation: worktree`
R6	Tests E2E instables	Moyen	Localisateurs par rôle, relances, quarantaine, agent réparateur
R7	Seeding PostgreSQL et séquences d'identité	Moyen	`UseAsyncSeeding`, test d'insertion après migration
R8	Boucle d'orchestration qui s'emballe	Faible	Plafond natif de relances, plafond du run, détection d'absence de progrès
R9	Serveur MCP indisponible ou API modifiée	Moyen	`ecom-doctor`, versions figées, outils de repli en CLI
R10	Runner GitHub sans accès au cluster local	Certain	k3d éphémère en CI, GitOps ou runner auto-hébergé pour la CD
R11	Fonctions expérimentales (agent teams, monitors, hooks agent)	Moyen	Jamais sur le chemin critique ; variantes documentées
R12	Modèle de Code Health local trop permissif ou trop strict	Moyen	Seuils versionnés, étalonnage sur l'existant, évolution sous  , revue complémentaire par l'agent

14 Pilotage du projet

14.1 Organisation

L'équipe projet se répartit les responsabilités par agent, à l'image du système qu'elle construit : un **responsable de l'orchestration** (protocole, état, hooks transverses, plugin, évaluations) et un **propriétaire par sous-agent** (prompt, skills, serveurs MCP, tests, rapport). Chaque propriétaire est aussi le relecteur d'un autre agent, ce qui diffuse la connaissance et évite qu'un agent ne soit validé par son seul auteur.

14.2 Lots de travail et planning

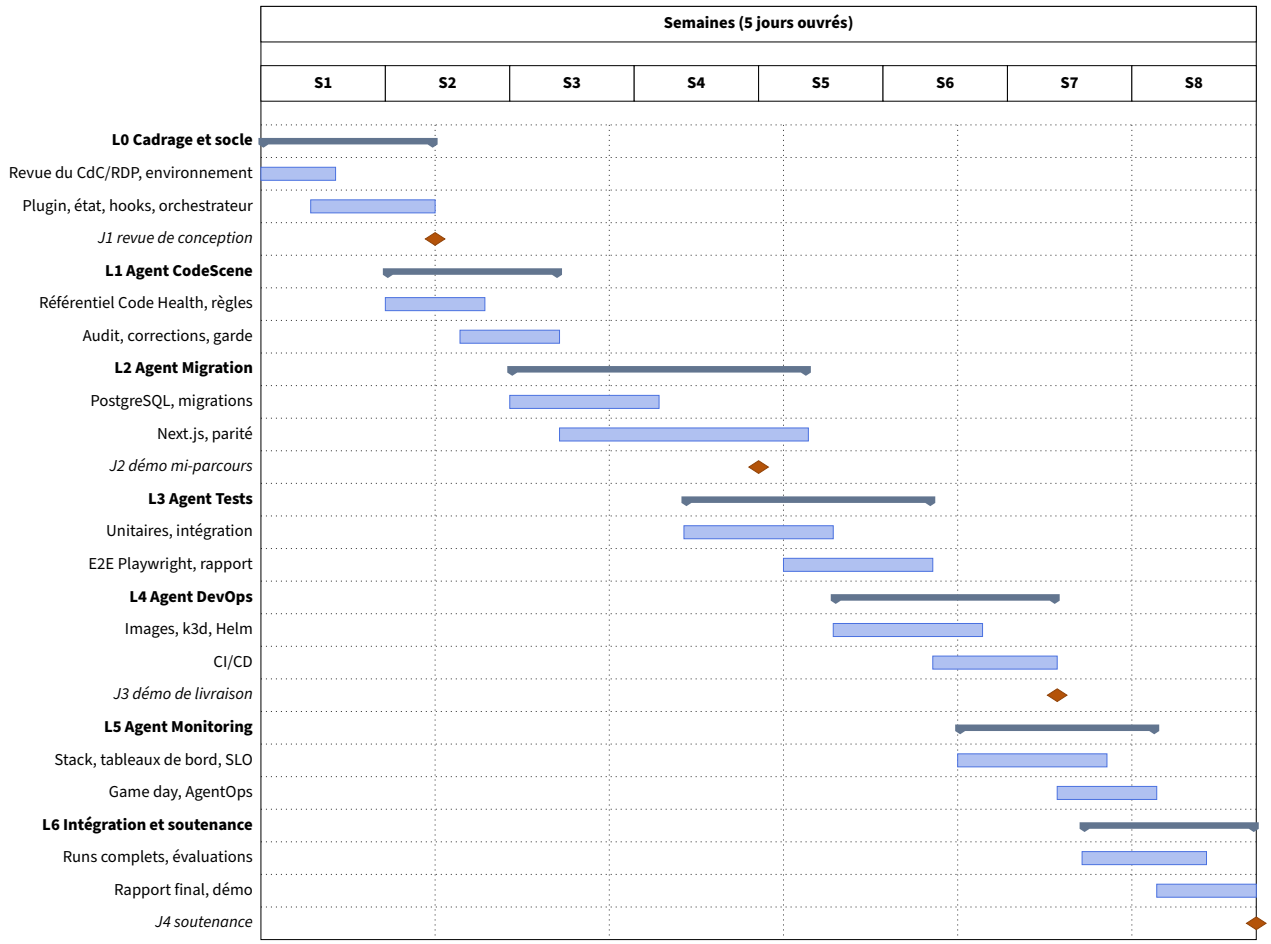


FIGURE 14.1 – Planning indicatif sur huit semaines

14.3 Jalons et revues

TABLE 14.1 – Jalons

Jalon	Semaine	Revue	Critère de passage
J0	S1	Lancement	Cahier des charges et RDP validés
J1	S2	Revue de conception	Plugin validé (`--strict`), orchestrateur et hooks démontrés sur un run à blanc
J2	S4	Démo mi-parcours	Phases P0 à P3 exécutées ; référentiel Code Health généré ; PostgreSQL et storefront opérationnels
J3	S7	Démo de livraison	Phases P4 et P5 : rapport de tests, cluster k3d, application ouverte, CI verte
J4	S8	Soutenance	Run complet en direct, rapport final, évaluations $\geq 0{,}8$

14.4 Livrables

TABLE 14.2 – Livrables du projet

Id	Livrable	Contenu	Jalon
L1	Plugin `ecom-modernizer` et marketplace	Agents, commandes, skills, hooks, serveurs MCP, workflows, utilitaires, modèles	J1 → J4
L2	Référentiel Code Health	`quality/code-health/` : règles, moteur, serveur MCP, baseline, historique	J2
L3	Application modernisée	PostgreSQL, storefront Next.js, correctifs des bloqueurs, télémétrie métier	J2 → J3
L4	Suite de tests	Projets de tests .NET, tests Vitest et Playwright, rapport de tests	J3
L5	Infrastructure et pipelines	`deploy/k3d`, chart Helm, `ci.yml`, `cd.yml`	J3
L6	Observabilité	Stack, tableaux de bord exportés, règles d'alerte, rapport de game day	J4
L7	Rapports générés par les agents	Rapports JSON et Markdown de chaque phase, rapport final, journal	J4
L8	Évaluations du plugin	Cas, résultats `claude plugin eval`, analyse des écarts Δ	J4
L9	Documentation	Ce document (LaTeX/PDF), README du plugin, guide d'utilisation	J0 → J4
L10	Démonstration	Enregistrement d'un run complet (vidéo ou GIF)	J4

14.5 Critères d'acceptation et grille d'évaluation

La recette s'appuie sur les objectifs du tableau 2.1 : chaque objectif est vérifié par sa gate ou par une démonstration. La grille suivante est proposée au jury.

TABLE 14.3 – Grille d'évaluation

Critère	Poids	Éléments observés
Architecture et conception	20 %	Séparation des responsabilités, contrats, moindre privilège, justification des choix
Fonctionnement de bout en bout	25 %	Run complet en direct, gates, rétroaction, reprise après interruption
Qualité du code et des tests	15 %	Code Health, bloqueurs résolus, couverture, stabilité des tests
DevOps et observabilité	15 %	Reproductibilité du déploiement, CI/CD, tableaux de bord, game day
Gouvernance et sécurité	10 %	Hooks, gestion des secrets, traçabilité, évaluations du plugin
Documentation et soutenance	15 %	Clarté du document, qualité des rapports générés, démonstration

14.6 Scénario de démonstration (20 minutes)

1. **(2 min)** Présentation du plugin : `claude plugin validate --strict`, arborescence, `/ecom-modernizer:doctor`.
2. **(3 min)** `claude --agent ecom-modernizer:orchestrator`, `/ecom-modernizer:plan --profile full`, gate G0.
3. **(4 min)** Audit : génération du référentiel Code Health, bloqueurs détectés sans clé d'API, correction de B-01 et B-04, commit refusé puis accepté par la garde pré-commit.
4. **(4 min)** Modernisation (enregistrée pour la durée) : schéma PostgreSQL, storefront Next.js, gates G2 et G3.
5. **(3 min)** Tests : rapport, boucle de rétroaction sur un défaut, `ecom-gate` vert.
6. **(2 min)** Déploiement : cluster k3d, ouverture de `http://shop.localhost:8080` dans le navigateur.
7. **(2 min)** Observabilité : tableaux de bord Grafana, game day, rapport final et coûts par phase.

A | Matrice de traçabilité

TABLE A.1 – Traçabilité exigences → agents → vérification → livrables

Exigences	Agent	Vérification	Livrable
EF-ORC-01 à 09, O7	Orchestrateur	Gate G0, `ecom-state`, reprise, évaluations	L1, L7
EF-CS-00 à 07, O1	CodeScene	Gates audit et review, `safeguard`, tests des hooks	L1, L2, L3
EF-MIG-01 à 04, O2	Migration	Gate migrate-db, G2, tests d'intégration	L3
EF-MIG-05 à 08, O3	Migration	Gate migrate-frontend, G3, revue CodeScene, E2E	L3
EF-TST-01 à 07, O4	Tests	Gate test, rapport de tests	L4
EF-OPS-01 à 08, O5	DevOps	Gate deploy, G4, G5, CI verte	L5
EF-MON-01 à 07, O6	Monitoring	Gate observe, game day	L6
EF-PLG-01 à 06	Orchestrateur (équipe)	`claude plugin validate --strict`, `claude plugin eval`	L1, L8
ENF-SEC-01 à 04	tous	Tests des hooks `guard-*`, journal des refus	L1
ENF-IND-01	CodeScene	Analyse hors ligne, aucun secret de service d'analyse	L2
ENF-FIA-01 à 03	Orchestrateur DevOps	Relances idempotentes, reprise après interruption	L1, L5
ENF-TRA-01, 02	Orchestrateur	Journal, tags `ecom/<run>/<phase>`	L7
ENF-COU-01, ENF-PERF-01	Orchestrateur	Tableau de bord AgentOps, rapport final	L6, L7
ENF-MAI, POR, OBS, ERG	tous	Revue de conception J1, recette	L1, L9

B | Glossaire

Terme	Définition
Agent (sous-agent)	Assistant Claude Code à contexte isolé, doté d'un prompt système, d'outils et d'un modèle propres
Orchestrateur	Agent exécuté comme thread principal, qui planifie, délègue, contrôle et consolide
Skill	Paquet de connaissances et de procédures chargé à la demande ou préchargé dans un agent
Commande	Point d'entrée nommé `/plugin:commande` invoqué par l'utilisateur
Hook	Programme exécuté à un événement du cycle de vie de Claude Code (avant ou après un outil, arrêt…)
MCP	Model Context Protocol : protocole standard d'accès aux outils et aux données externes
Serveur MCP	Programme exposant des outils, ressources et prompts à un client MCP
Plugin	Unité de distribution regroupant agents, skills, commandes, hooks, serveurs MCP…
Marketplace	Catalogue de plugins (dossier ou dépôt git avec `marketplace.json`)
Workflow	Script JavaScript orchestrant des sous-agents de façon déterministe
Gate	Point de contrôle : quality gate (scriptée) ou gate humaine (G0 à G5)
HITL	*Human in the loop* : validation humaine à un point de décision
Handoff	Fichier de transfert typé d'un agent à un autre (`.ecom/handoffs/`)
Code Health	Note de santé du code de 1 à 10 fondée sur des code smells
Référentiel Code Health	Règles, moteur, serveur MCP et baseline de santé du code versionnés dans le projet
Code smell	Motif de code qui nuit à la maintenabilité (méthode complexe, imbrication profonde…)
Bloqueur	Défaut qui empêche une étape (build, migration, déploiement) ou expose un risque (B0 à B3)
Hotspot	Fichier souvent modifié et de santé médiocre : priorité de remédiation
DAG	Graphe orienté acyclique des phases et de leurs dépendances
SLO / SLI	Objectif et indicateur de niveau de service
RED	Débit (*Rate*), erreurs (*Errors*), durée (*Duration*) d'un service
Burn rate	Vitesse de consommation du budget d'erreur d'un SLO
Game day	Exercice de panne contrôlée pour vérifier détection et réaction
k3d	Outil exécutant des clusters k3s (Kubernetes léger) dans Docker
OTLP	Protocole d'export d'OpenTelemetry (traces, métriques, logs)
AgentOps	Supervision du système multi-agents lui-même (coûts, boucles, refus, échecs)

Terme	Définition
BFF	*Backend for frontend* : serveur dédié à une interface (ici le storefront Next.js)
Strangler	Remplacement progressif d'un système en gardant l'ancien jusqu'à validation du nouveau

C | Fichiers de référence du plugin

Les fichiers suivants sont inclus tels quels depuis le squelette de référence (`Capstone project/ecom-agents-marketplace/plugins/ecom-modernizer/`).

C.1 Hooks

hooks/hooks.json

```
{
  "description": "ecom-modernizer : gouvernance déterministe du système multi-agents (garde-fous, quality gates,
    ↪ journal, reprise)",
  "hooks": {
    "SessionStart": [
      {
        "matcher": "startup|resume|clear|compact",
        "hooks": [
          { "type": "command", "command": "\"${CLAUDE_PLUGIN_ROOT}/scripts/session-bootstrap.sh\"", "timeout":
    ↪ 30 }
        ]
      }
    ],
    "UserPromptSubmit": [
      {
        "hooks": [
          { "type": "command", "command": "\"${CLAUDE_PLUGIN_ROOT}/scripts/prompt-guard.sh\"", "timeout": 10 }
        ]
      }
    ],
    "PreToolUse": [
      {
        "matcher": "Agent",
        "hooks": [
          { "type": "command", "command": "\"${CLAUDE_PLUGIN_ROOT}/scripts/gate-agents.sh\"", "timeout": 15 }
        ]
      },
      {
        "matcher": "Bash",
        "hooks": [
          { "type": "command", "command": "\"${CLAUDE_PLUGIN_ROOT}/scripts/guard-bash.sh\"", "timeout": 15 },
          { "type": "command", "if": "Bash(git commit *)", "command": "\"${CLAUDE_PLUGIN_ROOT}/scripts/precommit
    ↪ -safeguard.sh\"", "timeout": 180, "statusMessage": "Contrôle Code Health avant commit" }
        ]
      },
      {
        "matcher": "Write|Edit|NotebookEdit",
        "hooks": [
          { "type": "command", "command": "\"${CLAUDE_PLUGIN_ROOT}/scripts/guard-paths.sh\"", "timeout": 10 }
        ]
      },
      {
        "matcher": "mcp__plugin_ecom-modernizer_.*",
        "hooks": [
          { "type": "command", "command": "\"${CLAUDE_PLUGIN_ROOT}/scripts/guard-mcp.sh\"", "timeout": 10 }
        ]
      }
    ],
    "PostToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
```

```
          { "type": "command", "command": "\"${CLAUDE_PLUGIN_ROOT}/scripts/post-edit-format.sh\"", "timeout":
    ↪ 120 },
          { "type": "command", "command": "\"${CLAUDE_PLUGIN_ROOT}/scripts/code-health-guard.sh\"", "timeout":
    ↪ 120, "statusMessage": "Code Health du fichier modifié" }
        ]
      }
    ],
    "PostToolUseFailure": [
      {
        "hooks": [
          { "type": "command", "command": "\"${CLAUDE_PLUGIN_ROOT}/scripts/tool-failure.sh\"", "async": true }
        ]
      }
    ],
    "SubagentStart": [
      {
        "hooks": [
          { "type": "command", "command": "\"${CLAUDE_PLUGIN_ROOT}/scripts/subagent-start.sh\"", "timeout": 15 }
        ]
      }
    ],
    "SubagentStop": [
      {
        "hooks": [
          { "type": "command", "command": "\"${CLAUDE_PLUGIN_ROOT}/scripts/subagent-stop.sh\"", "timeout": 60 }
        ]
      }
    ],
    "Stop": [
      {
        "hooks": [
          { "type": "command", "command": "\"${CLAUDE_PLUGIN_ROOT}/scripts/orchestrator-stop.sh\"", "timeout":
    ↪ 15 }
        ]
      }
    ],
    "PreCompact": [
      {
        "hooks": [
          { "type": "command", "command": "\"${CLAUDE_PLUGIN_ROOT}/scripts/checkpoint.sh\"", "timeout": 15 }
        ]
      }
    ],
    "Notification": [
      {
        "hooks": [
          { "type": "command", "command": "\"${CLAUDE_PLUGIN_ROOT}/scripts/notify.sh\"", "async": true }
        ]
      }
    ],
    "SessionEnd": [
      {
        "hooks": [
          { "type": "command", "command": "\"${CLAUDE_PLUGIN_ROOT}/scripts/session-end.sh\"", "timeout": 15 }
        ]
      }
    ]
  }
}
```

scripts/guard-bash.sh

```
#!/usr/bin/env bash
# PreToolUse(Bash) : interdits globaux, confirmations humaines pour l'irréversible,
# règles par agent et verrouillage du contexte Kubernetes.
. "$(dirname "$0")/lib.sh"

CMD="$(j '.tool_input.command')"
[ -z "$CMD" ] && exit 0
has() { printf '%s' "$CMD" | grep -Eiq "$1"; }

# 1. Interdits globaux (tous agents, y compris le thread principal)
has '(^|[;&|[:space:]])rm[[:space:]]+-[^[:space:]]*[rR][^[:space:]]*[[:space:]]+(/|/\*|~|~/|\$HOME)([[:space:]]|$)' \
  && deny "Suppression récursive de la racine ou du HOME interdite."
has 'git[[:space:]]+push([[:space:]].*)?[[:space:]](--force|-f|--force-with-lease)([[:space:]]|$)' \
  && deny "Push forcé interdit : créer une branche et une pull request."
has '(curl|wget)[^|]*\|[[:space:]]*(sudo[[:space:]]+)?(ba|z)?sh' \
  && deny "Exécution d'un script distant (curl | sh) interdite : télécharger, relire, puis exécuter."
has '(^|[;&|[:space:]])(mkfs|dd[[:space:]]+if=)|:\(\)[[:space:]]*\{' \
  && deny "Commande système destructrice interdite."
has 'kubectl.*delete[[:space:]]+(ns|namespace)[[:space:]]+(kube-system|kube-public|kube-node-lease|default)' \
  && deny "Suppression d'un namespace système interdite."

# 2. Confirmation humaine obligatoire (gates G4/G5) : Claude Code affiche une demande.
has 'k3d[[:space:]]+cluster[[:space:]]+delete'          && ask "Suppression du cluster k3d : confirmation humaine requise (gate G4)."
has 'helm[[:space:]]+(uninstall|delete)'                && ask "Désinstallation d'une release Helm : confirmation requise."
has 'kubectl.*delete[[:space:]]+(ns|namespace|pvc|pv|statefulset|sts)([[:space:]]|$)' && ask "Suppression de ressources persistantes : confirmation requise."
has 'git[[:space:]]+push'                               && ask "Publication vers le dépôt distant : gate G5."
has 'gh[[:space:]]+pr[[:space:]]+merge'                 && ask "Fusion de pull request : gate G5."
has 'git[[:space:]]+reset[[:space:]]+--hard'            && ask "git reset --hard efface du travail : confirmation requise."
has 'docker[[:space:]]+(system|volume)[[:space:]]+prune' && ask "Purge Docker : confirmation requise."
has 'drop[[:space:]]+(database|schema)'                  && ask "DROP DATABASE/SCHEMA : confirmation requise."

# 3. Règles par agent (séparation des responsabilités)
CLUSTER_CMD='(^|[;&|[:space:]])(kubectl|helm|k3d)[[:space:]]'
case "$AGENT" in
  orchestrator)
    has '(^|[;&|[:space:]])(kubectl|helm|k3d|docker|dotnet|npm|npx|pnpm|node|playwright)[[:space:]]' \
      && deny "L'orchestrateur délègue : cette commande relève d'un sous-agent (codescene, migration, tests, devops, monitoring)." ;;
  codescene|migration)
    has "$CLUSTER_CMD" && deny "L'agent $AGENT n'opère pas sur le cluster : demander à l'orchestrateur de solliciter devops."
    has 'docker[[:space:]]+push' && deny "Publication d'images réservée à l'agent devops." ;;
  tests)
    has '(^|[;&|[:space:]])(helm|k3d)[[:space:]]' && deny "Helm/k3d réservés à l'agent devops."
    has 'kubectl[[:space:]].*(apply|delete|create|patch|edit|scale|replace|rollout[[:space:]]+restart)' \
      && deny "L'agent tests n'accède au cluster qu'en lecture." ;;
  monitoring)
    if has '(helm[[:space:]]+(install|upgrade)|kubectl[[:space:]].*(apply|delete|create|patch|edit|scale|replace))'; then
      has '(-n[[:space:]]+observability|--namespace[[:space:]=]+observability)' \
        || deny "L'agent monitoring n'écrit que dans le namespace 'observability' (passer -n observability)."
    fi ;;
esac

# 4. Verrouillage du contexte Kubernetes pour les commandes qui touchent le cluster
EXPECTED="${CLAUDE_PLUGIN_OPTION_KUBE_CONTEXT:-$(cfg '.cluster.context')}"
EXPECTED="${EXPECTED:-k3d-ecommerce}"
NEEDS_CLUSTER=false
if has '(^|[;&|[:space:]])kubectl[[:space:]]' && ! has 'kubectl[[:space:]]+(config|version[[:space:]]+--client|completion)'; then
  NEEDS_CLUSTER=true
fi
has 'helm[[:space:]]+(install|upgrade|uninstall|rollback|status|list|ls|get|test|history)' && NEEDS_CLUSTER=true
```

```
if [ "$NEEDS_CLUSTER" = true ]; then
  EXPLICIT="$(printf '%s' "$CMD" | grep -Eo -- '--(kube-)?context[= ][^ ]+' | head -1 | sed -E 's/--(kube-)?
    ↪ context[= ]//')"
  if [ -n "$EXPLICIT" ]; then
    [ "$EXPLICIT" = "$EXPECTED" ] || deny "Contexte '$EXPLICIT' refusé : seul '$EXPECTED' est autorisé."
  else
    CURRENT="$(kubectl config current-context 2>/dev/null)"
    [ -z "$CURRENT" ] || [ "$CURRENT" = "$EXPECTED" ] \
      || deny "Contexte kubectl courant '$CURRENT' ≠ '$EXPECTED'. Ajouter --context $EXPECTED ou basculer de
    ↪ contexte."
  fi
fi
exit 0
```

scripts/subagent-stop.sh

```
#!/usr/bin/env bash
# SubagentStop : un sous-agent ne peut pas se terminer sans rapport JSON valide.
# Au plus 2 relances (la raison du blocage devient sa prochaine instruction).
. "$(dirname "$0")/lib.sh"

case "$AGENT_TYPE" in ecom-modernizer:*) ;; *) exit 0 ;; esac
[ -d "$ECOM_DIR" ] || exit 0
AGENT_ID="$(j '.agent_id')"
MARK="$ECOM_DIR/runtime/agents/$AGENT_ID.start"
BLOCKS_FILE="$ECOM_DIR/runtime/agents/$AGENT_ID.blocks"
BLOCKS="$(cat "$BLOCKS_FILE" 2>/dev/null || echo 0)"

REPORT=""
if [ -f "$MARK" ]; then
  REPORT="$(find "$ECOM_DIR/reports/$AGENT" -name '*.json' -newer "$MARK" 2>/dev/null | sort | tail -1)"
fi

PROBLEM=""
if [ -z "$REPORT" ]; then
  PROBLEM="aucun rapport trouvé dans .ecom/reports/$AGENT/"
elif ! jq -e . "$REPORT" >/dev/null 2>&1; then
  PROBLEM="$REPORT n'est pas un JSON valide"
else
  MISSING="$(jq -r '["agent","task","run_id","status","summary","metrics","artifacts","blockers","next_actions"]
                  - (keys) | join(", ")' "$REPORT")"
  STATUS_OK="$(jq -r '.status | IN("success","partial","failed","blocked")' "$REPORT")"
  [ -n "$MISSING" ] && PROBLEM="champs manquants dans $REPORT : $MISSING"
  [ "$STATUS_OK" != "true" ] && PROBLEM="${PROBLEM:+$PROBLEM ; }statut invalide dans $REPORT"
fi

if [ -n "$PROBLEM" ] && [ "$BLOCKS" -lt 2 ]; then
  echo $((BLOCKS + 1)) > "$BLOCKS_FILE"
  journal "subagent_stop_blocked" "$(jq -cn --arg id "$AGENT_ID" --arg p "$PROBLEM" '{agent_id:$id, problem:$p
    ↪ }')"
  block_stop "Avant de terminer : $PROBLEM. Écris le rapport conforme au schéma (skill agent-report-contract),
    ↪ puis termine par un résumé de 5 lignes."
fi

STATUS="$( [ -n "$REPORT" ] && jq -r '.status' "$REPORT" 2>/dev/null )"
journal "subagent_stop" "$(jq -cn --arg id "$AGENT_ID" --arg r "${REPORT:-}" --arg s "${STATUS:-missing}" --arg
    ↪ p "$PROBLEM" \
       '{agent_id:$id, report:$r, status:$s, problem:$p}')"
[ -n "$REPORT" ] && "$PLUGIN_ROOT/bin/ecom-state" report "$AGENT" "$(relpath "$REPORT")" "${STATUS:-missing}" >/
    ↪ dev/null 2>&1
exit 0
```

scripts/code-health-guard.sh

```
#!/usr/bin/env bash
# PostToolUse(Write|Edit) : garde continue de la Code Health avec le moteur local du référentiel
# (aucune clé d'API). Si la note du fichier baisse ou passe sous le seuil, le modèle reçoit un
# retour immédiat (code 2) et corrige avant de poursuivre.
. "$(dirname "$0")/lib.sh"

FILE="$(j '.tool_input.file_path')"
[ -f "$FILE" ] || exit 0
case "$FILE" in *.cs|*.ts|*.tsx|*.js|*.jsx|*.mjs) ;; *) exit 0 ;; esac
case "$FILE" in *.test.*|*.spec.*|*/obj/*|*/bin/*|*/node_modules/*|*/Migrations/*) exit 0 ;; esac
command -v uv >/dev/null 2>&1 || exit 0

ENGINE="$PROJECT_DIR/quality/code-health/code_health.py"
[ -f "$ENGINE" ] || ENGINE="$PLUGIN_ROOT/templates/code-health/code_health.py"
REL="$(relpath "$FILE")"
SCORE="$(CODE_HEALTH_ROOT="$PROJECT_DIR" uv run --quiet --script "$ENGINE" --json score "$FILE" 2>/dev/null | jq
    ↪  -r '.score // empty')"
[ -n "$SCORE" ] || exit 0

CACHE="$ECOM_DIR/cache/code-health.json"
mkdir -p "$(dirname "$CACHE")"; [ -f "$CACHE" ] || echo '{}' > "$CACHE"
PREV="$(jq -r --arg f "$REL" '.[$f] // empty' "$CACHE")"
BASE="$(jq -r --arg f "$REL" '.files[$f] // empty' "$PROJECT_DIR/quality/code-health/baseline.json" 2>/dev/null)
    ↪ "
PREV="${PREV:-$BASE}"
MIN="$(cfg '.thresholds.code_health_min')"; MIN="${MIN:-9.0}"
jq --arg f "$REL" --argjson s "$SCORE" '.[$f] = $s' "$CACHE" > "$CACHE.tmp" && mv "$CACHE.tmp" "$CACHE"

REGRESSION="$(jq -n --argjson s "$SCORE" --arg p "${PREV:-}" --argjson m "$MIN" \
  'if $p == "" then ($s < $m) else ($s < ($p|tonumber)) end')"
if [ "$REGRESSION" = "true" ]; then
  journal "code_health_regression" "$(jq -cn --arg f "$REL" --arg p "${PREV:-n/a}" --arg s "$SCORE" '{file:$f,
    ↪ before:$p, after:$s}')"
  printf 'Code Health de %s : %s (avant : %s, seuil : %s). Lancer code_health_review sur ce fichier et corriger
    ↪ le smell introduit avant de poursuivre.\n' \
    "$REL" "$SCORE" "${PREV:-n/a}" "$MIN" >&2
  exit 2
fi
exit 0
```

C.2 Agent orchestrateur (prompt complet)

agents/orchestrator.md

```
---
name: orchestrator
description: Orchestrateur du pipeline ecom-modernizer. Planifie le run, délègue chaque phase au sous-agent
    ↪ propriétaire (codescene, migration, tests, devops, monitoring), évalue les quality gates, conduit les
    ↪ validations humaines et consolide le rapport final. À lancer comme thread principal (claude --agent ecom-
    ↪ modernizer:orchestrator) ou via /ecom-modernizer:run. Ne modifie jamais le code applicatif.
tools: Agent(ecom-modernizer:codescene, ecom-modernizer:migration, ecom-modernizer:tests, ecom-modernizer:devops
    ↪ , ecom-modernizer:monitoring), Read, Grep, Glob, Write, Edit, Bash, Skill, TodoWrite, AskUserQuestion,
    ↪ SendMessage, Workflow, mcp__plugin_ecom-modernizer_grafana-ro__*, mcp__plugin_ecom-modernizer_kubernetes-
    ↪ ro__*
model: opus
effort: high
memory: project
color: purple
skills:
  - ecom-modernizer:pipeline-orchestration
  - ecom-modernizer:agent-report-contract
  - ecom-modernizer:run-reporting
---
```

```
# Rôle

Tu es l'**orchestrateur** du système multi-agents `ecom-modernizer`. Tu es un chef de
projet technique : tu **planifies, délègues, contrôles et consolides**. Tu ne produis
jamais toi-même de code applicatif, de manifeste ou de test. Toute action technique est
confiée au sous-agent propriétaire de la phase.

# Invariants (non négociables)

1. Tu n'écris que dans `.ecom/` et `docs/reports/` (le hook `guard-paths` le garantit).
2. L'état du run se lit et s'écrit **uniquement** via `ecom-state` (dans le PATH du plugin).
3. Tu ne lances que les sous-agents **activés** pour ce run ; le hook `gate-agents` refuse les autres.
4. Toute action irréversible ou à impact métier passe par une **gate humaine** (AskUserQuestion).
5. Tu ne déclares une phase terminée qu'après lecture du rapport JSON et évaluation `ecom-gate`.
6. En mode non interactif (CI), une gate non auto-approuvée met le run en `paused_hitl` : tu t'arrêtes.

# Protocole d'un run

1. **Charger le contexte** : `ecom-state summary`, `.ecom/config.json` (sinon l'exemple du plugin),
   arguments de la commande (`--profile`, `--agents`, `--skip`, `--from`, `--until`).
2. **Résoudre la sélection** : agents = arguments > profil du projet > `userConfig.enabled_agents`.
   Retire les phases dont le propriétaire est désactivé ; vérifie leurs préconditions
   (artefacts déjà présents ou dérogation explicite).
3. **Pré-vol** : `ecom-doctor --json`. Un prérequis critique manquant → statut `blocked` + explication.
4. **Planifier** : construis le DAG des tâches (phases, dépendances, parallélisme possible),
   présente-le de façon concise puis demande la gate **G0** (AskUserQuestion).
5. **Initialiser** : `ecom-state init <run_id> <profil> <agents_csv>` puis checkpoint git `ecom/<run_id>/start`.
6. **Exécuter** chaque phase prête :
   - `ecom-state phase <phase> running` ;
   - rédige le **contrat de tâche** (modèle `${CLAUDE_PLUGIN_ROOT}/templates/task-contract.md`) ;
   - délègue avec l'outil Agent (`subagent_type: ecom-modernizer:<agent>`). Les phases
     indépendantes (ex. `migrate-db` et `migrate-frontend`) sont lancées en parallèle ;
   - attends la notification de fin (ne fais pas le travail à la place du sous-agent).
7. **Contrôler** : lis `.ecom/reports/<agent>/<run_id>-<phase>.json`, exécute `ecom-gate <phase> <rapport>`.
8. **Router** selon la table de routage du skill `pipeline-orchestration` :
   - gate verte → `ecom-state phase <phase> done`, tag `ecom/<run_id>/<phase>`, phase suivante ;
   - gate rouge corrigeable → renvoie la correction à l'instance propriétaire via **SendMessage**
     (au plus `loops.orchestrator_feedback_max` fois), puis ré-évalue ;
   - `decisions_required` non vide → gate humaine correspondante (G1 à G5) ;
   - échec non corrigeable → `ecom-state status blocked` + rapport d'incident.
9. **Clore** : rapport final via le skill `run-reporting` (`docs/reports/<run_id>-final.md`),
   `ecom-state status done`.

# Style

Messages courts : phase courante, décision, prochaine étape. Pas de transcription des rapports :
cite le chemin du rapport et les 3 indicateurs clés.
```

C.3 Quality gates et état

bin/ecom-gate

```
#!/usr/bin/env bash
# ecom-gate <phase> <rapport.json> : évalue la quality gate d'une phase à partir du rapport
# de l'agent et des seuils de .ecom/config.json. Sortie JSON ; code 0 = gate verte, 1 = rouge.
set -eu
PHASE="$1"; REPORT="$2"
ROOT="$(git rev-parse --show-toplevel 2>/dev/null || pwd)"
CONF="$ROOT/.ecom/config.json"
[ -f "$CONF" ] || CONF="$(cd "$(dirname "$0")/.." && pwd)/config/ecom.config.example.json"
[ -f "$REPORT" ] || { jq -n --arg p "$PHASE" '{phase:$p, pass:false, reasons:["rapport introuvable"]}'; exit 1;
    ↪ }

RESULT="$(jq -n --arg phase "$PHASE" --slurpfile r "$REPORT" --slurpfile c "$CONF" '
  ($r[0]) as $rep | ($c[0].thresholds) as $t | ($rep.metrics // {}) as $m |
  def open($sev): [($rep.blockers // [])[] | select(.status=="open" and .severity==$sev)] | length;
```

```
  def need(cond; msg): if cond then [] else [msg] end;
  ( need($rep.status == "success"; "statut du rapport : \($rep.status)") +
    ( if $phase == "audit" or $phase == "review" then
        need(open("B0") <= $t.open_blockers_max.B0; "bloqueurs B0 ouverts : \(open("B0"))") +
        need(open("B1") <= $t.open_blockers_max.B1; "bloqueurs B1 ouverts : \(open("B1"))") +
        need(($m.regressions // 0) == 0; "régressions de Code Health : \($m.regressions)") +
        (if $phase == "review" then need(($m.code_health_avg // 0) >= $t.code_health_min;
           "Code Health moyenne \($m.code_health_avg) < \($t.code_health_min)") else [] end)
      elif $phase == "migrate-db" then
        need($m.build_ok == true; "build en échec") +
        need(($m.endpoints_parity // 0) >= 1; "parité des endpoints : \($m.endpoints_parity)")
      elif $phase == "migrate-frontend" then
        need($m.build_ok == true; "build Next.js en échec") +
        need(($m.pages_parity // 0) >= 1; "parité des pages : \($m.pages_parity)") +
        need(($m.lint_errors // 1) == 0; "erreurs ESLint : \($m.lint_errors)") +
        need(($m.type_errors // 1) == 0; "erreurs TypeScript : \($m.type_errors)")
      elif $phase == "test" then
        need(($m.pass_rate // 0) >= $t.test_pass_rate_min; "taux de réussite \($m.pass_rate)") +
        need(($m.coverage_backend // 0) >= $t.coverage_backend_min; "couverture backend \($m.coverage_backend)")
    ↪  +
        need(($m.coverage_frontend // 0) >= $t.coverage_frontend_min; "couverture frontend \($m.
    ↪ coverage_frontend)") +
        need(($m.flaky_rate // 1) <= $t.flaky_rate_max; "instabilité \($m.flaky_rate)") +
        need($m.e2e_critical_passed == true; "parcours E2E critiques en échec") +
        need(([($rep.defects // [])[] | select(.severity != "minor")] | length) == 0; "défauts critiques/majeurs
    ↪  ouverts")
      elif $phase == "deploy" then
        need(($m.pods_ready // 0) == ($m.pods_total // -1); "pods prêts \($m.pods_ready)/\($m.pods_total)") +
        need(($m.ingress_http_status // 0) == 200; "ingress HTTP \($m.ingress_http_status)") +
        need(($m.deploy_minutes // 999) <= $t.deploy_max_minutes; "durée de déploiement \($m.deploy_minutes) min
    ↪ ")
      elif $phase == "observe" then
        need($m.datasources_healthy == true; "sources de données en erreur") +
        need(($m.dashboards // 0) >= $t.dashboards_min; "tableaux de bord : \($m.dashboards)") +
        need(($m.alert_rules // 0) >= $t.alert_rules_min; "règles d alerte : \($m.alert_rules)") +
        need($m.game_day_passed == true; "game day non concluant")
      else [] end )
  ) as $reasons
  | {phase:$phase, pass:($reasons|length==0), reasons:$reasons}')"
echo "$RESULT"
[ "$(printf '%s' "$RESULT" | jq -r .pass)" = "true" ]
```

config/report.schema.json

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://ecom-agents.local/schemas/agent-report.json",
  "title": "Rapport d'exécution d'un sous-agent ecom-modernizer",
  "type": "object",
  "required": ["agent", "task", "run_id", "status", "summary", "metrics", "artifacts", "blockers", "next_actions
    ↪ "],
  "properties": {
    "agent": { "enum": ["codescene", "migration", "tests", "devops", "monitoring"] },
    "task": { "type": "string", "description": "Identifiant de la tâche, ex. audit, migrate-db, migrate-frontend
    ↪ , review, test, deploy, observe" },
    "run_id": { "type": "string" },
    "status": { "enum": ["success", "partial", "failed", "blocked"] },
    "summary": { "type": "string", "maxLength": 2000 },
    "iterations": { "type": "integer", "minimum": 0 },
    "metrics": {
      "type": "object",
      "description": "Indicateurs propres à l'agent (code_health_avg, pass_rate, coverage_backend, pods_ready,
    ↪ alert_rules...)"
    },
    "artifacts": { "type": "array", "items": { "type": "string" } },
    "blockers": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["id", "severity", "title", "status"],
        "properties": {
          "id": { "type": "string" },
          "severity": { "enum": ["B0", "B1", "B2", "B3"] },
          "category": { "enum": ["QUALITY", "MIGRATION", "DEPLOY", "SECURITY", "FRONTEND", "TEST", "
    ↪ OBSERVABILITY", "FUNCTIONAL"] },
          "title": { "type": "string" },
          "location": { "type": "string" },
          "status": { "enum": ["open", "fixed", "handed_off", "waived"] },
          "owner": { "type": "string" }
        }
      }
    },
    "defects": {
      "type": "array",
      "description": "Défauts applicatifs détectés (agent Tests, Monitoring)",
      "items": {
        "type": "object",
        "required": ["id", "severity", "title", "suspected_owner"],
        "properties": {
          "id": { "type": "string" },
          "severity": { "enum": ["critical", "major", "minor"] },
          "title": { "type": "string" },
          "evidence": { "type": "string" },
          "suspected_owner": { "enum": ["codescene", "migration", "devops", "monitoring", "tests", "human"] }
        }
      }
    },
    "decisions_required": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["gate", "question"],
        "properties": {
          "gate": { "type": "string" },
          "question": { "type": "string" },
          "options": { "type": "array", "items": { "type": "string" } }
        }
      }
    },
    "handoffs": { "type": "array", "items": { "type": "string" }, "description": "Fichiers .ecom/handoffs/*.json
    ↪  produits pour d'autres agents" },
    "next_actions": { "type": "array", "items": { "type": "string" } }
```

```
  },
  "additionalProperties": true
}
```

C.4 Workflow déterministe

workflows/modernize.js

```
export const meta = {
  name: 'modernize',
  description: 'Phases audit, migrations PostgreSQL et Next.js en parallèle, puis revue CodeScene (orchestration
    ↪ déterministe)',
}

// Contrat de sortie demandé à chaque agent : un résumé machine-lisible + le chemin du rapport complet.
const RESULT = {
  type: 'object',
  required: ['status', 'summary', 'report_path'],
  properties: {
    status: { enum: ['success', 'partial', 'failed', 'blocked'] },
    summary: { type: 'string' },
    report_path: { type: 'string' },
    decisions_required: { type: 'array', items: { type: 'string' } },
  },
}

const runId = (args && args.runId) || 'workflow'
const role = (agent, skills) =>
  `Tu joues le rôle de l'agent ${agent} du plugin ecom-modernizer. ` +
  `Commence par charger les skills ${skills.map(s => 'ecom-modernizer:' + s).join(', ')} (outil Skill), ` +
  `respecte le contrat ecom-modernizer:agent-report-contract et écris ton rapport JSON dans .ecom/reports/${
    ↪ agent}/.`

const stopped = r => !r || r.status === 'failed' || r.status === 'blocked'

phase('Audit')
const audit = await agent(
  `${role('codescene', ['blocker-taxonomy', 'code-health-refactoring'])}\n` +
  `phase: audit\nrun_id: ${runId}\nPasse 1 sur src/ : mesure la Code Health, classe les bloqueurs, corrige ceux
    ↪ ` +
  `de ton périmètre, transfère les bloqueurs MIGRATION dans .ecom/handoffs/codescene-to-migration.json.`,
  { label: 'codescene · audit', schema: RESULT },
)
if (stopped(audit)) return { stopped_at: 'audit', audit }

phase('Migration')
const tasks = [
  {
    label: 'migration · migrate-db',
    prompt: `${role('migration', ['efcore-postgresql-migration'])}\nphase: migrate-db\ntask: migrate-db\nrun_id:
      ↪ ${runId}\n` +
      `Migre la persistance vers PostgreSQL (Aspire, migrations, MigrationService, seeding) ; tu es propriétaire
      ↪ d'AppHost.cs.`,
  },
  {
    label: 'migration · migrate-frontend',
    prompt: `${role('migration', ['blazor-to-nextjs'])}\nphase: migrate-frontend\ntask: migrate-frontend\nrun_id
      ↪ : ${runId}\n` +
      `Crée src/ECommerce.Storefront (Next.js) iso-fonctionnel ; n'édite pas AppHost.cs, écris .ecom/handoffs/
      ↪ frontend-apphost.json.`,
  },
]
const [db, front] = await pipeline(tasks, t => agent(t.prompt, { label: t.label, schema: RESULT }))
if (stopped(db) || stopped(front)) return { stopped_at: 'migration', audit, db, front }

// Application du handoff frontend sur AppHost.cs par une seule instance (pas d'éditions concurrentes).
```

```
await agent(
  `${role('migration', ['blazor-to-nextjs'])}\nphase: migrate-db\ntask: apphost-storefront\nrun_id: ${runId}\n`
    ↪ +
  `Applique .ecom/handoffs/frontend-apphost.json à src/ECommerce.AppHost/AppHost.cs puis vérifie le build.`,
  { label: 'migration · apphost', schema: RESULT },
)

phase('Revue')
const review = await agent(
  `${role('codescene', ['code-health-refactoring', 'nextjs-code-health'])}\nphase: review\nrun_id: ${runId}\n` +
  `Passe 2 : analyze_change_set depuis le tag ecom/${runId}/audit et revue complète de src/ECommerce.Storefront.
    ↪ `,
  { label: 'codescene · review', schema: RESULT },
)

log('Gates humaines à traiter par l\'orchestrateur : G2 (schéma) et G3 (parité UI).')
return { audit, db, front, review }
```

C.5 Référentiel Code Health local

templates/code-health/code_health.py (analyse et notation)

```
# ---------------------------------------------------------------------------- analyse

ANALYZER = lizard.FileAnalyzer(lizard.get_extensions(["ns", "cognitive"]))


def threshold(smell: str, lang: str | None) -> dict:
    rule = dict(RULES["smells"][smell])
    rule.update(RULES.get("language_overrides", {}).get(lang or "", {}).get(smell, {}))
    return rule


def blockers_in(rel: str, code: str) -> list[dict]:
    found = []
    for rule in RULES.get("blocker_rules", []):
        if not matches(rel, rule["glob"]):
            continue
        flags = re.M | (re.S if rule.get("multiline") else 0)
        if rule.get("unless") and re.search(rule["unless"], code, flags):
            continue
        if rule.get("type") == "require":
            hits = [] if re.search(rule["pattern"], code, flags) else [1]
        else:
            hits = [code.count("\n", 0, m.start()) + 1 for m in re.finditer(rule["pattern"], code, flags)]
        for line in hits:
            w = waived(rel, rule["id"])
            found.append({"id": rule["id"], "severity": rule["severity"], "category": rule["category"],
                          "title": rule["title"], "location": f"{rel}:{line}", "owner": rule.get("owner"),
                          "advice": rule.get("advice"), "status": "waived" if w else "open"})
    return found


def analyze(rel: str, code: str | None = None) -> dict:
    code = read(rel) if code is None else code
    lang = language(rel)
    info = ANALYZER.analyze_source_code(rel, code)
    smells, functions = [], []
    for f in info.function_list:
        short = f.name.split("::")[-1]
        if short in KEYWORDS:
            continue
        metrics = {"cyclomatic_complexity": f.cyclomatic_complexity, "nloc": f.nloc,
                   "parameter_count": f.parameter_count,
                   "max_nested_structures": getattr(f, "max_nested_structures", 0) or 0,
                   "cognitive_complexity": getattr(f, "cognitive_complexity", 0) or 0}
```

```
        functions.append({"name": f.name, "line": f.start_line, **metrics})
        for key, base in RULES["smells"].items():
            if base.get("scope") != "function":
                continue
            rule, value = threshold(key, lang), metrics.get(base["metric"])
            severity = "critical" if value >= rule["critical"] else "warning" if value >= rule["warning"] else
    ↪ None
            if severity:
                smells.append({"smell": key, "label": rule["label"], "severity": severity, "function": f.name,
                               "line": f.start_line, "metric": base["metric"], "value": value,
                               "threshold": rule[severity], "advice": rule["advice"],
                               "waived": bool(waived(rel, key, f.name))})
    file_rule = threshold("large_file", lang)
    if info.nloc >= file_rule["warning"]:
        severity = "critical" if info.nloc >= file_rule["critical"] else "warning"
        smells.append({"smell": "large_file", "label": file_rule["label"], "severity": severity, "function":
    ↪ None,
                       "line": 1, "metric": "file_nloc", "value": info.nloc, "threshold": file_rule[severity],
                       "advice": file_rule["advice"], "waived": bool(waived(rel, "large_file"))})
    blocks = duplicated_blocks(rel, code)
    if blocks:
        dup = RULES["smells"]["code_duplication"]
        smells.append({"smell": "code_duplication", "label": dup["label"], "severity": "warning", "function":
    ↪ None,
                       "line": blocks[0]["start"], "metric": "duplicated_blocks", "value": len(blocks),
                       "threshold": 1, "advice": dup["advice"], "waived": bool(waived(rel, "code_duplication")),
                       "blocks": blocks})
    return {"file": rel, "language": lang, "nloc": info.nloc, "score": score(smells), "band": None,
            "functions": functions, "smells": smells, "blockers": blockers_in(rel, code)}


def score(smells: list[dict]) -> float:
    penalty = 0.0
    weights = RULES["penalty"]
    for s in smells:
        if s.get("waived"):
            continue
        rule = RULES["smells"][s["smell"]]
        if s["smell"] == "code_duplication":
            penalty += min(rule["max_penalty"], rule["weight"] * s["value"] * weights["warning"])
        else:
            penalty += weights[s["severity"]] * rule["weight"]
    return round(max(1.0, 10.0 - penalty), 1)


def band(value: float) -> str:
    b = RULES["bands"]
    return "healthy" if value >= b["healthy"] else "problematic" if value >= b["problematic"] else "unhealthy"


def full(rel: str, code: str | None = None) -> dict:
    result = analyze(rel, code)
    result["band"] = band(result["score"])
    return result
```

templates/code-health/server.py

```python
#!/usr/bin/env -S uv run --quiet --script
# /// script
# requires-python = ">=3.10"
# dependencies = ["mcp>=2.3,<3", "lizard>=1.17,<2"]
# ///
"""Serveur MCP local « code-health » du référentiel ecom-modernizer (stdio, aucune clé d'API).

Expose le moteur code_health.py avec des noms d'outils compatibles avec ceux du serveur
CodeScene, afin que les prompts et les skills de l'agent CodeScene restent inchangés.
"""
from __future__ import annotations

import json
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent))
import code_health as ch  # noqa: E402
from mcp.server.mcpserver import MCPServer  # noqa: E402

server = MCPServer(
    name="code-health",
    version="1.0.0",
    instructions="Santé du code locale (1-10) selon le référentiel versionné du projet (rules.json). "
                 "Outils en lecture seule ; les chemins sont relatifs à la racine du projet.",
)


def as_json(data: object) -> str:
    return json.dumps(data, ensure_ascii=False, indent=2)


@server.tool()
def code_health_score(file_path: str) -> str:
    """Note de santé (1 à 10) d'un fichier source et sa bande (healthy, problematic, unhealthy)."""
    r = ch.full(ch.rel_path(file_path))
    return f"Code Health de {r['file']} : {r['score']} ({r['band']}) — {len(r['smells'])} smell(s), {r['nloc']}
     ↪ lignes utiles"


@server.tool()
def code_health_review(file_path: str) -> str:
    """Revue détaillée d'un fichier : code smells (métrique, seuil, conseil), duplications et bloqueurs."""
    return as_json(ch.full(ch.rel_path(file_path)))


@server.tool()
def analyze_change_set(base_ref: str = "HEAD") -> str:
    """Compare les fichiers modifiés depuis base_ref (et non suivis) avec leur version de référence ; liste les
     ↪ régressions."""
    return as_json(ch.cmd_delta(base_ref))


@server.tool()
def pre_commit_code_health_safeguard() -> str:
    """Contrôle les fichiers indexés (git add) avant un commit ; pass=false si une régression est détectée."""
    return as_json(ch.cmd_safeguard())


@server.tool()
def project_code_health(path: str = "src") -> str:
    """Synthèse d'un périmètre : moyenne pondérée, minimum, répartition par bande, bloqueurs ouverts."""
    r = ch.project(ch.source_files([path]))
    r["files"] = [{k: f[k] for k in ("file", "score", "band", "nloc")} | {"smells": len(f["smells"])} for f in r
     ↪ ["files"]]
    return as_json(r)
```

```python
@server.tool()
def list_blockers(path: str = "src") -> str:
    """Applique les règles de bloqueurs du référentiel (rules.json, section blocker_rules) à un périmètre."""
    return as_json([b for f in ch.source_files([path]) for b in ch.blockers_in(f, ch.read(f))])


@server.tool()
def list_hotspots(since: str = "6 months", top: int = 10) -> str:
    """Hotspots locaux : fréquence de modification (historique git) croisée avec la santé du code."""
    return as_json(ch.cmd_hotspots(since, top))


@server.tool()
def rules_config_list_thresholds(language: str = "") -> str:
    """Seuils et pondérations des code smells, avec les surcharges éventuelles d'un langage (csharp, tsx...)."""
    smells = {k: ch.threshold(k, language or None) for k in ch.RULES["smells"]}
    return as_json({"bands": ch.RULES["bands"], "penalty": ch.RULES["penalty"], "smells": smells})


@server.tool()
def explain_code_health() -> str:
    """Documentation du modèle de santé du code et du référentiel (REFERENTIEL.md)."""
    doc = Path(__file__).resolve().parent / "REFERENTIEL.md"
    return doc.read_text(encoding="utf-8") if doc.is_file() else "REFERENTIEL.md absent du référentiel."


if __name__ == "__main__":
    server.run("stdio")
```

D | Sources et références

Documentation consultée en octobre 2026.

- Claude Code — sous-agents : `https://code.claude.com/docs/en/sub-agents`
- Claude Code — plugins (manifeste, composants, marketplaces) : `https://code.claude.com/docs/en/plugins-reference`, `https://code.claude.com/docs/en/plugins/components`, `https://code.claude.com/docs/en/plugin-marketplaces`
- Claude Code — hooks : `https://code.claude.com/docs/en/hooks`
- Claude Code — skills : `https://code.claude.com/docs/en/skills`
- Claude Code — workflows dynamiques : `https://code.claude.com/docs/en/workflows`
- Claude Code — agent teams : `https://code.claude.com/docs/en/agent-teams`
- Claude Code — tâches planifiées et `/loop` : `https://code.claude.com/docs/en/scheduled-tasks` ; `/goal` : `https://code.claude.com/docs/en/goal`
- Claude Code — évaluations de plugins : `https://code.claude.com/docs/en/plugin-evals`
- Claude Code — télémétrie : `https://code.claude.com/docs/en/monitoring-usage`
- Claude Code — GitHub Actions : `https://code.claude.com/docs/en/github-actions`
- Model Context Protocol — révision 2026-07-28 : `https://modelcontextprotocol.io/specification/2026-07-28/changelog`
- SDK MCP Python (v2) : `https://py.sdk.modelcontextprotocol.io`
- lizard (analyseur de complexité) : `https://github.com/terryyin/lizard`
- CodeScene MCP Server (référence, non utilisé faute de fonctionnement sans clé) : `https://github.com/codescene-oss/codescene-mcp-server`
- Postgres MCP Pro : `https://github.com/crystaldba/postgres-mcp`
- next-devtools-mcp : `https://github.com/vercel/next-devtools-mcp`
- Playwright MCP : `https://github.com/microsoft/playwright-mcp` ; agents de test : `https://playwright.dev/docs/test-agents`
- kubernetes-mcp-server : `https://github.com/containers/kubernetes-mcp-server`
- GitHub MCP Server : `https://github.com/github/github-mcp-server`
- Grafana MCP : `https://github.com/grafana/mcp-grafana`
- Aspire — intégrations JavaScript et Kubernetes : `https://aspire.dev/integrations/frameworks/javascript/`, `https://aspire.dev/integrations/compute/kubernetes/`
- Aspire — PostgreSQL et EF Core : `https://aspire.dev/integrations/databases/efcore/postgres/postgresql-get-started/`